ARM PrimeCell

Synchronous Serial Port Master and Slave (PL022)

Design Manual

PL022 DDES 0000 A

ARM Confidential

ARM PrimeCell Synchronous Serial Port Master and Slave (PL022) Design Manual

Proprietary notice

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Synchronous Serial Port

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT **1**
1.1.1 Change history 1

1.2 References **1**

1.3 Terms and abbreviations **1**

2 SCOPE **2**

3 INTRODUCTION **2**

4 SSP DESIGN OVERVIEW **3**

4.1 Functional Description **3**

4.2 Signal Description **4**

4.3 Programmer's Model **6**
4.3.1 Functional Mode Registers 6
4.3.2 Test Mode Registers 7
4.3.3 Register Memory Map 8

4.4 Block Partitioning **9**
4.4.1 APB Interface 9
4.4.2 Register Block 11
4.4.3 Clock Pre-scaler 12
4.4.4 Transmit / Receive Control logic 13
4.4.5 Transmit FIFO 15
4.4.6 Receive FIFO 16
4.4.7 Receive Time Out Generation Logic 18
4.4.8 Interrupt Generation Logic 19
4.4.9 DMA Interface 20
4.4.10 Test logic 21
4.4.11 Synchronisers 22
4.4.12 Revision Designator block 23
4.4.13 Clock Ratios 23

5 SSP IMPLEMENTATION DETAILS **24**

5.1 Design Hierarchy **24**

5.2 SSP Top Level Block (Ssp) **24**

5.3 SSP APB Interface (SspApbif) **24**
5.3.1 Write Interface 25
5.3.2 Read Interface 25

5.4 SSP Register Core (SspRegCore) **25**
5.4.1 Synchronisation of the SSPCR0 register to the SSPCLK domain 25

5.4.2 Synchronisation of the SSPCPSR register to the SSPCLK domain 25

5.5 SSP Pre-scale Counter (SspScaleCntr) 26

5.6 Transmit FIFO (SspTxFIFO) 27

5.7 Transmit FIFO control logic (SspTxFCntl) 27

5.8 Transmit FIFO Register File (SspTxRegFile) 28

5.9 SSP Transmit Data Pre-Processor (SspTxLJustify) 29

5.10 Receive FIFO (SspRxFIFO) 29

5.11 Receive FIFO Controller (SspRxFCntl) 30

5.12 Receive FIFO Register File (SspRxRegFile) 30

5.13 Master Transmit / Receive Control state machine (SspMTxRxCntl) 31
5.13.1 Exception Handling 33
5.13.2 Texas Instruments' Synchronous Serial Frame Format 33
5.13.3 Motorola's SPI Frame Format 39
5.13.4 National Microwire Frame Format 43

5.14 Slave Transmit / Receive Control state machine (SspSTxRxCntl) 47
5.14.1 Exception Handling 48
5.14.2 Texas Instruments' Synchronous Serial Frame Format 49
5.14.3 Motorola's SPI Frame Format 51
5.14.4 National Microwire Frame Format 53

5.15 Receive Timeout interrupt generation (SspDataStp) 54

5.16 SSP Interrupt generator (SspIntGen) 55

5.17 DMA Interface (SspDMA) 55

5.18 Synchronisers to SSPCLK domain (SspSynctoSSPCLK) 56

5.19 Synchronisers to PCLK domain (SspSynctoPCLK) 56

5.20 Test logic (SspTest) 57
5.20.1 Integration testing of block inputs 57
5.20.2 Integration testing of block outputs 58

6 SSP FUNCTIONAL VERIFICATION DETAILS 61

6.1 SSP VHDL Verification Testbench 61
6.1.1 Unit Under Test 63
6.1.2 APB Bridge Model 63
6.1.3 Trickbox 63
6.1.4 Protocol Checker 63
6.1.5 APB Multiplexer 64
6.1.6 Test Vectors 64
6.1.7 Generics 64

6.1.8 Running Simulations 65

6.2 SSP Verilog Verification Testbench 66
6.2.1 Unit Under Test 68
6.2.2 APB Bridge Model 68
6.2.3 Trickbox 68
6.2.4 Protocol Checker 69
6.2.5 APB Multiplexer 69
6.2.6 Vector Sets 69
6.2.7 Parameters 69
6.2.8 Running Simulations 70

6.3 SSP Trickbox 71
6.3.1 Trickbox Signal Description 71
6.3.2 Trickbox functional description 72
6.3.3 Trickbox Register Description 74

6.4 Free-running Functional Tests 86
6.4.1 Register Tests 86
6.4.2 Non Back-to-Back Test (Master mode) 87
6.4.3 Data Setup/Hold Test (Master mode) 87
6.4.4 Transmission and Reception Test (Master mode) 87
6.4.5 Continuous mode Tests (Master Mode) 87
6.4.6 Interrupt Tests 87
6.4.7 FIFO pointer test (Master mode) 88
6.4.8 FIFO pointer cross-over Tests 88
6.4.9 Receive Overrun Interrupt Tests (Master mode) 88
6.4.10 Scan Mode Test 88
6.4.11 Loop back Test 89
6.4.12 SSP Disable Test (Master mode) 89
6.4.13 Non Back-to-Back Test (Slave mode) 89
6.4.14 Data Setup/Hold Test (Slave mode) 89
6.4.15 Transmission and Reception Test (Slave mode) 89
6.4.16 Continuous mode Tests (Slave Mode) 90
6.4.17 Receive Overrun Interrupt Tests (Slave mode) 90
6.4.18 FIFO pointer test (Slave mode) 90
6.4.19 SOD Test 90
6.4.20 SOD Continuous Test 90
6.4.21 Extended SSPFSSIN test (TI mode only) 90
6.4.22 SSP Disable Test (Slave mode) 91
6.4.23 Free-running-SSPCLKIN Tests 91
6.4.24 Master Slave Test 91
6.4.25 Master Slave SOD Test 91
6.4.26 Frame Deactivation Test 91
6.4.27 SSPFSSIN negation Tests 92
6.4.28 Data Discard Tests 92
6.4.29 FRF change Tests 92
6.4.30 Receive Timeout interrupt tests 92
6.4.31 DMA Interface Transmit tests 92
6.4.32 DMA Interface Receive tests 93
6.4.33 Additional Code Coverage Tests 93

7 SSP INTEGRATION TEST DETAILS 94

7.1 SSP VHDL Integration testbench 94

7.1.1 Unit Under Test 96
7.1.2 Test Vectors 96
7.1.3 Running Simulations 96

7.2 SSP Verilog Integration testbench 97
7.2.1 Unit Under Test 99
7.2.2 Test Vectors 99
7.2.3 Running Simulations 99

7.3 Integration Tests 99
7.3.1 Integration Testing of Intra-Chip and Primary Inputs 99
7.3.2 Integration Testing of Intra-Chip and Primary Outputs 99

1 ABOUT THIS DOCUMENT

1.1.1 Change history

Issue	Date	Change
A	20th March, 2001	First Release.

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM DDI 0194	ARM PrimeCell Synchronous Serial Port Master and Slave (PL022) Technical Reference Manual
2	PL022 INTM 0000	ARM PrimeCell Synchronous Serial Port Master and Slave (PL022) Integration Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 00	Example AMBA System User Guide
6	ARM SCB00 QPRO 0008	PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application Specific Integrated Circuit
FIFO	First-In-First-Out buffer
HDL	Hardware Description Language
SPI	Serial Peripheral Interface
SSP	Synchronous Serial interface Port Master and Slave
TI	Texas Instruments
TIC	Test Interface Controller
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell SSP (Synchronous Serial interface Port Master and Slave). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification are described in Section 6.

Section 4 presents an overview of the SSP design, describing the functional partitioning of the SSP and the signal interconnections. Section 5 presents detailed descriptions on the SSP design with simulation waveforms.

3 INTRODUCTION

The SSP is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell Synchronous Serial Port Master and Slave (PL022) Technical Reference Manual [Ref.1]. The ARM PrimeCell Synchronous Serial Port Master and Slave (PL022) Integration Manual [Ref.2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for system integration test.

4 SSP DESIGN OVERVIEW

The ARM PrimeCell SSP is an APB slave block that enables synchronous serial communication with peripheral devices that have either a Motorola SPI compatible interface, Texas Instruments Synchronous Serial interface or National Semiconductor Microwire interface. The SSP can behave as a master or a slave interface and performs parallel-to-serial conversion on data written to an internal 16-bit wide and 8-deep transmit FIFO. It also performs serial-to-parallel conversion on the serial input data and buffers it in a receive FIFO which is also 16 bits wide and 8-deep. The SSP block asserts interrupts to request servicing of the transmit and receive buffers, to indicate an overrun or receive timeout condition in the receive FIFO.

4.1 Functional Description

A block diagram of the ARM PrimeCell SSP (PL022) is shown in ***Figure 4.1-1***.

The APB interface generates decoded write signals for registers in the device. The Register Block contains the registers in the SSP. The control information written into the SSPCR0 and the SSPCPSR registers is synchronised to the SSPCLK clock domain through synchronisation control logic located in the Register block and in the Clock Pre-Scaler.

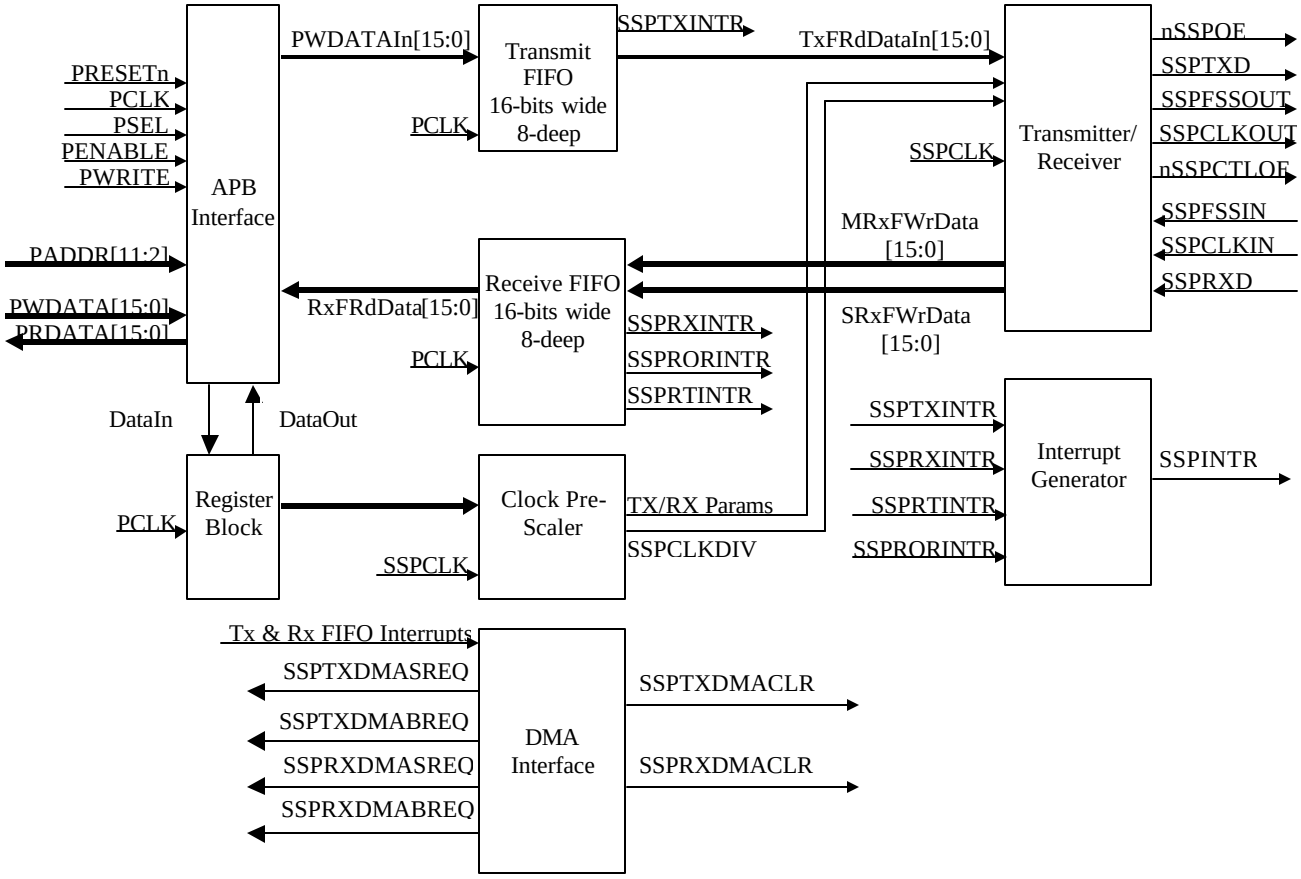

Figure 4.1-1 SSP Block Diagram

When the MS bit in the SSPCR1 register is cleared, the SSP behaves as a master, when set, the SSP behaves as a slave. In the Slave mode, the SSP acts as a receive-only slave when the SOD bit in the SSPCR1 register is set, when cleared it can also transmit. In the master mode, SSPCLK is first divided by the programmed pre-scale value and then by the SCR value to generate the SSPCLKOUT signal. In the slave mode, the slave interface makes use of the SSPFSSIN and SSPCLKIN signals from the external master. The Transmitter, in accordance with the transmission parameters programmed into the Control registers, converts data written in to the Transmit FIFO into a serial bit stream on the SSPTXD output line. The Receiver performs serial-to-parallel conversion on data received on the SSPRXD line. Received data is buffered in the 16-bit wide 8-deep receive FIFO. The Transmit FIFO and the Receive FIFO are 16-bits wide and 8-deep. They generate the service requests, SSPTXINTR and SSPRXINTR. The Receive FIFO also generates the Receive Overrun interrupt signal, SSPRORINTR and the Receive Timeout Interrupt signal, SSPRTINTR. The Interrupt generation block uses the SSPTXINTR, SSPRXINTR, SSPRORINTR and SSPRTINTR signals to generate the combined interrupt, SSPINTR.

4.2 Signal Description

The following tables summarise the SSP signal list. ***Table 4.2-1*** lists the APB interface signals.

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Bus Clock. The frequency of this clock should be equal to or greater than that of the main SSP clock, SSPCLK.
PRESETn	IN	Reset Controller	Bus reset input signal (active low).
PADDR[11:2]	IN	APB Bridge	This is part of the peripheral address bus, which is used for decoding the internal address space.
PWDATA[15:0]	IN	APB Bridge	Data is written into the peripheral on this bus.
PRDATA[15:0]	OUT	APB Bridge	Data is read out of the peripheral via this bus.
PENABLE	IN	APB Bridge	APB enable signal, asserted HIGH for one cycle of PCLK to enable a bus transfer.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the peripheral has been selected by the APB Bridge.

Table 4.2-1: APB signals

Table 4.2-2 lists the block-specific signals.

Signal Name	Type	Source / Destination	Description
SSPCLK	IN	Clock Generator	SSP clock input
nSSPRST	IN	Reset Controller	SSP reset signal to SSPCLK clock domain, active LOW. The reset controller must use PRESETn to assert nSSPRST asynchronously but negate it synchronously with SSPCLK.
SSPTXINTR	OUT	Interrupt Controller	Transmit FIFO Service Request
SSPRXINTR	OUT	Interrupt Controller	Receive FIFO Service Request
SSPRORINTR	OUT	Interrupt Controller	SSP Receive Overrun Interrupt
SSPRTINTR	OUT	Interrupt Controller	SSP Receive Timeout Interrupt
SSPINTR	OUT	Interrupt Controller	SSP Interrupt. This interrupt is the OR of the four individual interrupts SSPTXINTR, SSPRXINTR, SSPRORINTR and SSPRTINTR.
SSPTXDMACLR	IN	DMA Controller	TX DMA Clear
SSPRXDMACLR	IN	DMA Controller	RX DMA Clear
SSPTXDMASREQ	OUT	DMA Controller	TX DMA Single Request
SSPTXDMABREQ	OUT	DMA Controller	TX DMA Burst Request
SSPRXDMASREQ	OUT	DMA Controller	RX DMA Single Request
SSPRXDMABREQ	OUT	DMA Controller	RX DMA Burst Request
SCANINPCLK	IN	Test Controller	Place holder for Scan data input signal (PCLK domain)
SCANOUTPCLK	OUT	Test Controller	Place holder for Scan data output signal (PCLK domain)
SCANINSSPCLK	IN	Test Controller	Place holder for Scan data input signal (SSPCLK domain)
SCANOUTSSPCLK	OUT	Test Controller	Place holder for Scan data output signal (SSPCLK domain)
SCANENABLE	IN	Test Controller	Place holder for Scan path select signal
SSPFSSOUT	OUT	PAD	SSP Serial frame output (master)
SSPCLKOUT	OUT	PAD	SSP Serial clock output (master)
SSPRXD	IN	PAD	SSP Serial data input
SSPTXD	OUT	PAD	SSP Serial data output
nSSPCTLOE	OUT	PAD	Output Enable signal for SSPCLKOUT and SSPFSSOUT outputs from the SSP. This output is set when the device is in Master mode and cleared when the device is in Slave mode.
SSPFSSIN	IN	PAD	SSP serial frame or slave select input
SSPCLKIN	IN	PAD	SSP serial clock input (slave)
nSSPOE	OUT	PAD	Output Enable signal to indicate when SSPTXD is valid.

Table 4.2-2 Block-specific signal descriptions

4.3 Programmer's Model

The SSP registers are shown in Table 4.3-1 and Table 4.3-2. The base address is implementation dependent. The registers located at offset 0x80 to 0x8C are reserved for test purposes and should not be used during normal operation.

4.3.1 Functional Mode Registers

Address	Type	Width	Reset Value	Name	Description
SspBase + 0x00	R/W	16	0x0000	**SSPCR0**	Control Register 0.
SspBase + 0x04	R/W	4	0x0	**SSPCR1**	Control Register 1.
SspBase + 0x08	R /W	16	0x--	**SSPDR**	Data Register. Writes to this register will load the Transmit FIFO. Reads from this register will return data from the Receive FIFO.
SspBase + 0x0C	R	5	0x03	**SSPSR**	Status Register. Reads from this register return SSP status.
SspBase + 0x10	R/W	8	0x00	**SSPCPSR**	SSP Clock Pre-scale Register. This register specifies the factor by which SSPCLK is to be divided in the Clock pre-scaler. The division factor has to be an even number from 2 to 254.
SspBase + 0x14	R /W	4	0x0	**SSPIMSC**	Interrupt Mask Set and Clear register.
SspBase + 0x18	R	4	0x0	**SSPRIS**	Raw Interrupt Status register.
SspBase + 0x1C	R	4	0x0	**SSPMIS**	Masked Interrupt Status register.
SspBase + 0x20	W	2	0x0	**SSPICR**	Interrupt Clear register.
SspBase + 0x24	R/W	2	0x0	**SSPDMACLR**	DMA Control register.
SspBase + 0x28-0x7C					Reserved
SspBase + 0x80-0x8C					Reserved (for test purposes)
SspBase + 0x90-0xFCC					Reserved
SspBase + 0xFD0-0xFDC					Reserved for future expansion
SspBase + 0xFE0	R	8	0x22	**SSPPeriphID0**	Peripheral identification register bits 7:0

SspBase + 0xFE4	R	8	0x10	**SSPPeriphID1**	Peripheral identification register bits 15:8
SspBase + 0xFE8	R	8	0x04	**SSPPeriphID2**	Peripheral identification register bits 23:16
SspBase + 0xFEC	R	8	0x00	**SSPPeriphID3**	Peripheral identification register bits 32:24
SspBase + 0xFF0	R	8	0x0D	**SSPCellID0**	PrimeCell identification register bits 7:0
SspBase + 0xFF4	R	8	0xF0	**SSPCellID1**	PrimeCell identification register bits 15:8
SspBase + 0xFF8	R	8	0x05	**SSPCellID2**	PrimeCell identification register bits 23:16
SspBase + 0xFFC	R	8	0xB1	**SSPCellID3**	PrimeCell identification register bits 32:24

Table 4.3-1 Functional Mode Registers

4.3.2 Test Mode Registers

Address	Type	Width	Reset Value	Name	Description
SspBase + 0x80	R/W	2	0x0	**SSPTCR**	SSP Test Control Register.
SspBase + 0x84	R/W	5	0x00	**SSPITIP**	Integration test input register.
SspBase + 0x88	R/W	14	0x0000	**SSPITOP**	Integration test output register.
SspBase + 0x8C	R	16	0x0000	**SSPTDR**	SSP Test Data Register.

Table 4.3-2 Test Mode Registers

4.3.3 Register Memory Map

Address	Read Location	Write Location
SSP Base + 0x00	SSPCR0	SSPCR0
SSP Base + 0x04	SSPCR1	SSPCR1
SSP Base + 0x08	SSPDR	SSPDR
SSP Base + 0x0C	SSPSR	-
SSP Base + 0x10	SSPCPSR	SSPCPSR
SSP Base + 0x14	SSPIMSC	SSPIMSC
SSP Base + 0x18	SSPRIS	-
SSP Base + 0x1C	SSPMIS	-
SSP Base + 0x20	-	SSPICR
SSP Base + 0x24	SSPDMACR	SSPDMACR
SSP Base + 0x28 – 0x7C	Reserved	Reserved
SSP Base + 0x80	SSPTCR (Test Mode Only)	SSPTCR (Test Mode Only)
SSP Base + 0x84	SSPITIP (Test Mode Only)	SSPITIP (Test Mode Only)
SSP Base + 0x88	SSPITOP (Test Mode Only)	SSPITOP (Test Mode Only)
SSP Base + 0x8C	SSPDR (Test Mode Only)	SSPDR (Test Mode Only)
SSP Base + 0x90 – 0xFCC	Reserved	Reserved
SSP Base + 0xFD0 – 0xFDC	Reserved	Reserved
SSP Base + 0xFE0	SSPPeriphID0	-
SSP Base + 0xFE4	SSPPeriphID1	-
SSP Base + 0xFE8	SSPPeriphID2	-
SSP Base + 0xFEC	SSPPeriphID3	-
SSP Base + 0xFF0	SSPCellID0	-
SSP Base + 0xFF4	SSPCellID1	-
SSP Base + 0xFF8	SSPCellID2	-
SSP Base + 0xFFC	SSPCellID3	-

Table 4.3-3 SSP Register map

4.4 Block Partitioning

The sub-blocks in the design are:

- AMBA APB Interface (Input module and Output module)
- Register block
- Clock Pre-scaler
- Transmit / Receive Control logic (Master and Slave modes)
- Transmit FIFO
- Receive FIFO
- Receive Timeout Interrupt Generation Logic
- Interrupt generation logic
- DMA interface
- Test logic
- Synchronising registers and logic
- Revision Designator Logic block

The design of these sub-blocks is detailed in the subsequent sections.

4.4.1 APB Interface

The APB interface comprises an input and output module. This interface deals with reads and writes to registers from the APB. A simplified block diagram of the APB interface block is shown ***Figure 4.4-1***.

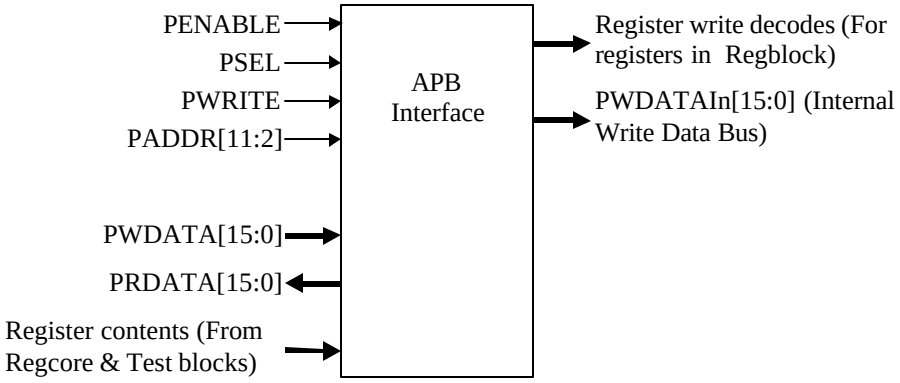

Figure 4.4-1 APB Interface Block diagram

Input Module

This section of the APB write interface generates decodes for writes to all registers in the device. Writes to the SSPDR register result in writes to the Transmit FIFO. Reads from the SSPDR register return data from the Receive FIFO.

Each register is clocked by PCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial address decode and PENABLE, PSEL and PWRITE signals.

The intra-chip inputs SSPTXDMACLR and SSPRXDMACLR are multiplexed with their corresponding test register bits in the SspTest block. The block diagram in ***Figure 4.4-2*** illustrates the structure of the input module. In this

block diagram, the write decodes are generated in the APB interface block, while the registers are actually located in the register block, SspRegCore and in the Test block, SspTest.

Since SSPCLK is the main clock for the SSP core logic, control register bits are synchronised to SSPCLK before use in the SSP core.

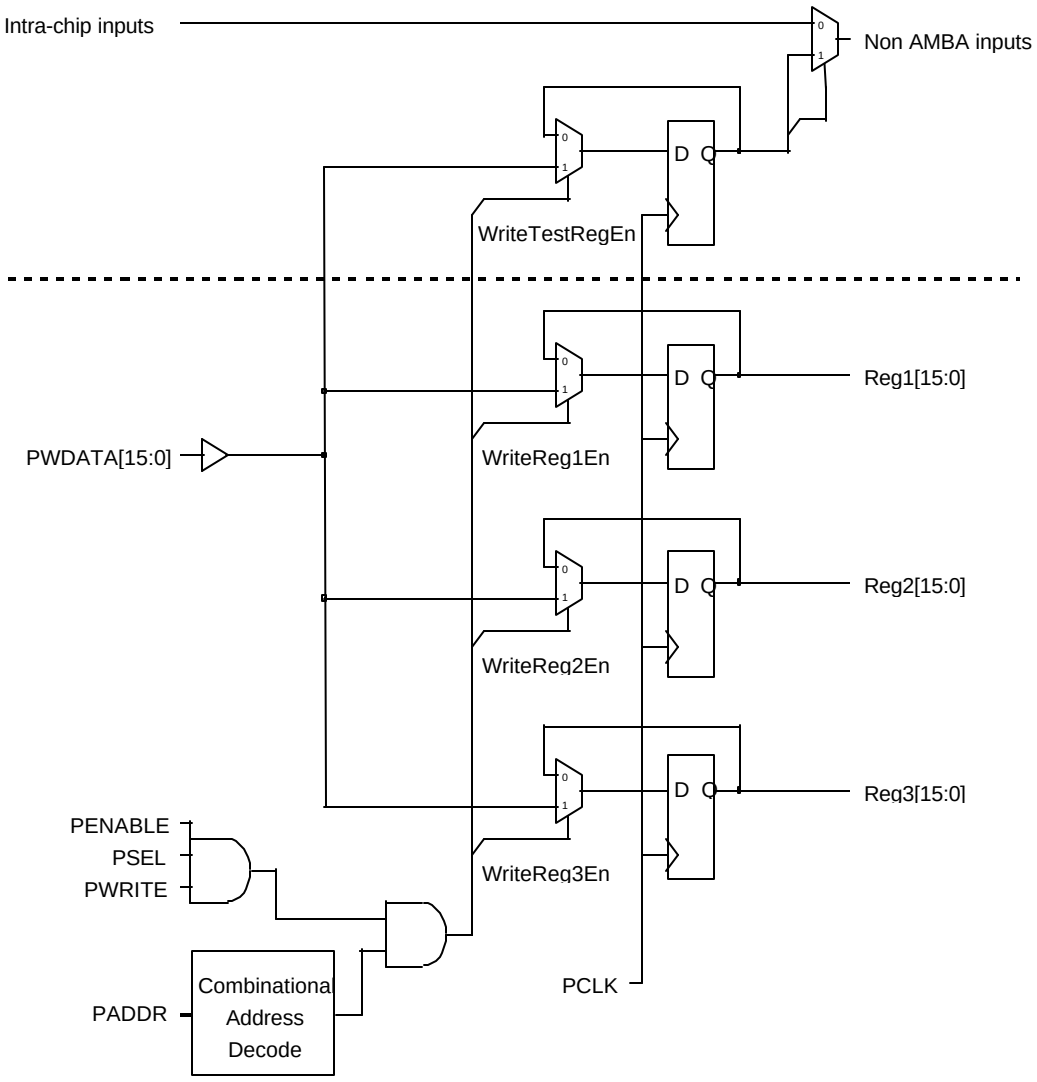

Figure 4.4-2 Input Module

Output module

The output module in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read databus. The output of the multiplexer is fed to a bank of flip-flops clocked by the system PCLK. A combinatorial address decode is used to select which of the output sources are driven onto the bus. The non-AMBA outputs, SSPTXINTR, SSPRXINTR, SSPRTINTR and SSPRORINTR are also made observable through APB read accesses to interrupt status registers. The default state of the output multiplexer is such that zeros are driven on the data bus. This ensures that there are no restrictions placed on the type of external bus connection method used for the data bus on the APB. The external data buses of the peripherals on the APB may then be connected to the ASB-to-APB bridge using the Muxed or the ORed bus connection method. A block diagram of the output module is shown in ***Figure 4.4-3***. The registers shown in this block diagram are

actually located in the register block and in the test block, while the output multiplexer and the final data register on the output of the multiplexer are located in the APB interface block.

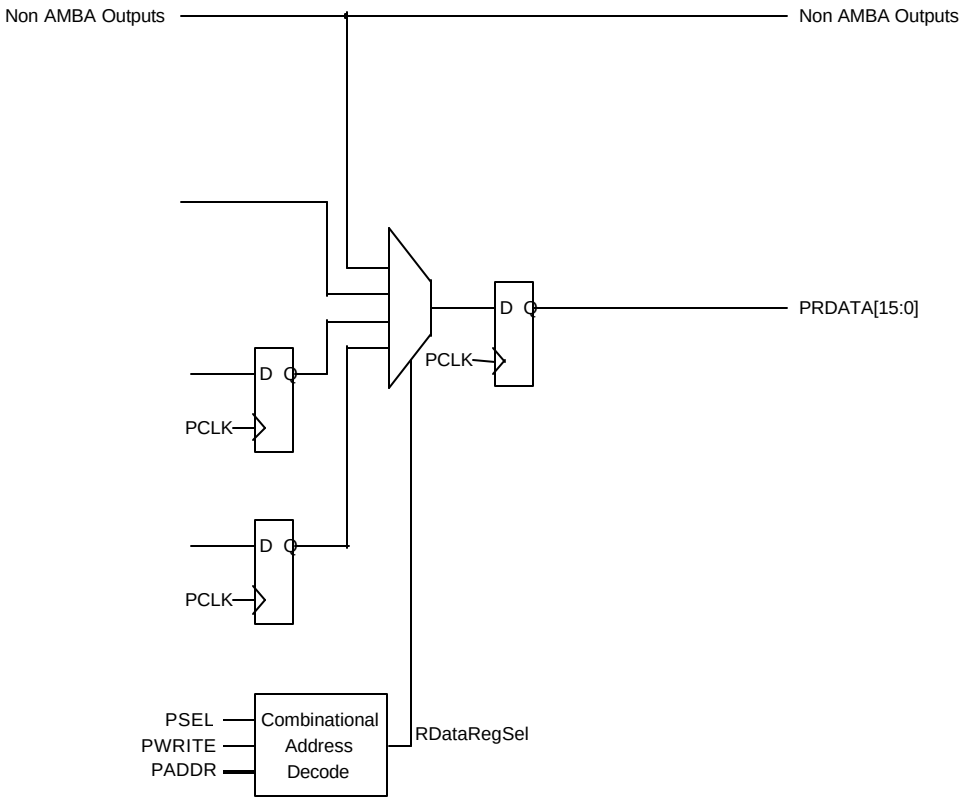

Figure 4.4-3 Output module

4.4.2 Register Block

Besides the normal mode registers themselves, this block also contains the control logic required to synchronise the contents of the SSP Control Register 0 (SSPCR0) and the SSP Clock Prescale Register (SSPCPSR) to the SSPCLK domain. The test registers are located in the SspTest block.

Classical double-synchronisation has not been used to synchronise the control register bits to the SSPCLK domain. This is because the control registers contain multiple-bit fields like Data Size Select (DSS[3:0]) and Serial Clock Rate (SCR[7:0]). If double synchronisation is used, then some bits in these fields could get synchronised one SSPCLK later than other bits. Thus, there could be a one-SSPCLK window during which the bit fields are incorrect. The consequences could be undesirable as in the case of, for example, the wrong SCR value getting reloaded into a counter which would get corrected, in the worst case, only after 256 SSPCLKDIV periods.

A two-buffer technique is used to synchronise the contents of the SSP control register 0 (SSPCR0) to the SSPCLK domain. A separate 2-buffer technique is used to synchronise the contents of the SSPCPSR register to the SSPCLK domain.

In this 2-buffer technique, writes from the APB to the SSPCR0 register, go to the first stage buffers. When there is a write to SSPCR0, a control signal toggles. This signal is double-synchronised to the SSPCLK domain. An edge on the synchronised signal is detected by generating a delayed version and XOR-ing the synchronised signal with its delayed version. Thus, a one-SSPCLK-wide load pulse is generated with every write to the SSPCR0 register. When this load signal is asserted, the contents of the first stage SSPCR0 buffer are copied into the second stage buffer in the SSPCLK domain. Software is not expected to write to the SSPCR0 register during this synchronisation period and hence the contents of the first stage buffer will be stable throughout the

synchronisation period. This is a reasonable assumption since these registers are control registers and are not expected to be written to frequently. Metastability on the second stage is eliminated.

Figure 4.4-4 illustrates the 2-buffer mechanism.

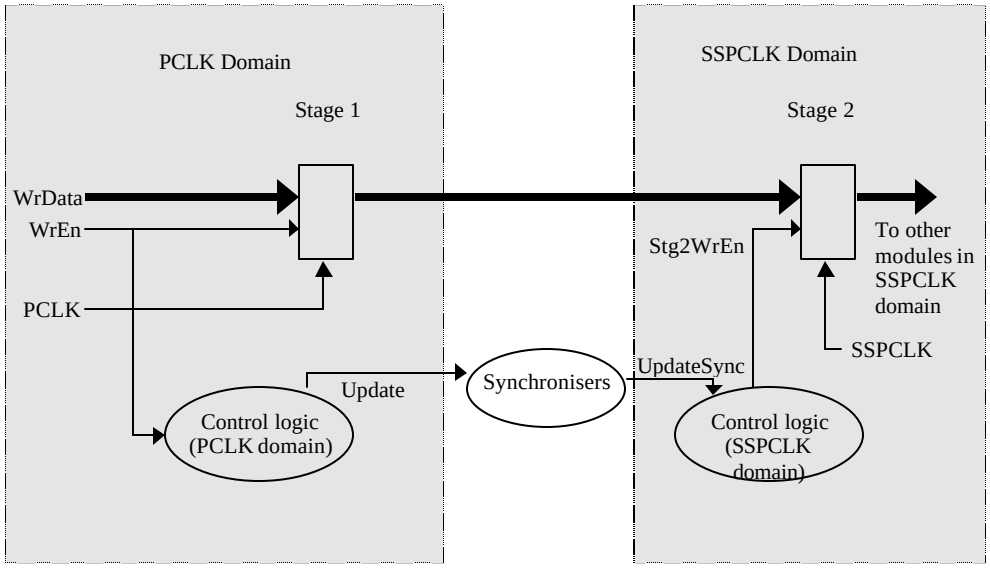

Figure 4.4-4 Two-buffer synchronisation mechanism for the SSPCR0 register and the SSPCPSR register

Another identical set of buffers and control logic is used to synchronise the contents of the SSPCPSR register to the SSPCLK domain.

4.4.3 Clock Pre-scaler

The Pre-scaler contains a 7-bit free-running counter clocked by SSPCLK. This counter is used to generate the SSPCLKDIV signal by dividing the SSPCLK signal by half the value programmed into the SSPCPSR register. The remaining divide-by-2 is done in the Transmit/Receive control state machine. This has been done to ensure that the Transmit/Receive control section has to divide by an even number. Even if the programmed SCR value is even, thereby requiring a division by (SCR +1) which would be an odd number, the additional factor-of-2 division leads to a situation where the division to be performed by the Transmit/Receive section is by an even number. The pre-scale counter is loaded with all-zeros on reset. When the SSP is disabled, the counter is loaded with half the programmed reload value. After the SSP is enabled, the counter counts down with every SSPCLK. When the counter reaches one, a one SSPCLK-wide pulse is created on the SSPCLKDIV signal and the counter is reloaded with half the programmed reload value. The SSPCLKDIV signal is not used as an actual clock signal in the SSP. It is used as an enable signal to other logic, which are clocked by the main SSP clock, SSPCLK.

This block also contains the SSPCLK-domain part of the control logic used to synchronise the contents of the SSPCR0 and SSPCPSR registers to the SSPCLK domain.

The DSS[3:0], FRF[1:0], SPO, SPH and SCR[7:0] outputs from this block are the SSPCLK-synchronised versions of the corresponding bits in the SSPCR0 register. These are connected to the Transmit / Receive block for conveying transmission/reception parameter information.

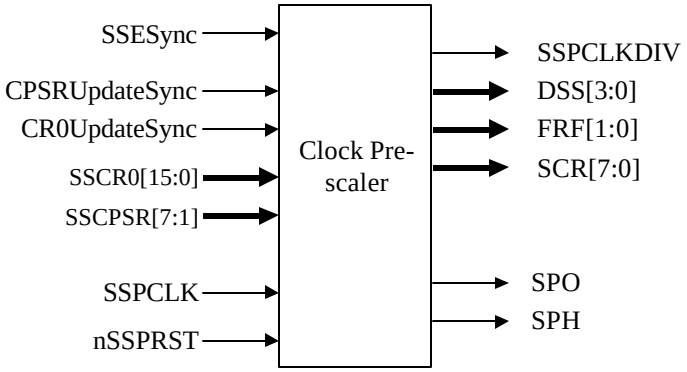

Figure 4.4-5 Clock Pre-scaler

4.4.4 Transmit / Receive Control logic

Due to the synchronous full-duplex nature of two (TI Synchronous Serial Frame Format and Motorola's SPI Frame Format) of the three protocols, the transmit and receive logic have been implemented as a single state machine each in master and slave modes. The Transmit/Receive Control logic converts the parallel transmit data into a serial bit stream with the programmed bit rate and shifts in received data into the receive shift register.

In the master mode, when the TxDataAvlblSync input signal is asserted, the control logic copies the Transmit data being driven on the TxFRdDataIn[15:0] bus into an internal shift register and toggles the TxFRdPtrInc output signal. This signal is synchronised to the PCLK domain and is used by the Transmit FIFO to increment the read pointer. To ensure that the synchronisation is completed before one frame duration, there is a restriction on the relation between the frequencies of PCLK and SSPCLK. It is assumed that PCLK runs at a frequency that is equal to or greater than SSPCLK. The SSP drives the SSPCLKOUT, SSPFSSOUT, nSSPOE, nSSPCTLOE and SSPTXD lines in accordance with the transmission parameters specified in the Data Size Select (DSS[3:0]), Frame Format (FRF[1:0]) and the Serial Clock Rate (SCR[7:0]) bits and the transmit data on the TxFRdDataIn[15:0] input. To account for pad delays, the nSSPOE signal is asserted one SSPCLKOUT phase time before and after SSPTXD is supposed to be valid. The output enable signal, nSSPCTLOE, for SSPFSSOUT and SSPCLKOUT, remains set in Master mode and is cleared in slave mode.

In slave mode when FSSINSync is asserted, the start of a new frame is detected. The SSP then transmits / receives at the positive / negative edge of the CLKINSync signal.

As transmission is in progress, the control logic performs serial-to-parallel conversion on the serial data stream received on the SSPRXD input pin. It is assumed that the SSPRXD line has valid data with sufficient setup and hold times with respect to the SSPCLKIN / SSPCLKOUT edge on which data is to be sampled. When the programmed number of bits have been shifted in, the control logic copies the contents of the Receive Shift register into the Receive buffer and drives the contents of the Receive buffer on the MRxFWrData[15:0] / SRxFWrData[15:0] lines. It also toggles the RxFWr signal. The RxFWr signal indicates to the Receive FIFO that valid data is available on the MRxFWrData[15:0] / SRxFWrData[15:0] lines to be clocked into the FIFO. This signal is double synchronised to the PCLK domain and fed to the Receive FIFO. To ensure synchronisation to the PCLK domain before one frame duration, the RxFWr signal is held at the same level for atleast 4 bit periods (the minimum number of bits per received frame is 4). Since PCLK is assumed to run at a frequency that is equal to or greater than that of SSPCLK, the synchronisation of the RxFWr signal to the PCLK domain within this period is assured.

In master mode, the SSPCLKDIV input signal is divided in this block by twice the value specified by the Serial Clock Rate bits, to generate the SSPCLKOUT signal. As mentioned before in ***Section 4.4-3***, this is done so that the division factor is an even number. This is adjusted against the division factor used in the clock prescaler, so that the overall division is by the specified factor of (CPSDVSR x (SCR + 1)), where CPSDVSR is the value written into the SSPCPSR register.

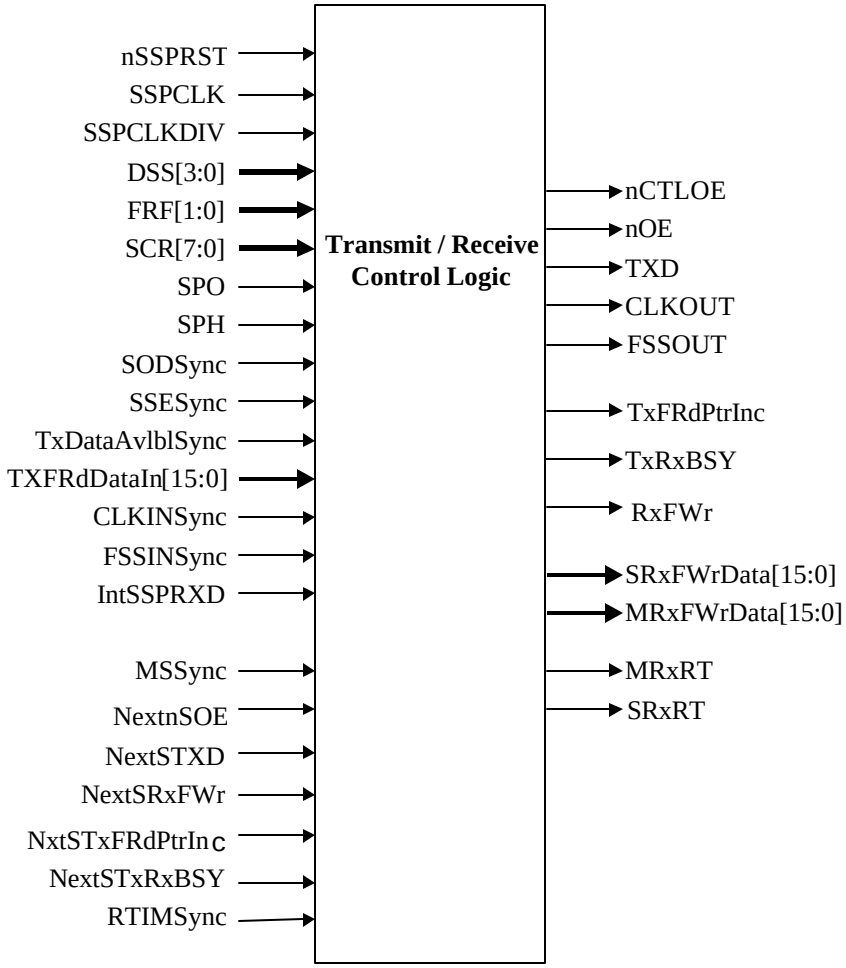

Figure 4.4-6 SSP Transmit / Receive Control Logic

In the master mode, when a complete frame has been transmitted and the TxDataAvlblSync signal continues to remain asserted, indicating the existence of transmit data, the sequence is repeated for the next data value. In the case of slave mode, data is expected to be present in the Transmit FIFO for the slave to transmit when FSSINSync is asserted for successive frames.

When the SSESync input signal is negated, transmission is disabled. This signal is the SSPCLK-synchronised version of the SSE bit in the SSPCR1 control register.

When the Transmit / Receive section is busy transmitting/receiving, the TxRxBSY output signal is asserted. This signal is synchronous to SSPCLK. After synchronisation to the PCLK domain this signal is used in the Transmit FIFO block to generate the BSY signal, which is reflected in the BSY bit of the SSP Status Register (SSPSR).

In the master mode, the CLKOUT signal activity is monitored and if held inactive for more than thirty two bit periods whilst data is still present it the receive FIFO, then the receive timeout interrupt , SSPRTINTR, is asserted. In a similar manner, when in slave mode the SSPCLKIN signal activity is monitored and if inactive for more than thirty two bit periods whilst data is still present in the receive FIFO, then the receive timeout interrupt, SSPRTINTR, is asserted. The master MRxRT and slave SRxRT signals are asserted whenever activity is detected in the respective mode of operation, which in turn causes the timeout down counter to be reloaded with it's maximum value.

4.4.5 Transmit FIFO

The Transmit FIFO consists of an 8-deep 16-bit wide circular buffer. Reads and writes to the FIFO result in accesses to buffer locations addressed by a read pointer and a write pointer respectively. The pointer values are compared to deduce the FIFO fill level. Transmit FIFO initiates the assertion of a service request signal, SSPTXINTR, when the fill level in the FIFO has become equal to or less than four.

Writes to the Transmit FIFO occur through the APB interface in the PCLK domain. The SSPDRWr signal is a decode of writes to the SSPDR register. When the write enable signal SSPDRWr is asserted, data present on the PWDATAIn[15:0] bus is written, on the rising edge of PCLK, into the FIFO location pointed to by the current value of the write pointer. At the end of every FIFO write, the write pointer is incremented.

Both the read pointer and the write pointer are clocked on PCLK. Reads from the transmit FIFO are done by the Transmit section, which runs on SSPCLK. The Transmit FIFO left-justifies the transmit data and always drives the TxFRdDataIn[15:0] bus with the contents of the FIFO location pointed to by the current value of the read pointer. If the FIFO is non-empty, then this is indicated by the assertion of the TxDataAvlbl signal. After the Transmit section has copied the data driven on the TxFRdDataIn[15:0] bus into the internal Transmit Shift Register, the TxFRdPtrInc signal is asserted. This signal is synchronised to the PCLK domain and is used to increment the read pointer.

The control logic inhibits writes to the FIFO when it is full and reads from the FIFO when it is empty. Besides, it also generates status signals for the FIFO.

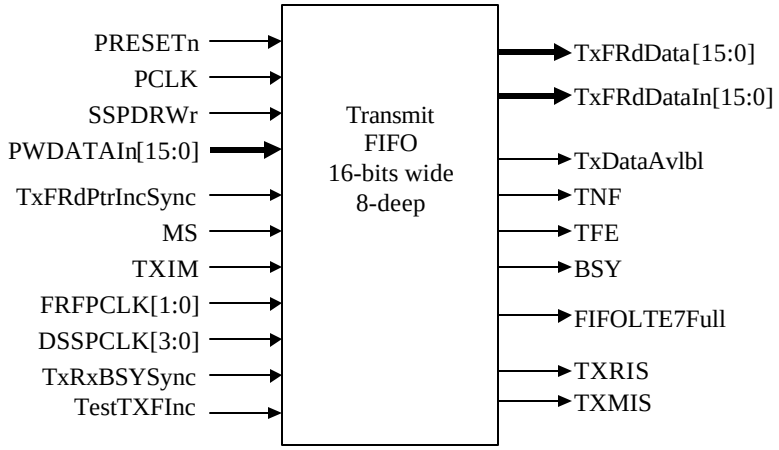

Figure 4.4-7 Transmit FIFO

The interface between the FIFO control logic and the actual data buffer is shown in ***figure 4.4-8***. The write address WrPtr[2:0] and the read address RdPtr[2:0] are driven with the current values of the write pointer and the read pointer respectively. The RegFileWrEn signal is the same as the write enable input, SSPDRWr, to the FIFO, qualified with the FIFO fill status to prevent writes to the transmit FIFO if it is already full. PCLK is used to clock the storage elements in the register file. The Register bank includes the output multiplexer for reads. This multiplexer uses the RdPtr[2:0] bus to select the buffer location whose contents have to be driven out onto the TxFRdData[15:0] bus.

The transmit FIFO raw interrupt service request (TXRIS) is generated when the Transmit FIFO Interrupt FIFO is less than or equal to half full. This request can be masked by the transmit interrupt mask signal (TXIM) to produce the masked interrupt signal (TXMIS) signal. The raw interrupt is dynamically cleared when the condition that caused the interrupt is no longer present.

The TNF (Transmit FIFO Not Full) signal is readable through the SSPSR register. When asserted, it indicates that the transmit FIFO has seven or fewer entries. The TFE (Transmit FIFO Empty) signal is also readable through the SSPSR register. When asserted, it indicates that the Transmit FIFO does not have any valid entries.

The FIFOLTE7Full signal is asserted when there is at least one free location within the transmit FIFO and is used within the DMA interface.

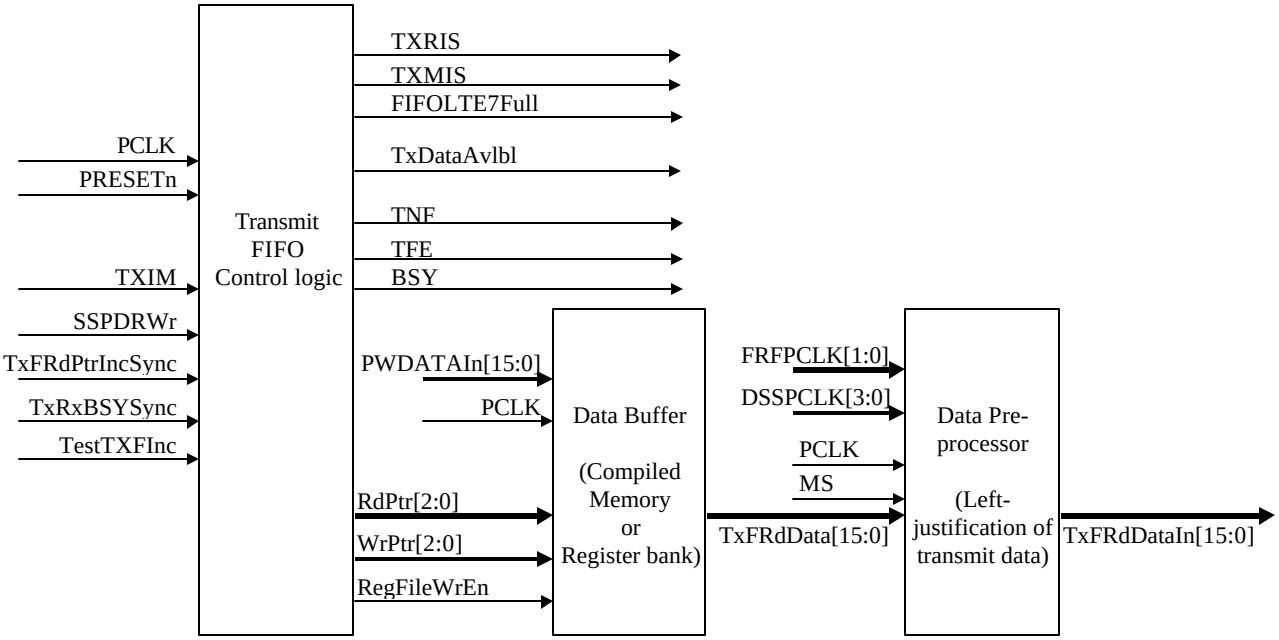

Figure 4.4-8 Transmit FIFO control logic interaction with Data buffer

4.4.6 Receive FIFO

The receive FIFO is similar to the transmit FIFO in most respects. The receive FIFO is 16 bits wide and 8-deep. Writes to the receive FIFO occur from the receive section. Reads from the receive FIFO occur through the APB interface.

For the purpose of evaluating the FIFOFillLevel, the read pointer and the write pointer operate on the same clock PCLK domain. Operating them on SSPCLK would mean data cannot be served consistently onto the APB since two successive reads from the APB could occur with a separation of just one PCLK period (and the synchronisation of the read pointer increment signal in itself would consume two PCLK cycles). Hence both the read pointer and the write pointer operate on PCLK.

On receiving a full data value, the Receiver section drives the received data onto the MRxFWrData[15:0] bus (master mode) and SRxFWrData[15:0] (Slave mode). Also, the write enable signal synchronous to SSPCLK, is toggled. This signal is synchronised to PCLK and is seen as the RxFWrSync input of the receive FIFO. When an edge is detected on the RxFWrSync signal, data on the MRxFWrData[15:0] / SRxFWrData[15:0] bus is clocked into the FIFO location pointed to by the current value of the write pointer. The selection between MRxFWrData[15:0] and SRxFWrData[15:0] is done based on the MS (Master/Slave select) input signal. After every write, the write pointer is incremented.

On the read side, the contents of the FIFO element selected by the current value of the read pointer is always driven on the RxFRdData[15:0] bus. The RxFRdPtrInc signal which indicates that the read pointer must be incremented, is derived from a decode of APB read accesses to the SSPDR register.

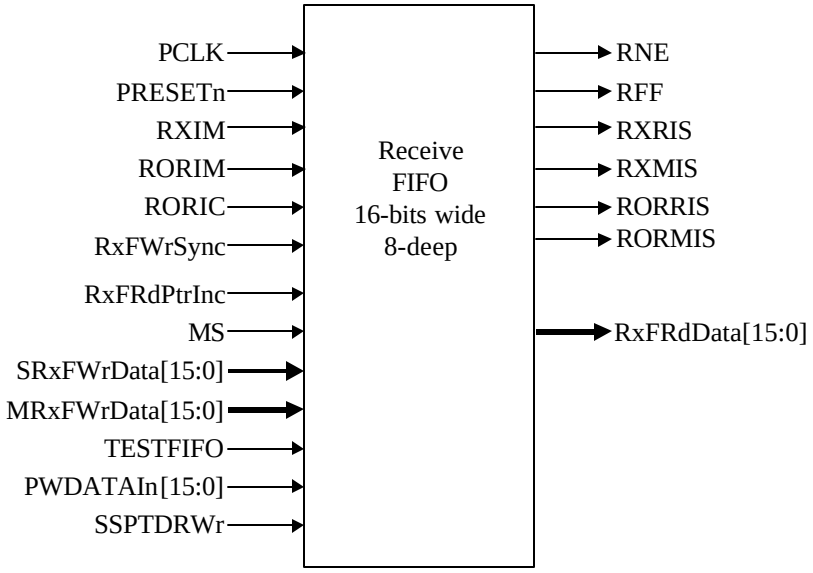

Figure 4.4-9 Receive FIFO

The receive FIFO controller asserts the receive overrun raw interrupt signal RORIS when the FIFO is already full and an additional data frame is received, thereby resulting in an overrun of the FIFO. In this case, the data that caused the overrun, is discarded. The RORIS can be masked by the RORIM signal. The final external SSPRORINTR signal is readable as the RORIS bit in the raw interrupt status register, SSPRIS or the masked version in the masked interrupt status register, SSPMIS. The Overrun condition is cleared when the RORIC signal is asserted.

The RNE (Receive FIFO Not Empty) signal is asserted when there is at least one valid entry in the Receive FIFO. This signal is readable through the SSPSR (SSP Status Register). The receive raw interrupt status signal, RXRIS, is asserted when there are four or more valid entries in the Receive FIFO. The RXRIS signal can be masked by the RXRIM signal. The final external SSPRXINTR is readable as the RXRIS bit in the raw interrupt status register SSPRIS or the masked version in the masked interrupt status register, SSPMIS. The RXRIS interrupt signal is cleared by reading from the receive FIFO until there are less than four values in it i.e. this interrupt is dynamic, based on the FIFO fill level.

Figure 4.4-10 illustrates the interface between the FIFO control logic and the data buffer. Writes to the register bank occur to the location pointed to by the current value of the WrPtr[2:0]. In the case of reads, the content of the location pointed to by the current value of the RdPtr[2:0] signal is always driven onto the RxFRdData[15:0] output lines.

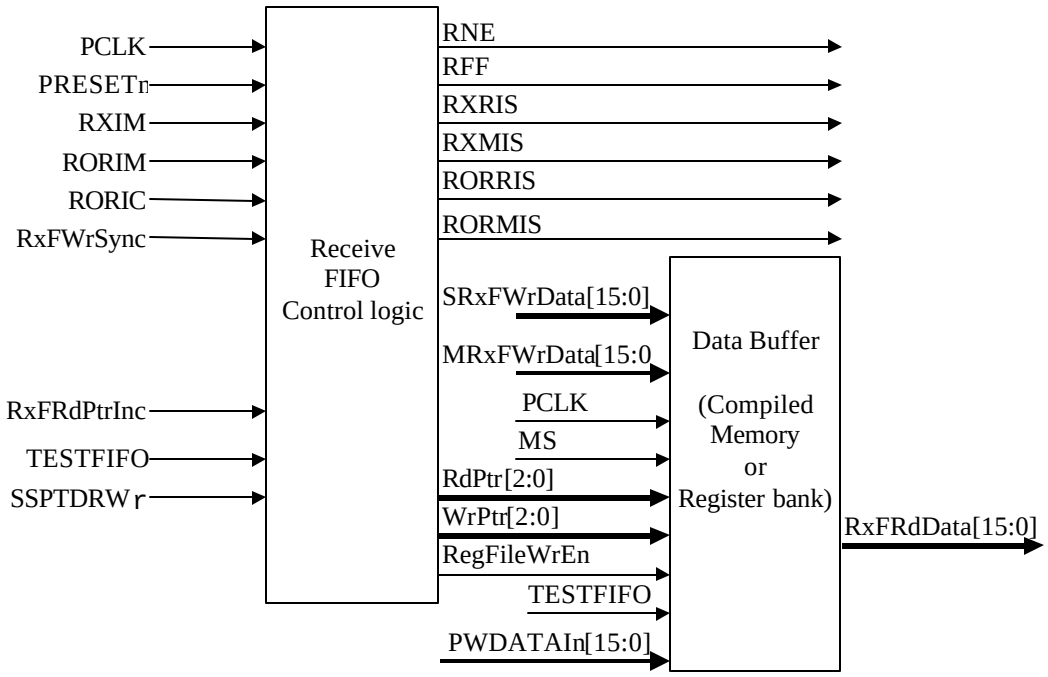

Figure 4.4-10 Receive FIFO Control logic interaction with Data buffer

4.4.7 Receive Time Out Generation Logic

This block is the source of the Receive Timeout (SSPRTINTR) interrupt which signifies that the receive FIFO has data within it and has not received any new data activity for a further 32 bit periods. It's function is to alert the system that data still remains in the receive FIFO and should be read.

In master mode, the MRxRT line, when asserted high, causes the timeout counter to loaded with it's maximum value of 31, effectively resetting the count. Similarly, in slave mode the SRxRT signal has the same function. If either of these signals are not asserted and RNESync is asserted, then the timeout counter is decremented when the IncRxTimeOut enable signal is active high. On reaching zero, the DataStp interrupt source is asserted high, and can only be cleared by application an RTICSync pulse.

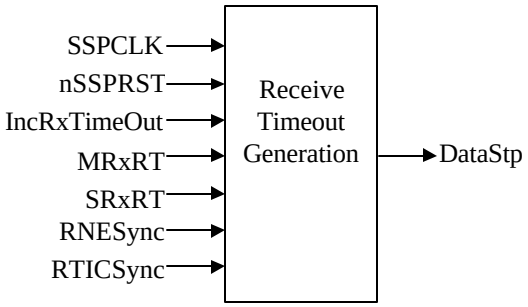

Figure 4.4-11 Recieve Timeout Generation Block

This block detects the SCLKIN line going idle. If the SCLKIN line goes to LOW state for more than a 32-bit period and if the Receive FIFO is not empty, the SCLKIN line is said to have gone idle. This is signalled by the block

output, DataStp. Once set, the DataStp signal can be cleared by reading the Rx Fifo contents until empty or if there is new activity on the recieve input. Alternatively it can be cleared by writing a "1" to the RTIC bit within the SSPICR register. To time the 32-bit period, the watchdog counter, that counts down with every IncRxTimeOut pulse, is initially loaded with it's maximum value of 31.

4.4.8 Interrupt Generation Logic

The INTR interrupt, which becomes the final external SSPINTR interrupt, is generated when there is a service request from either of the Transmit or Receive FIFOs, a receive FIFO overrun or a receive timeout. This combined interrupt output, which is effectively the logical OR of the individual masked interrupts, is cleared when the condition that caused the interrupt is removed.

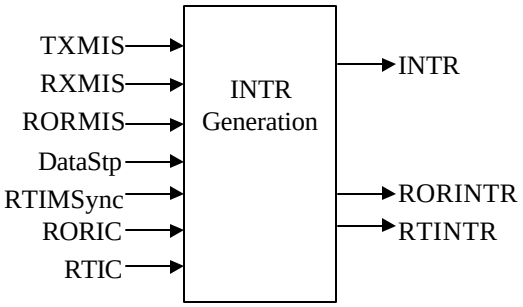

Figure 4.4-12 INTR Generation

Interrupts generated in the SSPCLK domain are treated as being asynchronous to the final processor clock domain. These interrupts are asserted in the SSPCLK domain, but synchronously removed by writing to bits within the SSP interrupt clear register (SSPICR) which reside in the PCLK domain. To reduce the latency of interrupt clearance, the reset action is initiated and removed in the PCLK domain, but time must be allowed for the actual clear signal to be synchronised back to the SSPCLK domain. Having cleared within the SSPCLK domain, the asynchronous interrupt path is re-enabled.

4.4.9 DMA Interface

This block provides an interface to connect to a DMA controller. It generates four DMA signals, two for the transmit requests SSPTXDMASREQ, SSPTXDMABREQ, and two for the receive requests SSPRXDMASREQ, SSPRXDMABREQ.

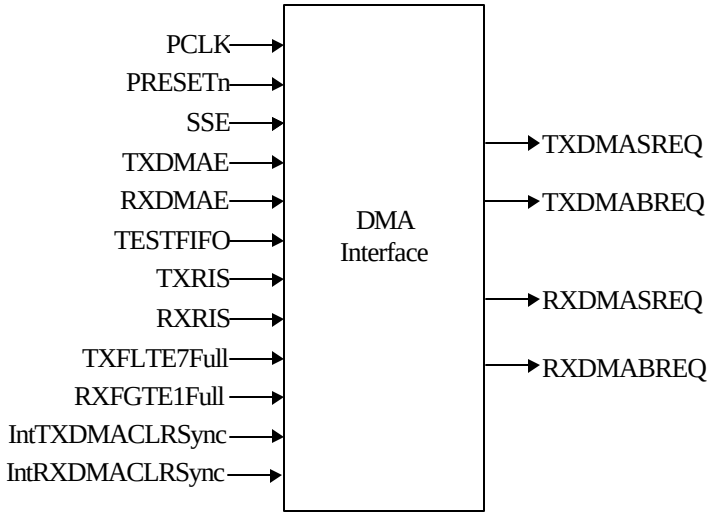

Figure 4.4-13 DMA Interface

DMA Transmit Interface

The block diagram is shown in **Figure 4.4-13 DMA Interface**. In normal operation, the DMA transmit interface is enabled when the SSP is enabled and the transmit DMA is enabled i.e. when signals SSE and TXDMAE are high. It can also be enabled in test mode, in which case only TESTFIFO and TXDMAE have to be high.

The level within the transmit FIFO is depicted by the signals TXRIS and TXFLTE7Full. The TXRIS signal is set when the data within the transmit FIFO is less than watermark level whilst the TXFLTE7Full is set when there is seven or less spaces within the transmit FIFO. The transmit DMA single request TXDMASREQ signal is set when TXFLTE7Full is active high. The transmit DMA burst request TXDMABREQ is set when the TXRIS signal is active high. Therefore, it is possible for both TXDMASREQ and TXDMASREQ to be active together.

The DMA controller has to be programmed with the most suitable burst length implied by the fixed transmit FIFO watermark level.

The TXDMASREQ and TXDMABREQ signals are cleared by the application of the SSPTXDMACLR signal from the DMA controller, which signifies that the end of a single/burst transfer is about to occur. They are also cleared when the transmit DMA function is disabled i.e. when TXDMAE is low.

DMA Receive Interface

In normal operation, the DMA receive interface is enabled when the SSP is enabled and receive DMA is enabled i.e. when signals SSE and RXDMAE are high. It can also be enabled in test mode, in which case only TESTFIFO and RXDMAE have to be high.

The level within the receive FIFO is depicted by the signals RXRIS and RXFGTE1Full. The RXRIS signal is set when the data within the receive FIFO is more than the watermark level whilst the RXFGTE1Full is set when there is data within the receive FIFO. The receive DMA single request RXDMASREQ signal is set when RXFGTE1Full is active high. The receive DMA burst request RXDMABREQ is set when the RXRIS is active high. Therefore, it is possible for both RXDMASREQ and RXDMASREQ to be active together.

The DMA controller has to be programmed with the most suitable burst length implied by the programmed receive FIFO watermark level.

The RXDMASREQ and RXDMABREQ signals are cleared by the application of the SSPRXDMACLR signal from the DMA controller, which signifies that the end of a single/burst transfer is about to occur. They are also cleared when the receive DMA function is disabled i.e. when RXDMAE is low.

4.4.10 Test logic

The block diagram is shown in **Figure 4.4-14 SSP Test Logic**. It contains the test mode registers in the SSP, the logic for the loopback mode and the logic for the integration tests.

The integration test logic allows all inputs and outputs to be read from and written to.

The inputs can be read via the integration input (SSPITIP) register and the outputs can be written via the integration output (SSPITOP) register.

All the inputs to the SSP enter the SspTest block and are multiplexed with the signals from the SSPITIP register under the control of the integration test enable (ITEN) signal.

Similarly, all the SSP outputs exit from the SspTest block and are multiplexed with the signals from the SSPITOP register under the control of the ITEN signal.

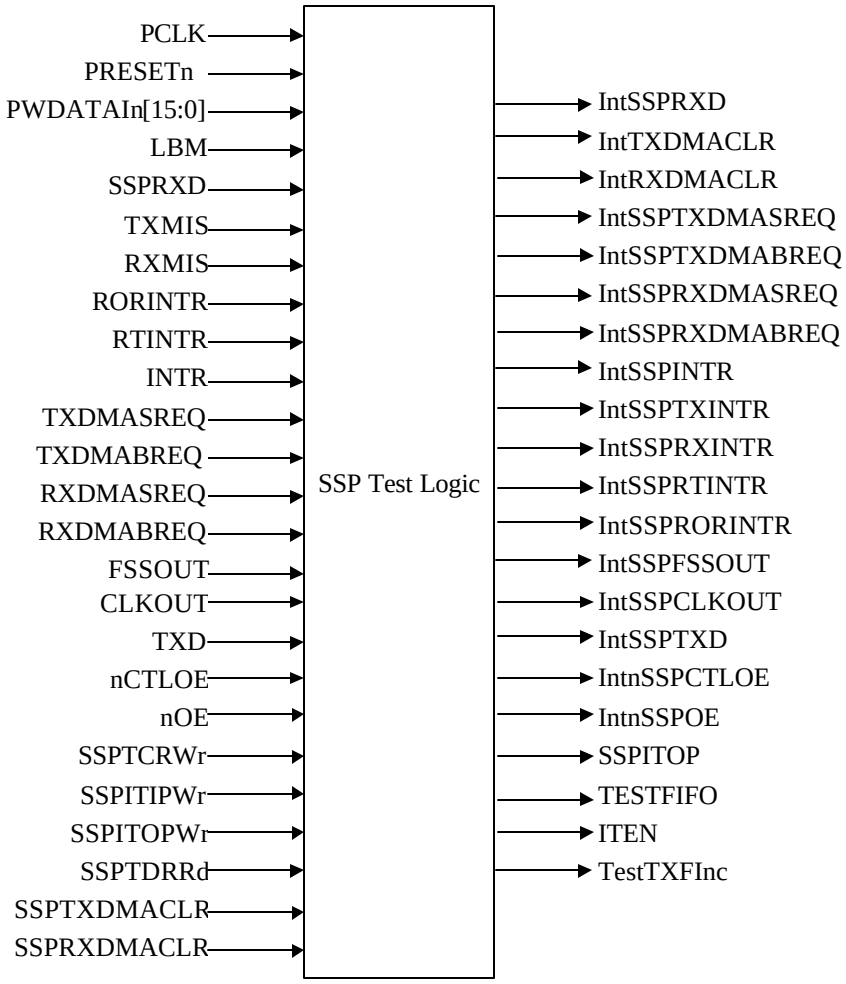

Figure 4.4-14 SSP Test Logic

When the LBM input is asserted in the master mode, the SSP is entered into the Loopback mode and the signal value on the SSPTXD output line is used as the internal receive bit stream and the SSPRXD input is ignored. Loopback is not applicable in slave mode.

4.4.11 Synchronisers

All synchronisers for signals crossing clock domains have been grouped into two sub-blocks – one for signals crossing over from the PCLK domain into the SSPCLK domain (SynctoSSPCLK) and the other for signals crossing over from the SSPCLK domain into the PCLK domain (SynctoPCLK).

The signals crossing clock domains are double synchronised with flip-flops operating on the rising edge of the clock into which the signals get synchronised.

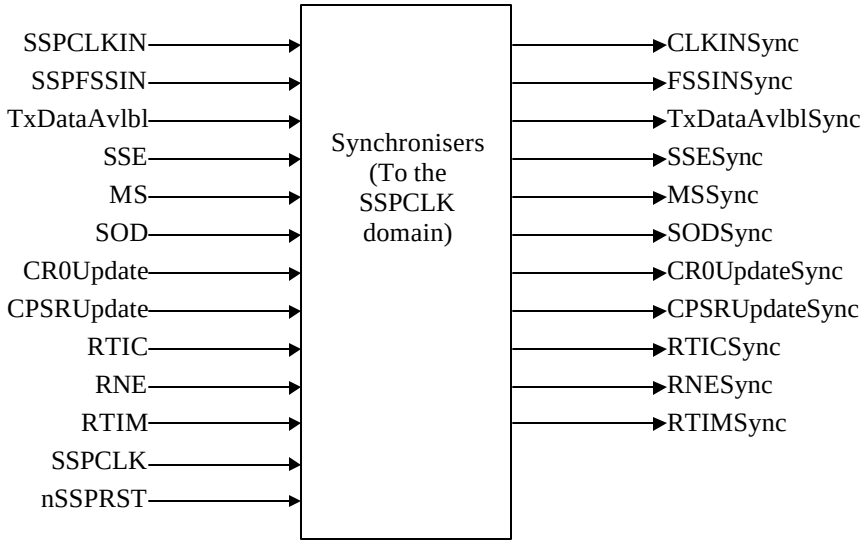

Figure 4.4-13 Synchronisers for signals crossing into SSPCLK domain

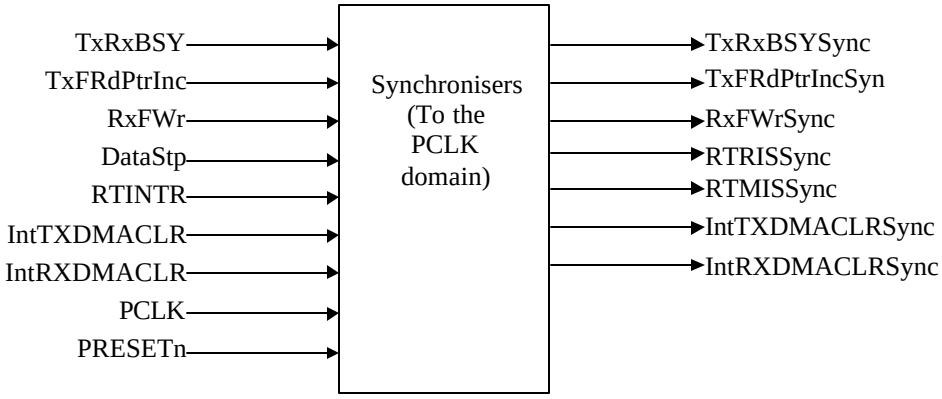

Figure 4.4-14 Synchronisers for signals crossing into PCLK domain

4.4.12 Revision Designator block

The block diagram is shown in **Figure 4.4-15 Revision Designator block.** It contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

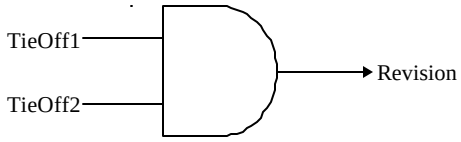

Figure 4.4-15 Revision Designator block

4.4.13 Clock Ratios

There is a constraint on the ratio of the frequencies of PCLK to SSPCLK. The frequency of SSPCLK must be less than or equal to that of PCLK. This is to ensure that control signals from the SSPCLK domain to the PCLK domain are assured to get synchronised before one frame duration.

$$fSSPCLK \leq fPCLK$$

In the slave mode of operation, the SSPCLKIN signal from the external master is double synchronised and then delayed to detect an edge. It would take three SSPCLKs to detect an edge on SSPCLKIN. SSPTXD now has less setup time to the falling edge of SSPCLKIN on which the master should be sampling the line. The setup and hold times on SSPRXD with reference to SSPCLKIN will also have to be more conservative to ensure that it is at the right value when the actual sampling occurs within the SSP. To ensure correct device operation, SSPCLK should be at least 12 times faster than the maximum expected frequency of SSPCLKIN.

The frequency selected for SSPCLK must accommodate the desired range of bit clock rates. The ratio of minimum SSPCLK frequency to SSPCLKOUT maximum frequency in the case of the slave mode is 12 and for the master mode it is 2.

Hence, to generate a maximum bit rate of 1.8432Mbps in the Master mode, the frequency of SSPCLK should be at least 3.6864MHz. With an SSPCLK frequency of 3.6864 MHz, the SSPCPSR register needs to be programmed with a value of 2 and the SCR[7:0] field in the SSPCR0 register needs to be programmed as 0.

To work with a maximum bit rate of 1.8432 Mbps in the slave mode, the frequency of SSPCLK should be atleast 22.12 MHz. With an SSPCLK frequency of 22.12 MHz, the SSPCPSR register may be programmed with a value of 12 and the SCR[7:0] field in the SSPCR0 register may be programmed as 0. Similarly the ratio of SSPCLK maximum frequency to SSPCLKOUT minimum frequency is 254 * 256.

The minimum frequency of SSPCLK is governed by the following equations, both of which need to be satisfied.

$$fSSPCLK(\min) \geq 2 \times fSCLKOUT(\max) \quad \text{[For Master mode]}$$

$$fSSPCLK(\min) \geq 12 \times fSCLKIN(\max) \quad \text{[For Slave mode]}$$

The maximum frequency of SSPCLK is governed by the following equations, both of which need to be satisfied.

$$fSSPCLK(\max) \leq 254 \times 256 \times fSCLKOUT(\min) \quad \text{[For Master mode]}$$

$$fSSPCLK(\max) \leq 254 \times 256 \times fSCLKIN(\min) \quad \text{[For Slave mode]}$$

5 SSP IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The hierarchy in the SSP is shown in ***figure 5.1-1***. A short description about the functionality of each of the sub-blocks is also given within brackets.

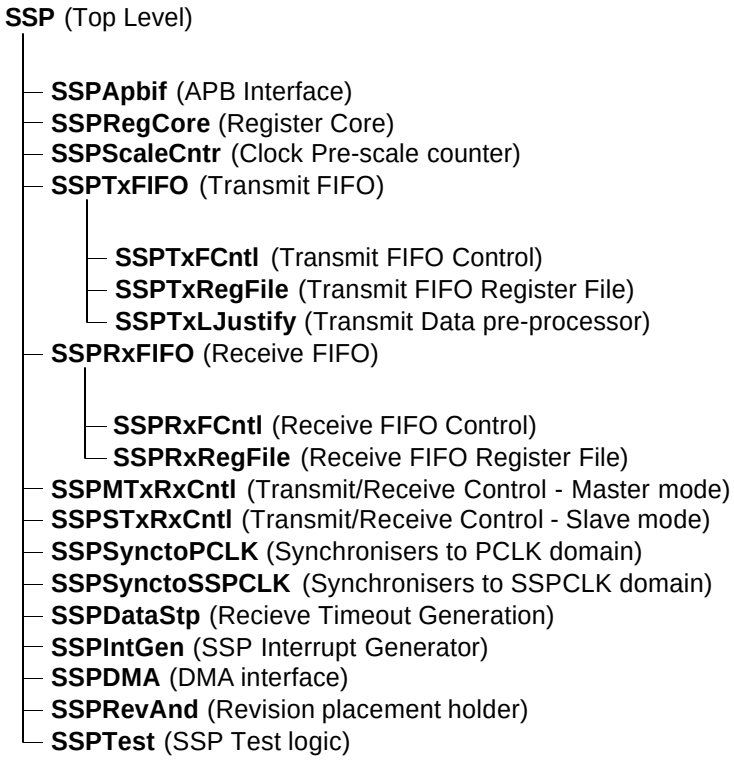

Figure 5.1-1 SSP Design Hierarchy

5.2 SSP Top Level Block (Ssp)

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the SSP.

5.3 SSP APB Interface (SspApbif)

The APB Interface generates decodes for writes into the registers in the address space of the SSP. It also contains the read interface for the SSP.

An internal write databus, PWDATAIn[15:0], is generated by gating the write databus input to the SSP (PWDATA[15:0]) with PSEL and PWRITE. Similarly, the address bus PADDR[11:2] are gated with PSEL to generate an internal address bus GatedPADDR[11:2]. These gating operations save power by preventing the toggling of the internal databus and the address bus when the device is not selected.

5.3.1 Write Interface

The write strobe for each register is generated with a decode of PSEL, PWRITE, PADDR[11:2] and PENABLE.

5.3.2 Read Interface

The read interface involves an output multiplexer followed by a bank of flip-flops, which drive data onto the PRDATA databus. These flip-flops are clocked by the rising edge of PCLK. Select inputs to the output multiplexer are generated with a combinational decode of PWRITE, PADDR[11:2], PENABLE and PSEL signals. Since these signals are stable from the rising edge of PCLK, almost one full PCLK period is available for the output of the multiplexer to stabilise before data is clocked in on the next rising edge of PCLK. Data is sampled by the APB Bridge on the next rising edge of PCLK on which PENABLE is sampled high. In order to ensure that PRDATA is driven only till the sampling edge of PCLK, the inverted version of PENABLE is used as a gating term in the generation of the select signal to the multiplexer.

The external data buses from multiple peripherals could be connected using one of many techniques so as to get a single read data bus driving the APB Bridge. The connection could be a multiplexed bus implementation or an OR bus implementation. For an OR bus implementation it is required that the peripheral drive zeros on the data bus when it is not selected. For a multiplexed bus implementation, there is no such requirement. In the SSP, the output multiplexer is left such that, when the device is not selected, it drives zeroes onto the read data bus. This ensures that there is no restriction placed on the type of external data bus connection used.

5.4 SSP Register Core (SspRegCore)

This block contains the normal mode registers in the SSP. In addition, it contains the first stage buffers for the SSPCR0 register and the SSPCPSR register used for the synchronisation of the register contents to the SSPCLK domain. When the Write enable signal for a register is asserted, data on the PWDATAIn bus is clocked into the first stage of the corresponding register on the rising edge of PCLK on which the Write enable signal is sampled high.

5.4.1 Synchronisation of the SSPCR0 register to the SSPCLK domain

A 2-buffer technique is used to synchronise the contents of the SSPCR0 register and the SSPCPSR register to the SSPCLK domain. Out of the two buffers, the first stage is located in the PCLK domain in the Register Core block (SspRegCore) and the final stage is located in the SSPCLK domain in the Pre-scale counter (SspScaleCntr) block.

When there is a write to the SSPCR0 register, the first stage buffer is updated with the new data written. Besides updating the content of the register, the CR0Update signal toggles to indicate to the SSPCLK domain that the synchronisation process can be initaited. This signal is double synchronised to the SSPCLK domain where it becomes the CR0UpdateSync. A one cycle delayed version of this signal is generated by clocking this signal through a D-type clocked by SSPCLK. By XOR-ing the CR0UpdateSync signal with its delayed version, a one-SSPCLK wide load pulse is generated on the CR0Stg2WrEn signal. When the CR0Stg2WrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the SSPCLK domain.

5.4.2 Synchronisation of the SSPCPSR register to the SSPCLK domain

A similar 2-buffer technique is used to synchronise the contents of the SSPCPSR register to the SSPCLK domain. The first stage is located in the PCLK domain in the Register Core block (SspRegCore) and the final stage is located in the SSPCLK domain in the Pre-scale counter (SspScaleCntr) block.

When there is a write to the SSPCPSR register, the first stage buffer is updated with the new data written. Besides updating the content of the register, the CPSRUpdate signal toggles to indicate to the SSPCLK domain that the synchronisation process has to start. This signal is double synchronised to the SSPCLK domain where it becomes

the CPSRUpdateSync. A one cycle delayed version of this signal is generated by clocking this signal through a D-type clocked by SSPCLK. By XOR-ing the CPSRUpdateSync signal with its delayed version, a one cycle SSPCLK wide load pulse is generated on the CPSRStg2WrEn signal. When the CPSRStg2WrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the SSPCLK domain.

5.5 SSP Pre-scale Counter (SspScaleCntr)

This block contains the pre-scale counter that divides SSPCLK by a value derived from value programmed in the SSPCPSR register. This block generates the SSPCLKDIV signal by dividing the input SSPCLK by half the value programmed in the SSPCPSR register. It also contains the second stage buffers for the SSPCR0 register and the SSPCPSR register.

When the CR0Stg2WrEn signal is asserted, the value on the SSPCR0[15:0] input is clocked into the second stage buffer, CR0[15:0]. Similarly, when the CPSRStg2WrEn signal is asserted, the value on the SSPCPSR[7:1] input is clocked into the second stage buffer, CPSR[7:1].

The value programmed into the SSPCR0 register is segregated into the individual register fields that are driven on the output signals SCR[7:0], DSS[3:0], FRF[1:0], SPO and SPH.

The pre-scale counter is necessary in order to allow for variation in the range of frequencies of the input SSPCLK for the same range of bit rates. For a nominal SSPCLK frequency of 7.3738MHz and for transmission rates ranging from 7.2Kbps to 1.8432Mbps, SSPCLK is to be divided by a fixed value of 4 (7.3738MHz/1.8432Mbps = 4) and then by a programmable number (SCR + 1) between 256 and 1. For the same range of transmission rates, to allow SSPCLK frequency to vary from a lower limit of the standard 3.6864MHz to an upper limit of around 50MHz, the factor of division will have to range from 2 (3.6864MHz/1.8432Mbps = 2) to a maximum of around 27 (50MHz/1.8432Mbps = 27.1267). Extending the division factors to the nearest power of 2, the desirable range in the factor of division is from 2 to 32 which implies a range of 3.6864MHz to 58.9MHz in the frequency of SSPCLK.

The SSPCLKDIV signal generated from the SspScaleCntr block is further divided in the SspMTxRxCntl state machine based on the SCR value. Division by (SCR + 1) would be inconvenient for odd numbered values of (SCR + 1). In order to account for this, the SspMTxRxCntl block divides the SSPCLKDIV signal by a value of 2*(SCR + 1). This is implemented using a down counter that has SCR as its reload value. The time duration required for this counter to count down to 0 corresponds to one SSPCLKOUT phase time. The division factor in the SspScaleCntr block is hence, one half of the Pre-Scale value programmed by software i.e. (CPSDVSR/2). The divide-by-two has been implemented by shifting the pre-scale value right by one bit (i.e.) the Least significant bit in the pre-scale value is ignored. Hence, only even-numbered pre-scale values can be written into the SSPCPSR register.

To accommodate higher frequencies of SSPCLK, the SSPCPSR register has been made 8-bits wide. The allowable range of pre-scale values that can be written to this register is hence, from 2 to 254. This allows for an SSPCLK frequency range from 3.6864MHz to around 500 MHz.

Also, the pre-scale counter needs to be only 7-bits wide although the SSPCPSR register (which specifies the reload value for this counter) is 8-bits wide from a software viewpoint.

The counter (SSCPSC[3:0]) that generates the SSPCLKDIV signal, operates on SSPCLK and is clock-enabled by the ClkEnable input signal. The counter is reloaded with the value in CPSR[7:1] when any of the following conditions occur:

- When the counter is at a count of 1. This is the normal case of reloading the counter on underflow. Since division is to be done by a factor of (CPSDVSR/2), reloading is done when the counter reaches 1. If reloading is done after the counter counts down to 0, the division would be by a factor of ((CPSDVSR/2) + 1).
- When the SSP is disabled (i.e.) when the SSESync input signal is not asserted. When the SSP is enabled, it must be made sure that the counter starts counting from the reload value. It is possible for the SSP to be disabled with the count somewhere between the reload value and one. Since the counter does not count when the SSP is disabled, the next time the SSP is enabled, the counter would start from the old value if the counter is not reloaded.

The SSPCLKDIV output is pulled HIGH whenever the count value is one.

5.6 Transmit FIFO (SspTxFIFO)

The Transmit FIFO block instantiates the transmit FIFO control logic (SspTxFCntl), the data storage elements (SspTxRegFile) and the Transmit data pre-processor (SspTxLJustify) block. Writes to the transmit FIFO occur from the APB through the APB interface, which operates on PCLK. Reads from the Transmit FIFO occur from the Transmit / Receive control logic, which operates on SSPCLK.

The transmit FIFO is 16-bits wide and 8-deep. Reads and writes to the FIFO are tracked with a Read pointer and a Write pointer respectively. The APB interface in the SSP drives the SSPDRWr input to the transmit FIFO with the decode of writes to the SSPDR register. The SSPDRWr input pulse is one PCLK wide. When the SSPDRWr input is asserted, the data on the PWDATAIn[15:0] input is clocked into the location in the Register file (SspTxRegFile) pointed to by the current value of the write pointer. The write pointer is controlled by the SspTxFCntl block. At the end of every write, the write pointer is incremented to point to the next write location. The Read pointer is also generated by the control logic. The TxFRdData[15:0] output is always driven with the contents of the FIFO location pointed to by the current value of the read pointer. Transmit data on the TxFRdData[15:0] bus, is left-justified and then driven on the TxFRdDataIn[15:0] output. The left-justification of transmit data in the SspTxLJustify block facilitates left-shifting of data bits onto the SSPTXD output line in the SspMTxRxCntl / SspSTxRxCntl blocks.

The TxDataAvlbl output signal indicates to the transmit / receive control logic that valid data is available in the FIFO for transmission. When the transmit section has completed transmission of one byte, it asserts the Read pointer increment signal which is seen as the TxFRdPtrIncSync input signal. After the completion of one frame, if there is further valid data available in the FIFO, the transmit section starts transmitting the data driven onto the TxFRdDataIn[15:0] lines which correspond to the next data element in the FIFO.

The TNF (Transmit FIFO Not Full) signal indicates the fill status of the FIFO. The assertion of the TNF signal indicates that there are 7 or fewer words in the transmit FIFO. This signal is readable through the TNF bit in the SSPSR register. The TFE (Transmit FIFO Empty) output signal is asserted when there are no valid data elements in the transmit FIFO. The Transmit FIFO Service request, SSPTXINTR, is generated in the SspTxFCntl block. The SSPTXINTR output is asserted if the Transmit FIFO contains four or fewer entries.

5.7 Transmit FIFO control logic (SspTxFCntl)

The Transmit FIFO control logic controls the read and write pointers for the FIFO, besides controlling accesses to the FIFO register file. It also generates the FIFO fill level status signals and the Transmit FIFO service request, SSPTXINTR.

The SSPDRWr input is a one-PCLK-wide pulse driven by the write decode to the SSPDR address. The TxFRdPtrIncSync signal is an indication originating from the transmit / receive control section that the transmit FIFO's read pointer can be incremented. This signal remains asserted for multiple PCLK cycles. To detect an edge on this signal, a delayed version (DelRdPtrInc) of this signal is generated by clocking it through a D-type flip-flop clocked by PCLK. An edge on TxFRdPtrIncSync is said to have been detected when TxFRdPtrIncSync is at a different level than DelRdPtrInc. The ClkEnable signal is the clock enable signal for the block. This signal is asserted continuously high in the normal mode and in a controlled manner in the test mode.

The WrPtr[2:0] output signal and the RdPtr[2:0] output signal indicate the current value of the write pointer and the read pointer respectively. The TNF signal indicates whether the transmit FIFO is full or not. The TFE signal when asserted, indicates that the transmit FIFO is empty i.e. the FIFO does not contain any valid data. The TxDataAvlbl signal indicates whether data is available in the transmit FIFO for transmission. The SSPTXINTR signal is the Transmit FIFO service request signal that is asserted when the Transmit FIFO contains four or fewer entries.

The WrPtrIncValid signal is asserted when a valid write access occurs to the FIFO. The following considerations apply when generating this signal:

- When there is a write access to the Transmit FIFO, the write data is clocked into the FIFO only if the FIFO is not already full. If the FIFO is full when the write access occurs, a simultaneous read should also occur for the write data to be clocked in. In normal operation, the Interrupt service routine that writes data into the FIFO is not expected to write more data than is required to fill the FIFO. This check is done to protect from such writes in case they occur.

The WrPtr[2:0] signal is used as the write pointer signal. The Write pointer is incremented every time the WrPtrIncValid signal is asserted.

The RdPtrIncValid signal is asserted when there is an edge detected on the TxFRdPtrIncSync signal and the FIFO is not already empty.

The RdPtr[2:0] signal is the Read pointer for the FIFO. The Read pointer is incremented every time the RdPtrIncValid signal is asserted.

There exists a possibility that the write pointer increments through “111” to “000”. The direct difference between the pointers will not give the correct fill level. To take this into account, the ‘Wrap’ signal has been introduced.

The Wrap signal is set when the Write pointer has wrapped around but the read pointer hasn’t. The signal is reset when the read pointer also subsequently wraps around. The Wrap signal is prepended to the write pointer and from the resultant value, the read pointer is subtracted to determine the fill level of the FIFO. The TxFFillLevel[3:0] signal indicates the number of locations in the FIFO that contain valid data. This signal takes a range of values starting with zero, when the FIFO is empty, one when there has been one write to the transmit FIFO and upto eight when the FIFO has all its eight locations filled.

The following considerations apply when generating the Wrap signal:

- When the write pointer is at “111” and the WrPtrIncValid signal is asserted and the RdPtrIncValid signal is not asserted then set the Wrap.
- When the read pointer is at “111” and the RdPtrIncValid signal is asserted, and the WrPtrIncValid signal is not asserted, clear Wrap. To combine this condition with the previous condition, toggle Wrap when either of the two conditions occurs.

When both the pointers are at “111” with the FIFO being full (Wrap already set) and both the WrPtrIncValid signal and the RdPtrIncValid signal are asserted, then the write is allowed to complete and the FIFO is still full, but retain the value of the Wrap bit i.e. let it remain set, thus indicating that the FIFO is still full.

Data is written into the register file location pointed to by the current value of the write pointer on the rising edge of PCLK on which the SSPDRWr signal is high.

When one character has been copied into the transmit shift register, the transmit / receive control logic asserts the TxFRdPtrInc signal, which gets synchronised to PCLK and is seen as the TxFRdPtrIncSync signal at the Transmit FIFO. A delayed version of this signal is generated as the DelRdPtrInc and is used to detect any edge in the TxFRdPtrIncSync signal.

The SSPTXINTR signal is generated on the basis of the FIFO fill level. The SSPTXINTR signal is asserted when the FIFO fill level becomes less than or equal to four. The interrupt is negated when the interrupting condition is no longer true. The assertion and de-assertion of SSPTXINTR occurs one PCLK after the respective conditions are satisfied.

5.8 Transmit FIFO Register File (SspTxRegFile)

The Register File is implemented as a bank of D-types. This block can be replaced with a compiled memory if required. The Register File also contains a 3-to-8 decoder to decode the Write pointer and a 8-to-1 bus multiplexer for the read data bus. Write data is clocked into the location pointed to by the WrPtr[2:0] input, on the rising edge of PCLK on which the Write Enable input (RegFileWrEn) is sampled high. The TxFRdData[15:0] bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (RdPtr[2:0]).

5.9 SSP Transmit Data Pre-Processor (SspTxLJustify)

In this block, the data read from the Transmit FIFO (TxFRdData[15:0]), which is in a right-justified form, is converted to a left-justified form and clocked out on the TxFRdDataIn[15:0] output from this block. This left-justification is needed since all the three protocols require that the most significant bit to be shifted out first, whereas transmit data is written into the FIFO in a right-justified manner. This block uses the DSS[3:0] input to deduce the number of valid bits in the Transmit data frame. The least significant valid bits in the TxFRdData[15:0] input form the most significant bits in the TxFRdDataIn[15:0] output. The rest of the bits in the TxFRdDataIn[15:0] output are filled with zeros. ***Figure 5.9-1*** illustrates this left-justification of transmit data. The FRF[1:0] signal is used to detect whether the programmed frame format is the National Microwire format. In this case, in Master mode, the transmit data is 8-bits wide irrespective of the data size value programmed in DSS[3:0]. The MS input is used to detect whether the SSP is in the master mode or the slave mode.

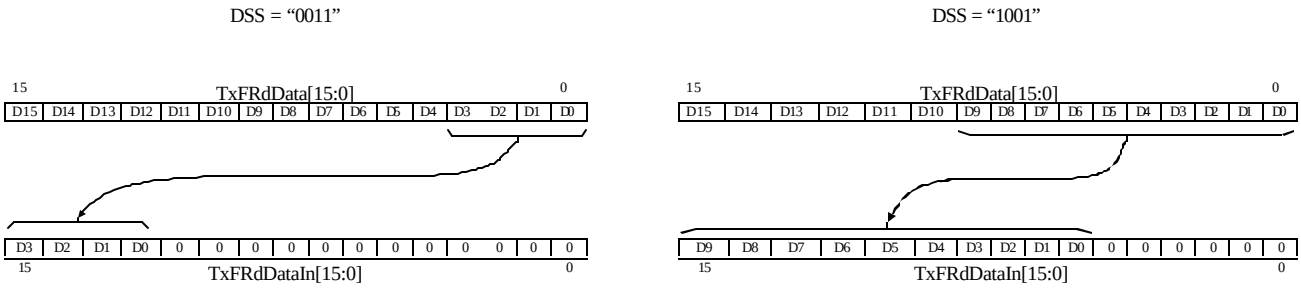

Figure 5.9-1 Left-Justification of Transmit Data

5.10 Receive FIFO (SspRxFIFO)

The Receive FIFO instantiates the Receive FIFO control logic (SspRxFCntl) and the Register File (SspRxRegFile). Writes to the Receive FIFO occur from the Receive section, which operates on SSPCLK. Reads from the Receive FIFO occur through the APB interface operating on PCLK.

The Receive FIFO is similar to the transmit FIFO. It is implemented as a circular buffer with read and write pointers.

When a full byte of data has been received, the Receive section drives the received data on the MRxFWrData[15:0] / SRxFWrData[15:0] bus. It then toggles the RxFWr signal, which gets synchronised to PCLK and is seen as the RxFWrSync input signal. When an edge is detected on the RxFWrSync input, data on the MRxFWrData[15:0] / SRxFWrData[15:0] input is clocked into the FIFO. The MS (Master/Slave select) input is used to select between MRxFWrData[15:0] and SRxFWrData[15:0].

The SSPRXINTR (Receive FIFO service request interrupt) output is asserted when there are 4 or more valid data elements in the receive FIFO and the interrupt is enabled i.e. the RIE input is asserted.

When the FIFO is already full and an attempt to perform another write is seen, the SSPRORINTR interrupt is asserted provided the RORIE (Interrupt enable for SSPRORINTR) input is asserted. This signal is cleared when the ClrRORINTR input signal is asserted. The RxFRdData[15:0] output from the FIFO is driven with the contents of the location pointed to by the current value of the read pointer.

The RNE (Receive FIFO not empty) and the RFF (Receive FIFO full) output signals provide information on the fill status in the FIFO.

5.11 Receive FIFO Controller (SspRxFCntl)

The Receive FIFO controller controls the read pointer and the write pointer. It also contains logic to detect and signal the FIFO Overrun condition. The receive interrupt (SSPRXINTR) is also generated by this block.

The RxFWrSync signal is clocked through a flip-flop clocked by PCLK to generate a delayed version DelRxFWrSync. The two signals are used to detect an edge on the RxFWrSync signal. When the RxFWrSync signal is at a different level than the DelRxFWrSync, an edge on the RxFWrSync signal is said to have been detected. If the FIFO is not already full, the data on the MRxFWrData[15:0] / SRxFWrData[15:0]input is clocked into the register file location currently being pointed to by the write pointer. On the same rising edge of PCLK, the Write pointer is also incremented.

Reads from the FIFO occur through the APB interface that operates on PCLK. When there is a read access from the SSPDR register, the APB interface asserts the RxFRdPtrInc signal. The RxFRdPtrInc signal is a one-PCLK-wide pulse used to indicate to the receive FIFO control logic that the read pointer can be incremented.

The Wrap signal is used to store the condition when the write pointer has wrapped around but the read pointer has not. The Wrap signal is used to calculate the FIFO fill level. The FIFOFillLevel signal is derived using the same mechanism as used for the transmit FIFO i.e. by subtracting the Read pointer from the value of the Write pointer with the Wrap bit prepended to the Write pointer.

The Receive FIFO service request interrupt SSPRXINTR is asserted when the SSPRXINTR interrupt enable (RIE) is asserted and the FIFO is filled to a level equal to or greater than the half fill level i.e. 4 or more data elements. The interrupt is cleared when the FIFO contents are read out so that there are 3 or fewer entries in the FIFO or if the SSPRXINTR Interrupt enable (RIE) is negated.

The Receive Overrun interrupt output signal (SSPRORINTR) is asserted when the SSPRORINTR interrupt enable (RORIE) is asserted and the Receive FIFO is full and a further data write access is detected without a simultaneous read access. The SSPRORINTR interrupt is cleared when there is a write to the SSPICR register or when the RORIE interrupt enable is cleared.

The SSPDRRd signal is a decode of PADDR[11:2], PSEL and PWRITE. The RxFRdPtrInc signal is generated by ANDing PENABLE with SSPDRRd. When the RxFRdPtrInc input is asserted, the Read pointer is incremented if the FIFO is not already empty. Read data from the FIFO is latched into the PRDATA register on the rising edge of PCLK on which SSPDRRd is high. The read pointer changes value on the rising edge of PCLK on which RxFRdPtrInc is sampled high.

If the SSPRXINTR enable (RIE) is not asserted, then the SSPRXINTR output is kept negated. If RIE is set and the Fill level in the Receive FIFO rises to 4 or more, SSPRXINTR is asserted on the next PCLK. When the Fill level falls below 4 or when the Interrupt is disabled, the SSPRXINTR is negated on the next PCLK. The one-PCLK delay in the de-assertion of the interrupt, will not re-trigger the interrupt given the fact that it takes several clocks for an interrupt service routine to complete after the bytes have been read out of the FIFO and for the interrupt to take effect again.

If the SSPRORINTR enable (RORIE) is not asserted, then the SSPRORINTR output is kept negated. If RORIE is set and the Receive FIFO is full and there is a further write access to the Receive FIFO detected (i.e. a rising edge on the RxFWrSync input is detected), SSPRORINTR is asserted. This interrupt is cleared when the interrupt is disabled (RORIE negated) or when the ClrRORINTR input is asserted. The ClrRORINTR input is asserted by the APB interface when there is a write access to the SSP Interrupt Clear register (SSPICR).

5.12 Receive FIFO Register File (SspRxRegFile)

The Receive Register file is similar to the transmit register file. Write data on the MRxFWrData[15:0] / SRxFWrData[15:0] input, is clocked into the location pointed to by the current value of the write pointer (WrPtr[2:0]) on the rising edge of PCLK on which the write enable (RegFileWrEn) is sampled high. The MS (Master / Slave select) input is used to select between write data from the master-mode controller and the slave-mode controller. The RxFRdData[15:0] bus is driven with the contents of the FIFO location pointed to by the

current value of the read pointer (RdPtr[2:0]). The Register File is implemented as a bank of D-types. This block can be replaced with a compiled memory if required. The Register File also contains a 3-to-8 decoder to decode the Write pointer and a 8-to-1 bus multiplexer for the read data bus.

5.13 Master Transmit / Receive Control state machine (SspMTxRxCntl)

The Master Transmit / Receive control state machine controls the shifting-out of the transmit data and the shifting-in of the receive data when the SSP is programmed to operate as a Master. This block also controls the nSSPOE, SSPFSSOUT and SSPCLKOUT outputs in accordance with the programmed transmission parameters.

This block operates on SSPCLK. The SSESync input to this block is the SSPCLK-synchronised version of the SSE bit in the SSPCR1 register. The SSPCLKDIV input is the output of the SspScaleCntr (SSP Clock Pre-Scale Counter) block. The TxDataAvlblSync input, when asserted, indicates that valid data for transmission is available in the Transmit FIFO. The MSSync input, when low, indicates that the SSP will operate in the Master mode. The input signals that indicate the transmission / reception parameters are DSS[3:0] (Data Size Select), FRF[1:0] (Frame Format), SCR[7:0] (Serial Clock Rate), SPO (SSPCLKOUT Polarity for the Motorola SPI frame format) and SPH (SSPCLKOUT Phase for the Motorola SPI frame format). The TxFRdDataIn[15:0] input is the transmit data from the Transmit FIFO. The IntSSPRXD input is the serial receive input.

The TxFRdPtrInc output from this block is the Read pointer increment signal to the Transmit FIFO. This signal is toggled at the beginning of the frame after the transmit data has been copied into the Transmit Shift register. In order to ensure that the signal gets synchronised to PCLK and is seen by the Transmit FIFO, the signal is kept stable for the duration of the frame. As the frequency of PCLK is equal to or greater than that of SSPCLK, this signal is assured to be seen in the PCLK domain.

The RxFWr output from this block is the write enable signal to the Receive FIFO. This signal is toggled when the last data bit is being received. On detecting an edge on this signal, the Receive FIFO clocks in data on the MRxFWrData[15:0] output from the SspMTxRxCntl block or the SRxFWrData[15:0] output from the SspSTxRxCntl block into the Receive FIFO. This signal is toggled at the end of a frame and kept stable till the end of the next frame.

The TxRxBSY output signal indicates that the SSP is busy with transmission / reception. This signal is used to generate the SSP Busy (BSY) signal in the Transmit FIFO. The SSPTXD output is the main serial transmit output from this block. The nSSPOE output signal is the Output Enable signal for the SSPTXD output. The SSPCLKOUT output is the Serial Clock output from this block and the SSPFSSOUT output is the Serial frame output. The output enable signal for SSPFSSOUT and SSPCLKOUT signal is the nSSPCTLOE signal and is set whenever SSP is in master mode.

The TxFRdPtrInc signal is common to the Master state machine (SspMTxRxCntl) and the Slave state machine (SspSTxRxCntl). The NxtSTxRdPtrInc signal from the Slave state machine is combined with the NextTxFRdPtrInc signal from the Master state machine using the MSSync (Master / Slave select) input to generate the TxFRdPtrInc signal to the Transmit FIFO. Similarly, the NextSnSSPOE, NextSSSPTXD, NextSRxFWr and NextSTxRxBSY signals from the Slave state machine are combined respectively with NextMnSSPOE, NextMSSPTXD, NextMRxFWr and NextMTxRxBSY from the Master state machine to generate respectively the nSSPOE, SSPTXD, RxFWr and TxRxBSY clocked outputs of the Master state machine. The MSSync bit is used to select one of the two sources for each of these signals.

The counters used in this state machine are:

- BitPeriodCnt[7:0]. 8-bit counter used to count SSPCLKDIV pulses and time SSPCLKOUT phase duration.
- BitCnt[3:0] Counter to track the number of bits being shifted.

The comparators used in this state machine are:

- BitPeriodCmp. To compare and check if BitPeriodCnt has reached “0000”
- BitCmp. To compare and check if BitCnt has reached “000”.

- BitCmpHalf. To compare and check if BitCnt has reached half the maximum count i.e. DSS[3:0] value shifted right by 1 bit position {0, DSS[3:1]}.

Transmit data on the TxFRdDataIn[15:0] input is first loaded into a 16-bit Transmit Shift Register (TxShft[15:0]) and bits are shifted out onto the SSPTXD output line (MSBit first). Receive data sampled on the IntSSPRXD input is shifted into an internal shift register (RxShft[15:0]) and when a complete frame is received, the received data is copied into a receive buffer (RxFWrData[15:0]) from the shift register. All outputs from this block hold their values once assigned, till they are re-assigned with a different value in another state / state-transition.

Activity on the SSPCLKOUT signal is monitored by continuously checking for edge transitions. On detection of either a positive or negative edge transition, then the signal MRxRT is asserted. This signal is used to reload the receive timeout counter in the SSPDataStp block. The output signal IncRxTimeout is the enable to this receive timeout counter. If there is data remaining in the receive FIFO and the timeout counter reaches a value of zero, which takes 32 bit periods, then the receive timeout interrupt SSPRTINTR, if not masked, is asserted. This is to alert the system that data still remains and should be read from the receive FIFO.

In the state diagrams in this section, circles represent normal states and circles with a bounding square represent hierarchical states. A Hierarchical state is a representation of a group of states that are functionally related. These hierarchical states, however, do not represent any further block hierarchy within the SspMTxRxCntl block.

The state machine is split into three main divisions corresponding to the three frame formats – TI Synchronous Serial Frame Format, Motorola SPI Frame Format and National Microwire Frame Format.

Figure 5.13-1 illustrates the overall structure of the state machine. On reset, the state machine enters the ST_MRESET state after initialising the internal counters, shift registers, buffers and the block outputs.

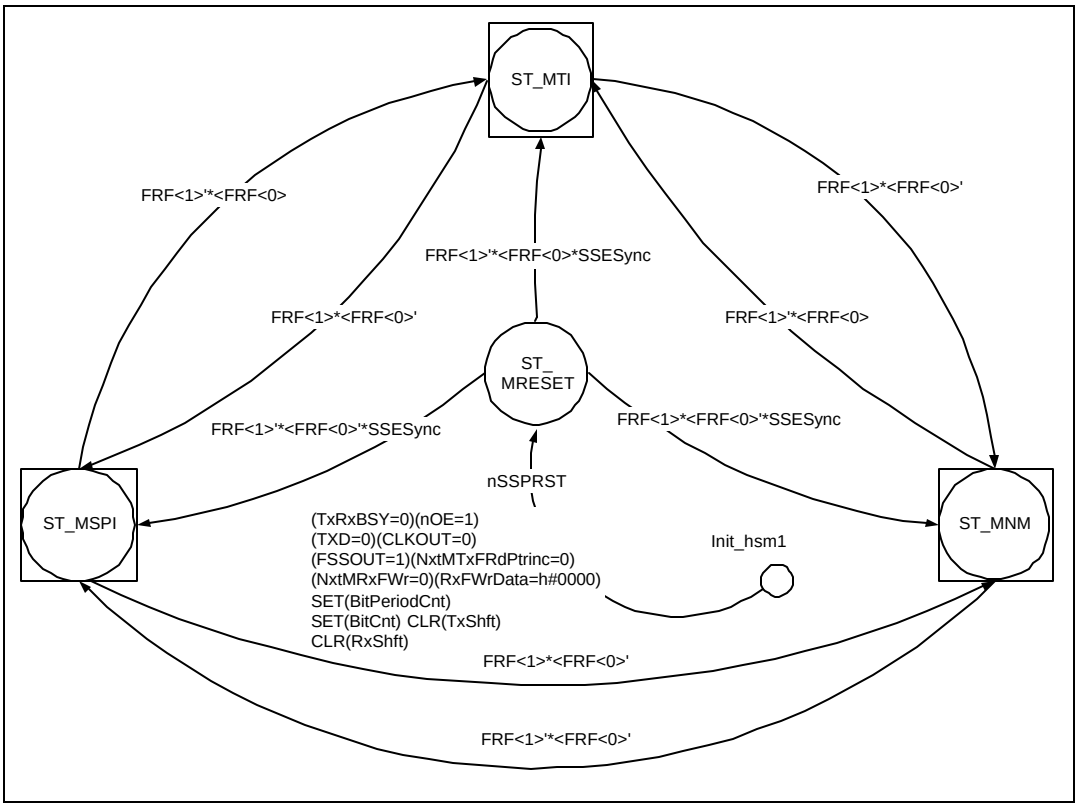

Figure 5.13-1 Three sections of the SspMTxRxCntl state machine

When the SSP is enabled i.e. when the SSESync signal is asserted, the state machine transitions to one of the hierarchical states ST_TI, ST_NM or ST_SPI based on the frame format programmed in the FRF[1:0] bits. Each of these hierarchical states contains the part of the state machine corresponding to one specific frame format. When

the state machine is in the idle state of one specific frame format and the FRF[1:0] bits are re-programmed to require a different frame format, the state machine transitions to the hierarchical state corresponding to the new frame format. The nSSPOE output is intended to be used as the output enable control for the pad associated with the SSPTXD output. To account for pad delays, nSSPOE is asserted one SSPCLKOUT phase time before SSPTXD is required to be valid and is negated one SSPCLKOUT phase time after SSPTXD is no longer required to be valid.

5.13.1 Exception Handling

When in any state in the entire state machine, if the SSP is disabled i.e. the SSESync input is negated, transmission / reception in progress is aborted and the state machine returns to the ST_MRESET state on the next SSPCLK after initialisation. Also, if the state machine inadvertently enters an unknown state (represented as the 'DEFAULT' state in the state diagram), it transitions to the ST_MRESET state on the next SSPCLK after initialisation.

The state diagram in ***figure 5.13-2*** illustrates this operation.

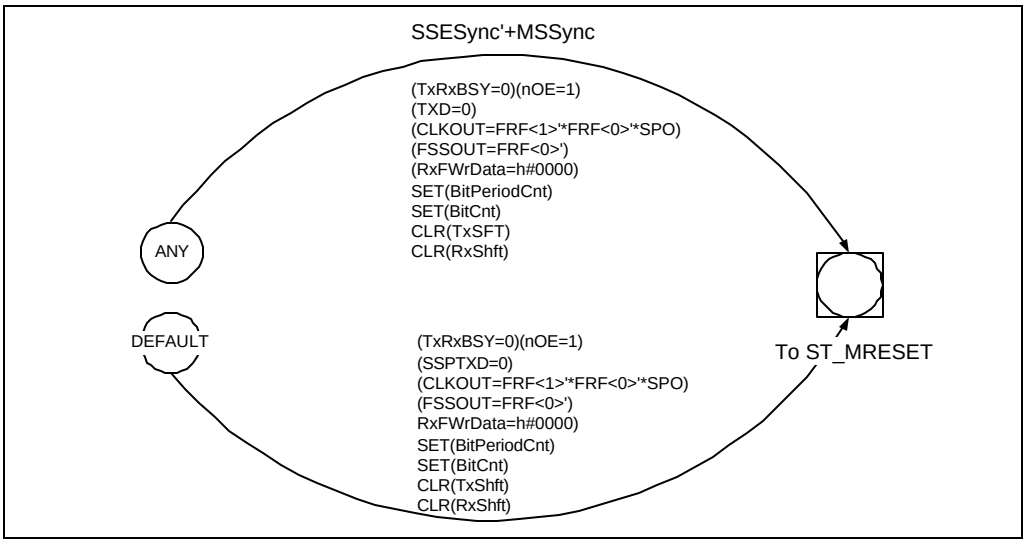

Figure 5.13-2 Exception handling in the SspMTxRxCntl state machine

5.13.2 Texas Instruments' Synchronous Serial Frame Format

One section of the state machine that handles TI Synchronous Serial Frame Format is shown in ***Figure 5.13-3***. In this frame format, transmit data is driven on the rising edge of SSPCLKOUT and receive data is sampled on the falling edge of SSPCLKOUT. The ST_MTIIDLE state is the idle state for the section of the state machine that handles this frame format. So long as the Frame format is not changed and there is no transmit data available, the state machine stays in this state. When the TxDataAvlblSync signal is asserted, it indicates that valid data is available in the Transmit FIFO. On detecting this signal the state machine transitions to the ST_MTISTART hierarchical state after asserting the TxRxBSY output signal to indicate that the transmit / receive controller is now busy. In this state, the state machine loads the transmit shift register with transmit data and pulls SSPCLKOUT high for one phase time. It also generates a one SSPCLKOUT-wide pulse on the SSPFSSOUT output.

On completion of the start phase, the state machine transitions to the ST_MTIBITS hierarchical state where the current data to be transmitted is shifted out bit by bit onto the SSPTXD output and the serial data input on the IntSSPRXD pin is sampled and shifted into the Receive Shift register.

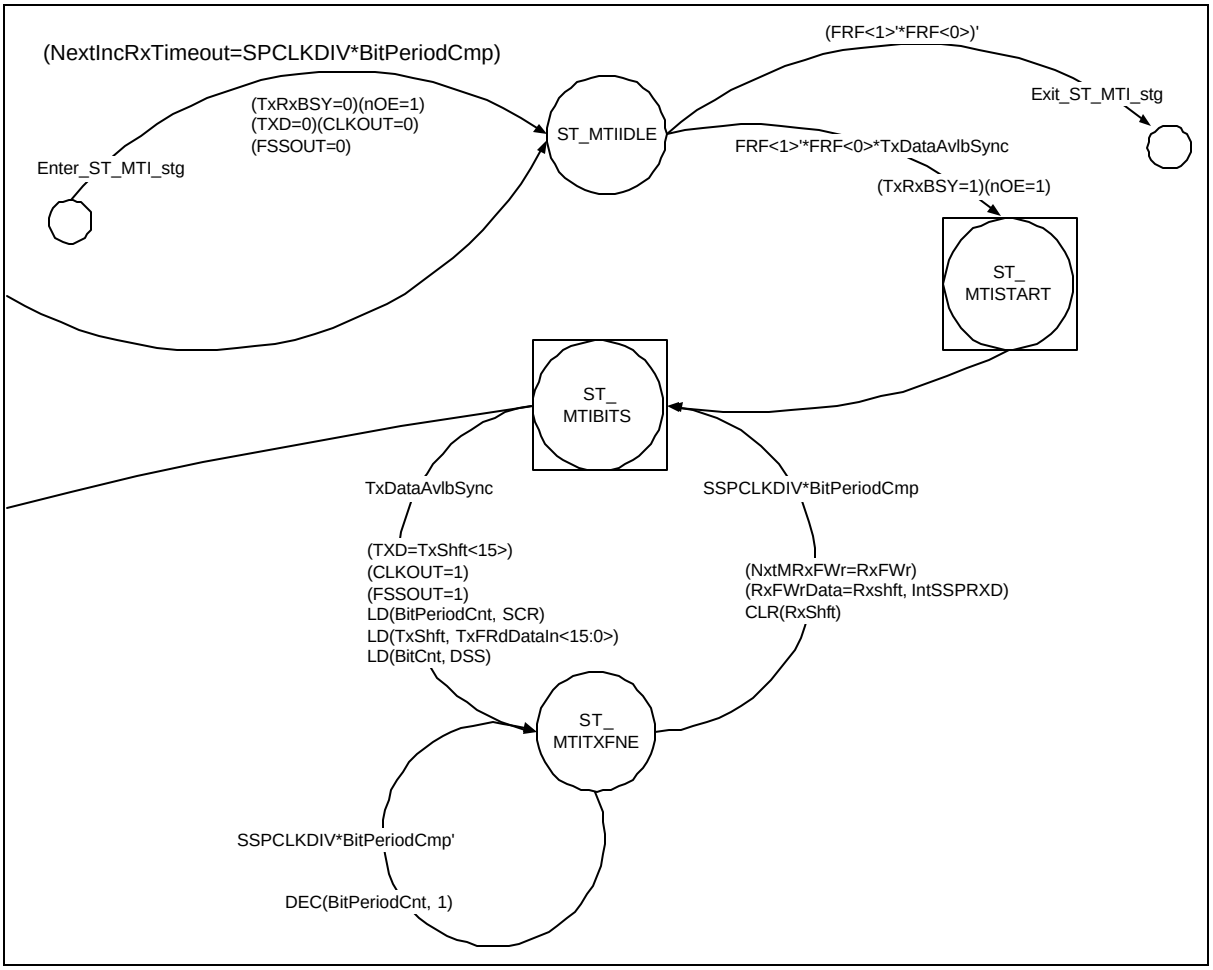

Figure 5.13-3 A section of the SspMTxRxCntl state machine for the TI Synchronous Serial Frame Format

When all but the last bit have been shifted out / shifted in, the state machine checks if further data is available as indicated by the assertion of the TxDataAvlblSync signal. If more transmit data is available, SSPFSSOUT is pulled high for one SSPCLKOUT duration as required by the protocol. After shifting out the last transmit bit in the current frame, the transmit shift register is reloaded with new transmit data, the counters are initialised and the state machine transitions to the ST_MTITXFNE state. In this state, the state machine waits for a period of time corresponding to one SSPCLKOUT phase. At the end of the phase time, SSPCLKOUT is pulled low and the last bit of the receive data is sampled on the IntSSPRXD input. The Receive buffer (MRxFWrData[15:0]) is then updated with the contents of the receive shift register (RxShft[14:0]) and the RxFWr output is asserted to indicate to the Receive FIFO that valid data is available on the MRxFWrData[15:0] output bus. The state machine then transitions to the ST_MTIBITS state again to continue with the transmission / reception of the rest of the frame.

Figure 5.13-4 illustrates the other section of the state machine that handles TI's Synchronous serial frame format. When all but the last bit of a frame have been shifted out / shifted in and there is no more data in the transmit FIFO to be transmitted as indicated by the de-assertion of the TxDataAvlblSync signal, the last bit is shifted out on the SSPTXD output and the BitPeriodCnt counter is reloaded with SCR[7:0].

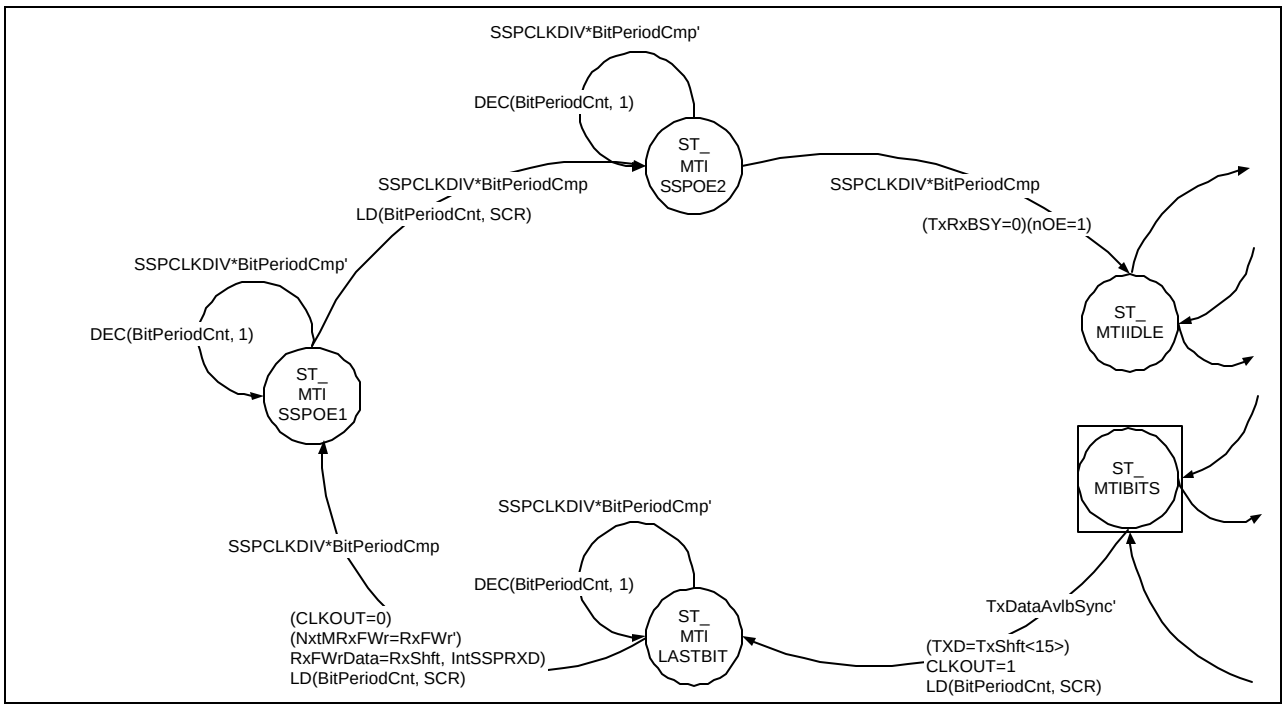

Figure 5.13-4 A section of the SspMTxRxCntl state machine for the TI Synchronous Serial Frame Format

The state machine transitions to the ST_MTILASTBIT state where it waits for a time duration corresponding to one SSPCLKOUT phase time. At the end of one SSPCLKOUT phase time, the SSPCLKOUT line is pulled low and the last receive data bit on the IntSSPRXD input is sampled and clocked into the MRxFWrData[15:0] receive buffer along with the contents of the receive shift register. The RxFWr signal is toggled to indicate to the receive FIFO that valid data is available on the MRxFWrData[15:0] bus to be clocked into the Receive FIFO. To complete the last frame, the nSSPOE output is negated after a time duration corresponding to one SSPCLKOUT time period (one SSPCLKOUT phase time after the last SSPCLKOUT falling edge as required by the protocol and an additional SSPCLKOUT phase time to account for pad delays). This wait is implemented in the ST_MTISSPOE1 and ST_MTISSPOE2 states. After nSSPOE is negated, the state machine transitions to the ST_MTIIDLE state.

The state diagram for the ST_MTISTART hierarchical state is shown in ***Figure 5.13-5***. When valid data is available in the transmit FIFO for transmission, the state machine waits for a valid pulse on the SSPCKLDIV input. If SSPCLKDIV is not asserted, the state machine transitions to the ST_MTIDATAAVLBL state and waits for the next SSPCLKDIV pulse. If SSPCLKDIV is asserted, the state machine transitions to the ST_MTISFRM1 state after pulling SSPCLKOUT high and SSPFSSOUT high. The transmit shift register (TxShft[15:0]) is loaded with data from the transmit FIFO on the TxFRdDataIn[15:0] input and the TxFRdPtrInc output is toggled to indicate to the transmit FIFO that a data character has been copied into the Transmit shift register and that the read pointer in the FIFO can be incremented. The BitCnt[3:0] counter is loaded with the value on the DSS[3:0] input.

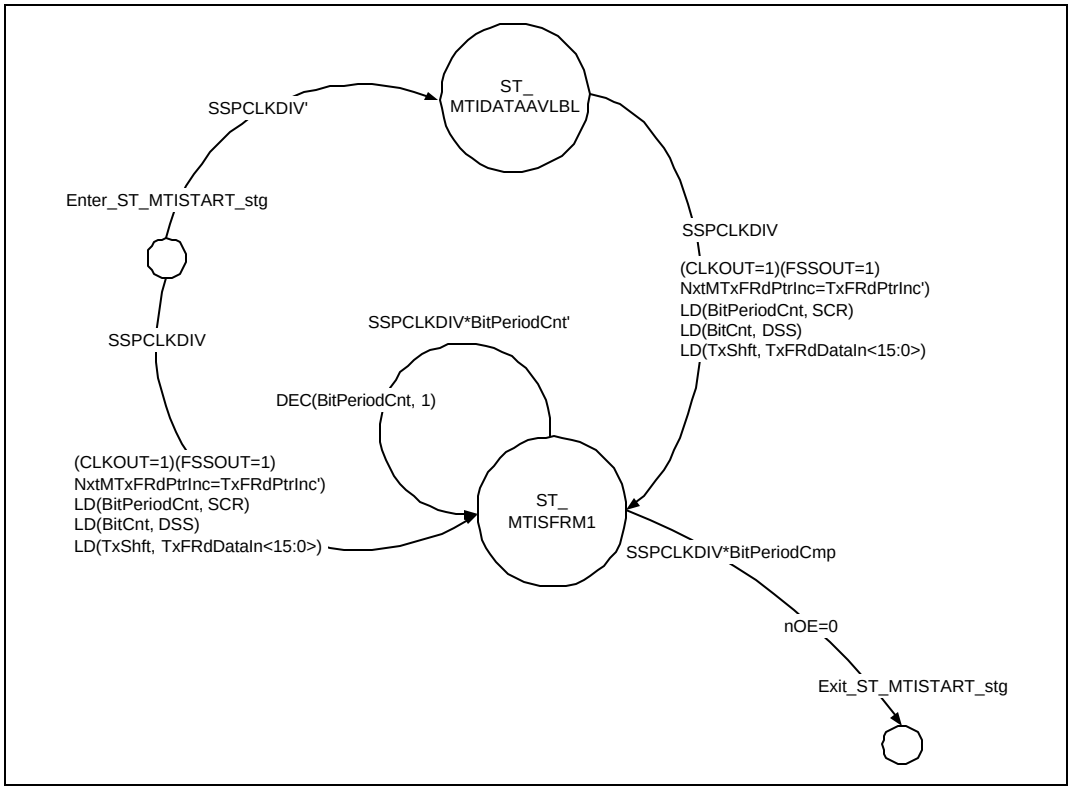

Figure 5.13-5 State diagram for ST_MTISTART hierarchical state

The BitPeriodCnt[7:0] counter is loaded with the value on the SCR[7:0] input. This counter is used to time the duration of one SSPCLKOUT phase. The counter is decremented with every pulse on the SSPCLKDIV input and when it reaches zero, one SSPCLKOUT phase time is said to have elapsed.

When one phase time of SSPCLKOUT has elapsed, nSSPOE is asserted and the state machine transitions to the ST_MTIBITS hierarchical state.

Figure 5.13-6 illustrates the state diagram for the ST_MTIBITS hierarchical state. Here, the first half of the frame is handled by the ST_MTISFRM2 and the ST_MTISHIFT1 states. The remaining part of the frame is handled by the ST_MTISTG2 and the ST_MTISHIFT2 states.

The BitPeriodCnt[7:0] counter is reloaded with SCR[7:0] and SSPCLKOUT is pulled low before the state machine transitions to the ST_MTISFRM2 state. In this state the state machine waits for one SSPCLKOUT phase time. This corresponds to the low phase of the one SSPCLKOUT-period-duration during which SSPFSSOUT is to be held high. When an SSPCLKOUT phase time has elapsed, SSPFSSOUT is pulled low and SSPCLKOUT is pulled high. The most significant bit in the transmit shift register is clocked out onto the SSPTXD output, the contents of the transmit shift register are shifted left by one bit position and the BitPeriodCnt[7:0] counter is reloaded with SCR[7:0]. Then the state machine transitions to the ST_MTISHIFT1 state where it waits for one SSPCLKOUT phase time. At the end of this time duration, SSPCLKOUT is pulled low and the value on the IntSSPRXD input is sampled and shifted into the receive shift register. The BitCnt[3:0] counter is decremented by 1 to indicate that one bit has been shifted out and one bit has been shifted in. The state machine then transitions to the ST_MTISFRM2 state.

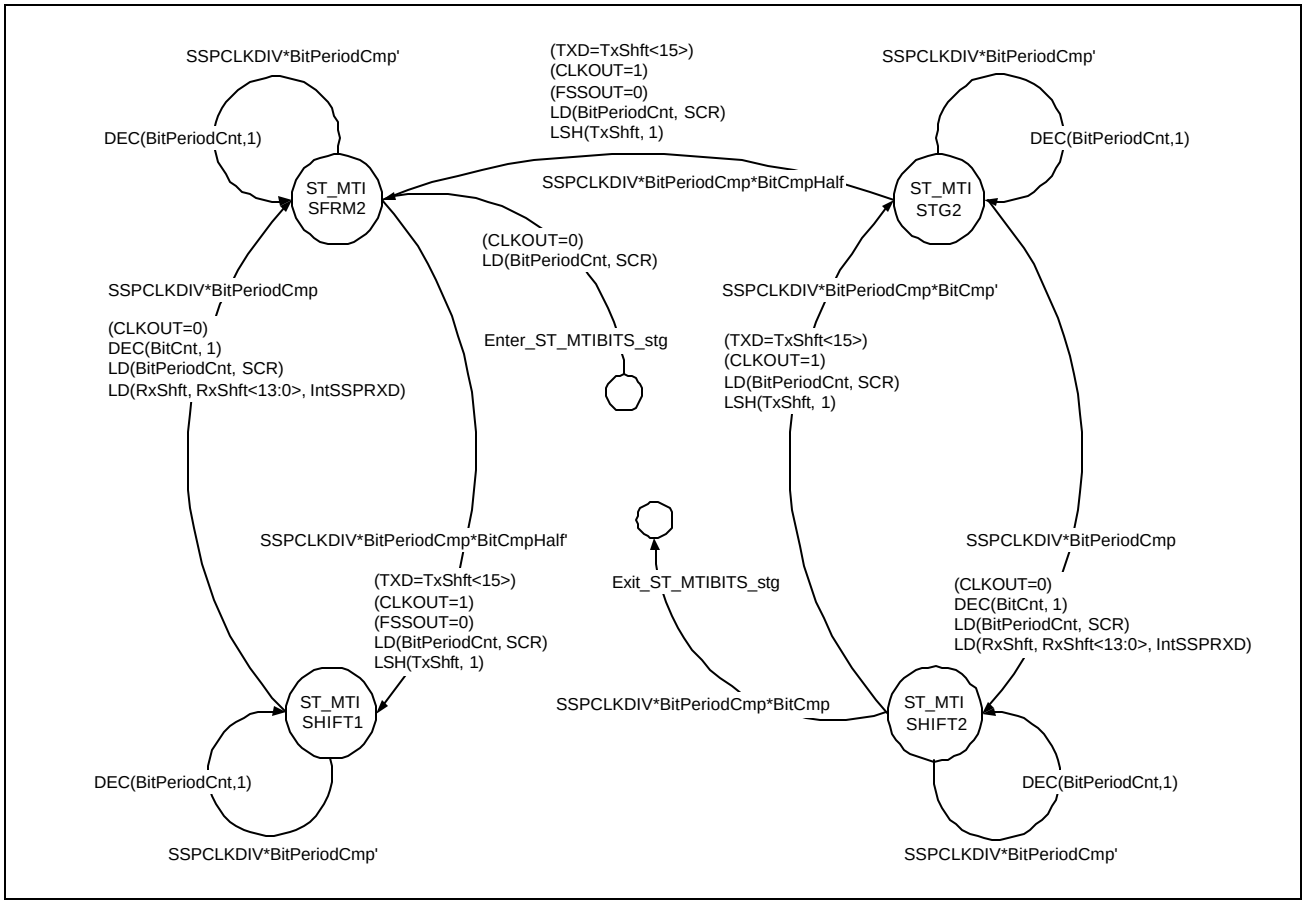

Figure 5.13-6 State diagram for the ST_MTIBITS hierarchical state.

When the BitCmpHalf signal is set, it indicates that half the frame has been completed. When a frame has been half completed, the state machine transitions to the ST_MTISTG2 state.

In the ST_MTISTG2 state, the state machine waits for one SSPCLKOUT phase time and then transitions to the ST_MTISHIFT2 state after shifting in the next receive data bit and reloading the BitPeriodCnt[7:0] counter with the value on SCR[7:0]. In the ST_MTISHIFT2 state, the state machine waits for one SSPCLKOUT phase time. If, at the end of this time duration, the BitCnt[3:0] counter is found to have counted down to zero as indicated by the assertion of the BitCmp signal, it indicates that all but the last data bit have been shifted out / shifted in. The state machine then transitions out of the ST_MTIBITS hierarchical state to check if any further data is available for transmission.

For a CPSDVSR value of 4 written to the SSPCPSR register and a value of 1 written to the SCR field in the SSPCR0 register, the relation of the frequency of SSPCLKOUT to the frequency of SSPCLK should be as shown below.

$$f_{SCLKOUT} \Leftarrow \frac{f_{SSPCLK}}{(SCR+1)*CPSDVSR}$$

$$\therefore\ f_{SCLKOUT} \Leftarrow \frac{f_{SSPCLK}}{(1+1)*4}$$

$$\therefore\ f_{SCLKOUT} \Leftarrow \frac{f_{SSPCLK}}{8}$$

The division is performed in two stages. The SSCPSC pre-scale counter in the SspScaleCntr generates the SSPCLKDIV signal that has a frequency of one half the frequency of SSPCLK. By using SCR[7:0] (1 in this example) as the reload value for the BitPeriodCnt[7:0] counter and by allowing the counter to count down with every SSPCLKDIV pulse, the phase duration of SSPCLKOUT is timed (4 SSPCLK periods per SSPCLKOUT phase in this example). After every phase time, the BitPeriodCnt counter is reloaded and SSPCLKOUT is toggled.

SSPFSSOUT is state machine goes into states ST_MTISFRM1 and ST_MTISFRM2 for the first time. Thereafter, the first part of the frame is transmitted / received with the state machine shuttling between the ST_MTISHIFT1 and the ST_MTISFRM2 states. The remaining part of the frame is transmitted / received with the state machine shuttling between the ST_MTISTG2 and the ST_MTISHIFT2 states.

5.13.3 Motorola's SPI Frame Format

The section of the SspMTxRxCntl state machine that handles the Motorola SPI Frame format is illustrated in ***Figure 5.13-8***. The ST_MSPIIDLE state is the idle state for this section of the state machine. The state machine remains in this state if there is no data available for transmission in the transmit FIFO provided the frame format is not altered. While in the ST_MSPIIDLE state, if the frame format is changed, the state machine transitions to the idle state corresponding to the new programmed frame format.

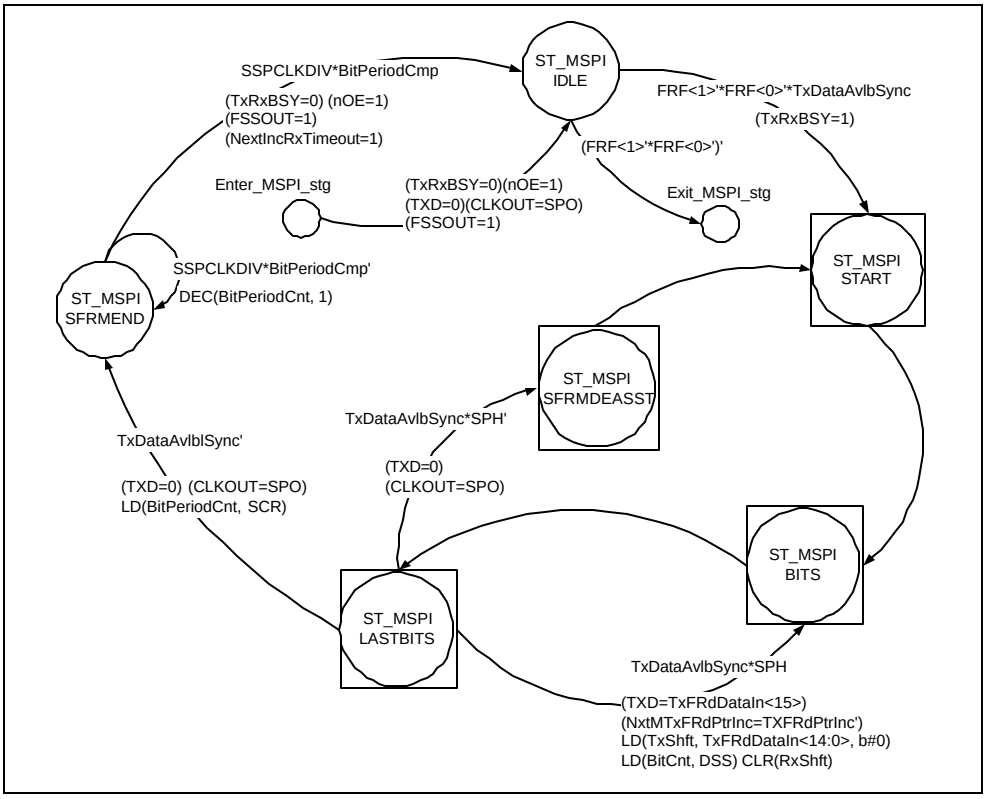

Figure 5.13-8 A section of the SspMTxRxCntl state machine for the Motorola SPI Frame Format

When valid data is present in the transmit FIFO as indicated by the assertion of the TxDataAvlblSync input signal, the state machine transitions to the ST_MSPISTART hierarchical state where SSPFSSOUT and nSSPOE are asserted. Then the state machine transitions to the ST_MSPIBITS hierarchical state where the shifting out / shifting in of data bits occurs. When this is complete, the state machine transitions to the ST_MSPILASTBITS state where the last bit is transmitted/received. If more transmit data is available and if SPH is set to 0, then SSPFSSOUT needs to be negated and asserted again. This is implemented in the ST_MSPISFRMDEASST state. If SPH is set to 1, then SSPFSSOUT can remain continuously asserted for successive transmissions. If no more transmit data is available in the transmit FIFO, the state machine transitions to the ST_MSPISFRMEND state where it waits for one SSPCLKOUT phase time before de-asserting nSSPOE.

Figure 5.13-9 illustrates the state diagram for the ST_MSPISTART hierarchical state. The assertion and de-assertion of SSPTXD, nSSPOE and SSPCLKOUT are in sync with pulses on the SSPCLKDIV input. If SSPCLKDIV is not asserted, the state machine waits in the ST_MSPIDATAAVLBL state for the next SSPCLKDIV pulse. When SSPCLKDIV is asserted, nSSPOE is pulled high and SSPFSSOUT is pulled low. Data on the TxFRdDataIn[15:0] input is copied into the Transmit Shift register (TxShft[15:0]) and the TxFRdPtrInc output is toggled to indicate to the transmit FIFO that the read pointer can be incremented. The state machine then enters the ST_MSPISFRM state where it waits for one SSPCLKOUT phase duration. At the end of a phase duration of SSPCLKOUT, the most significant bit in the transmit shift register is clocked into SSPTXD and the shift register is shifted left by one bit position before exiting the hierarchical state.

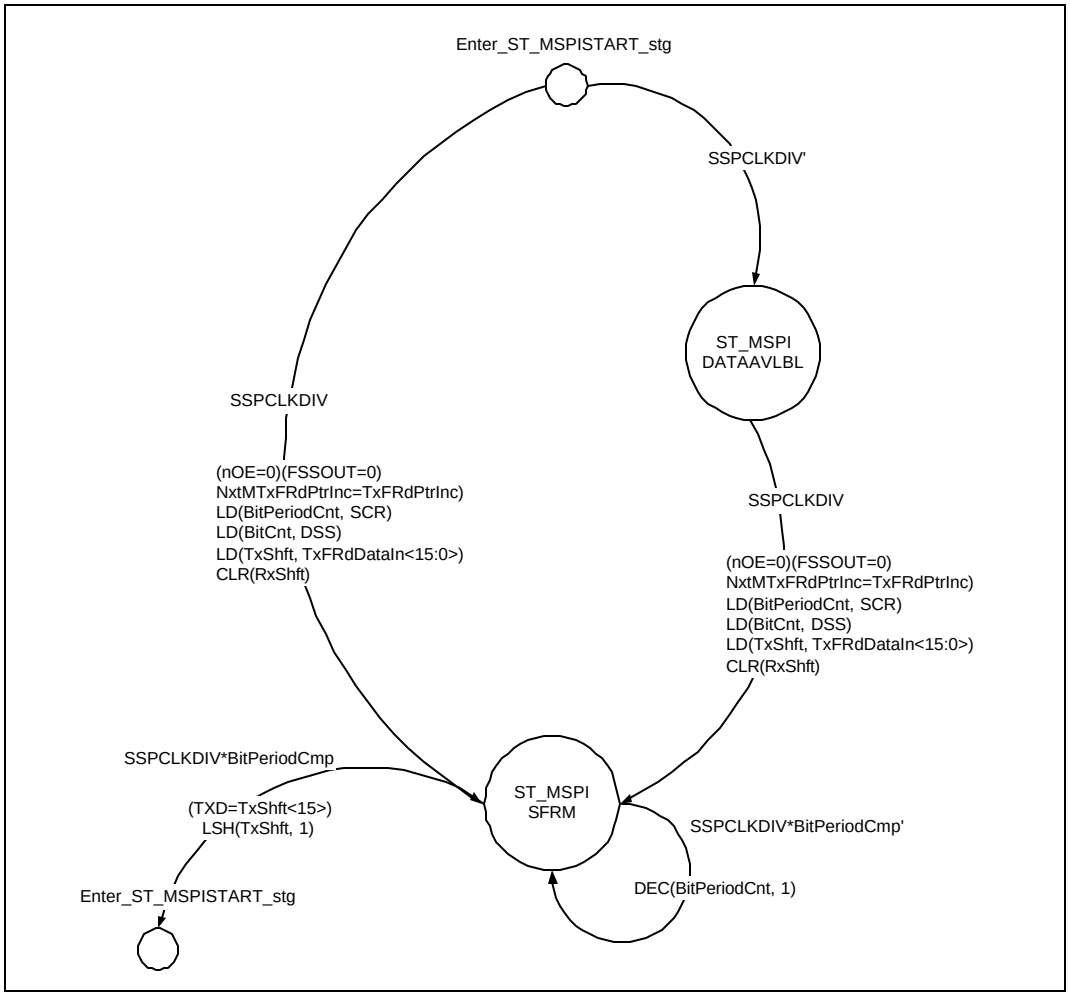

Figure 5.13-9 State diagram for the ST_MSPISTART hierarchical state

The state machine then enters the ST_MSPIBITS hierarchical state. ***Figure 5.13-10*** illustrates the state diagram for the ST_MSPIBITS hierarchical state. In this state machine, the first half of the frame is handled by the ST_MSPITX1 and the ST_MSPIRX1 states. The ST_MSPILASTBITS hierarchical state handles the transmission / reception of the last data bit in the frame.

One SSPCLKOUT phase time after SSPFSSOUT is pulled low, the state machine transitions to the ST_MSPITX1 state. The value to be driven on the SSPCLKOUT line is calculated as an XOR of the SPO and SPH inputs. The XOR operation is represented using the '!=' symbol in the state diagram. In the ST_MSPITX1 state, the state machine waits for one SSPCLKOUT phase time. After this SSPCLKOUT duration, the IntSSPRXD input is sampled and this data bit is shifted into the Receive shift register (RxShft[14:0]). The BitCnt[3:0] counter is decremented by 1 to indicate that one bit has been shifted out and one bit has been shifted in. The state machine then transitions to the ST_MSPIRX1 state where it waits for one SSPCLKOUT phase duration. At the end of this duration, the state machine again transitions to the ST_MSPITX1 state after shifting out the next bit onto the SSPTXD line and shifting the contents of the transmit shift register left by one bit position.

If half the frame has been completed as indicated by the assertion of the BitCmpHalf signal, the state machine transitions from the ST_MSPIRX1 state to the ST_MSPITX2 state shifting out the next transmit bit onto the SSPTXD output line. The state machine then shuttles between the ST_MSPITX2 state and the ST_MSPIRX2 state after every SSPCLKOUT phase time, each time shifting out a transmit data bit and shifting in a receive data bit. When the BitCnt[3:0] counter reaches zero, it indicates that all but the last bit in the frame have been

completed. The state machine then exits the ST_MSPIBITS hierarchical state and transitions to the ST_MSPILASTBITS hierarchical state.

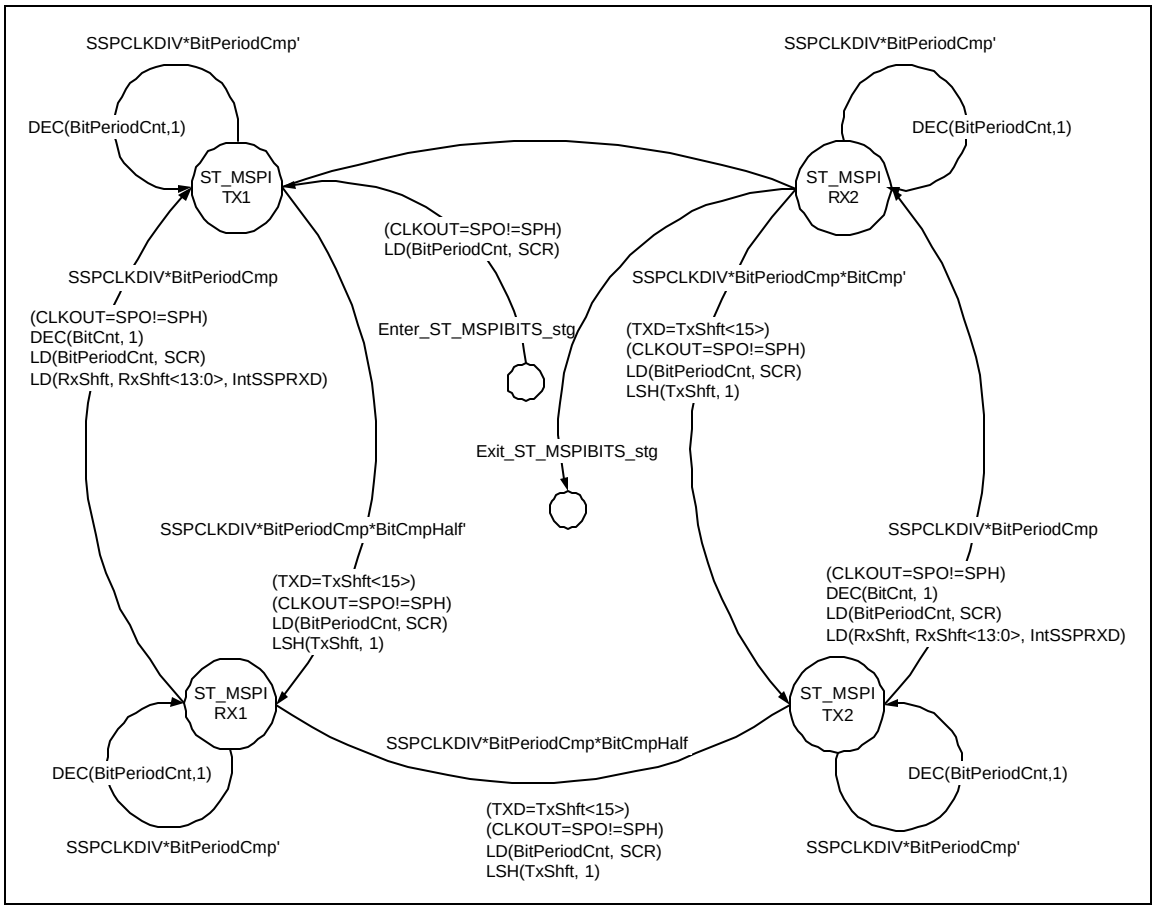

Figure 5.13-10 State diagram for the ST_MSPIBITS hierarchical state

Figure 5.13-11 illustrates the state diagram for the ST_MSPILASTBITS hierarchical state. After shifting out the last transmit bit onto the SSPTXD line, the state machine transitions to the ST_MSPILASTTX state. After a wait of one SSPCLKOUT phase duration in this state, the last receive data bit is sampled on the IntSSPRXD input line and the state machine transitions to the ST_MSPILASTRX state. After one SSPCLKOUT phase duration, the state machine then transitions out of the ST_MSPILASTBITS hierarchical state.

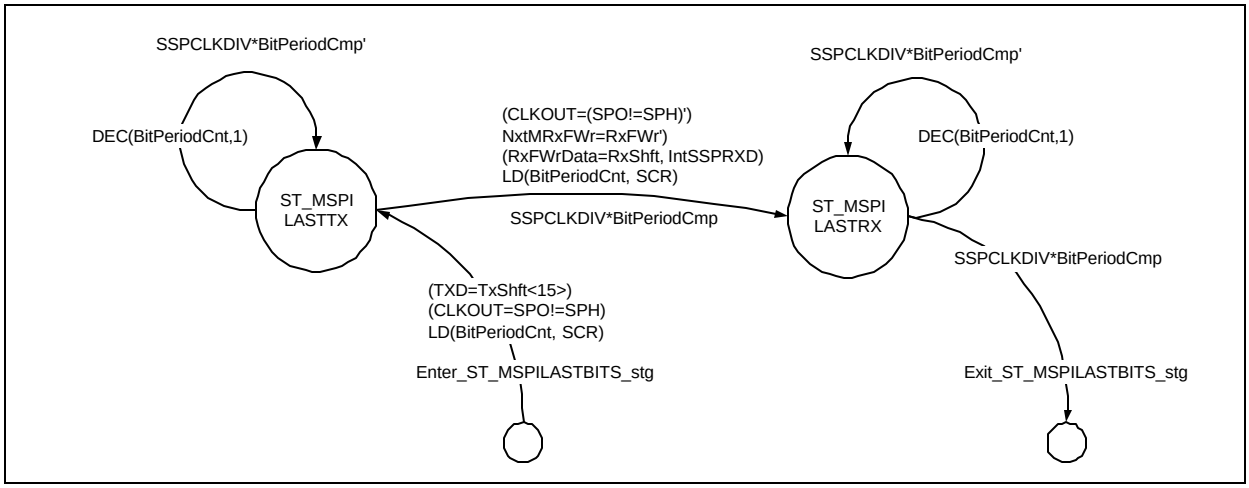

Figure 5.13-11 State diagram for the ST_MSPILASTBITS hierarchical state

If further data is available for transmission and if SPH is set to 0, the state machine transitions to the ST_MSPISFRMDEASST hierarchical state. The state diagram for the ST_MSPISFRMDEASST hierarchical state is shown in ***Figure 5.13-12***. The state machine waits in the ST_MSPISD1 state for one SSPCLKOUT phase duration and then pulls SSPFSSOUT high. Then it transitions to the ST_MSPISD2 state where the state machine waits for one SSPCLKOUT phase time. At the end of this wait, the state machine exits from the ST_MSPISFRMDEASST hierarchical state. The negation of SSPFSSOUT is not required in the case when SPH is set to 1 as per the specifications.

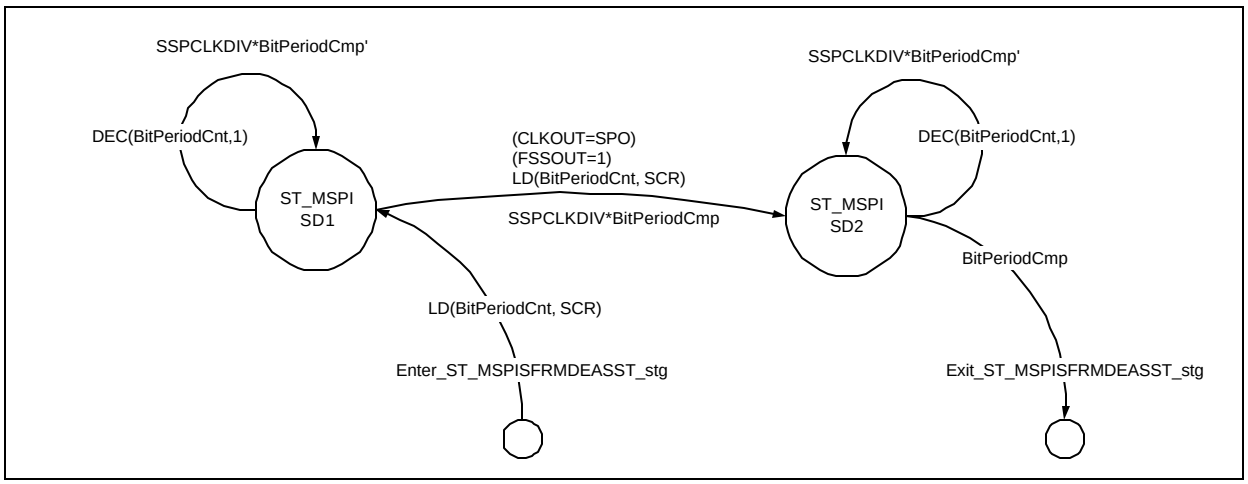

Figure 5.13-12 State diagram for the ST_MSPISFRMDEASST hierarchical state

When TxDataAvlblSync is asserted and the SSP is enabled, transmit data is copied into the transmit shift register. SSPFSSOUT is then pulled low to indicate the start of the frame. The first half of the frame is transmitted / received with the state machine shuttling between the ST_MSPITX1 and the ST_MSPIRX1 states. The remaining part of the frame is then transmitted / received with the state machine transitioning to the ST_MSPITX2 and the ST_MSPIRX2 states. After a data character has been fully received, it is copied into the receive buffer

MRxFWrData[15:0] and the RxFWr output is toggled to indicate that valid data is available to be written into the receive FIFO.

5.13.4 National Microwire Frame Format

The section of the SspMTxRxCntl state machine that handles the National Microwire frame format is illustrated in ***Figure 5.13-14***. The ST_MNMIDLE state is the idle state for the National Microwire frame format. The state machine remains in this state till valid data is available in the transmit FIFO as indicated by the assertion of the TxDataAvlblSync signal, provided the frame format is not changed. If the frame format is reprogrammed, the state machine transitions to the idle state corresponding to the new programmed frame format.

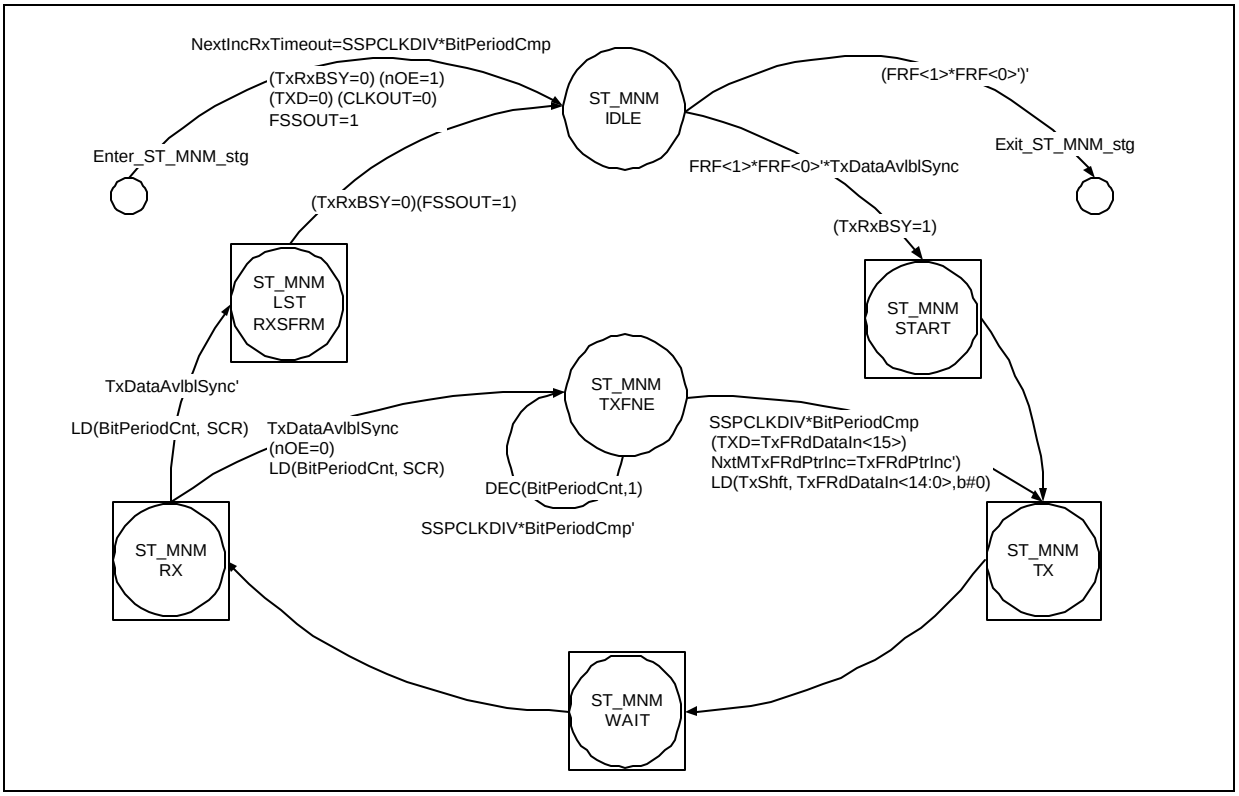

Figure 5.13-14 A section of the SspMTxRxCntl state machine that handles the National Microwire Frame Format

When the TxDataAvlblSync signal is detected, the state machine enters the ST_MNMSTART hierarchical state where SSPFSSOUT and nSSPOE are pulled low. After waiting for one SSPCLKOUT phase duration, the state machine enters the ST_MNMTX hierarchical state where Transmit data is shifted out onto the SSPTXD output. Then the state machine transitions to the ST_MNMWAIT hierarchical state that implements the one SSPCLKOUT duration gap between transmission and reception. The state machine then transitions to the ST_MNMRX hierarchical state where receive bits on the IntSSPRXD input are sampled and shifted into the receive shift register. At the end of reception, if further data is available in the transmit FIFO as indicated by the assertion of the TxDataAvlblSync signal, the state machine transitions to the ST_MNMTXFNE state and after waiting for one SSPCLKOUT phase time, transitions to the ST_MNMTX hierarchical state. If, at the end of the receive operation, no further data is available in the transmit FIFO, the state machine transitions to the ST_MNMLSTRXSFRM state where the state machine waits for one SSPCLKOUT duration before negating SSPFSSOUT and nSSPOE.

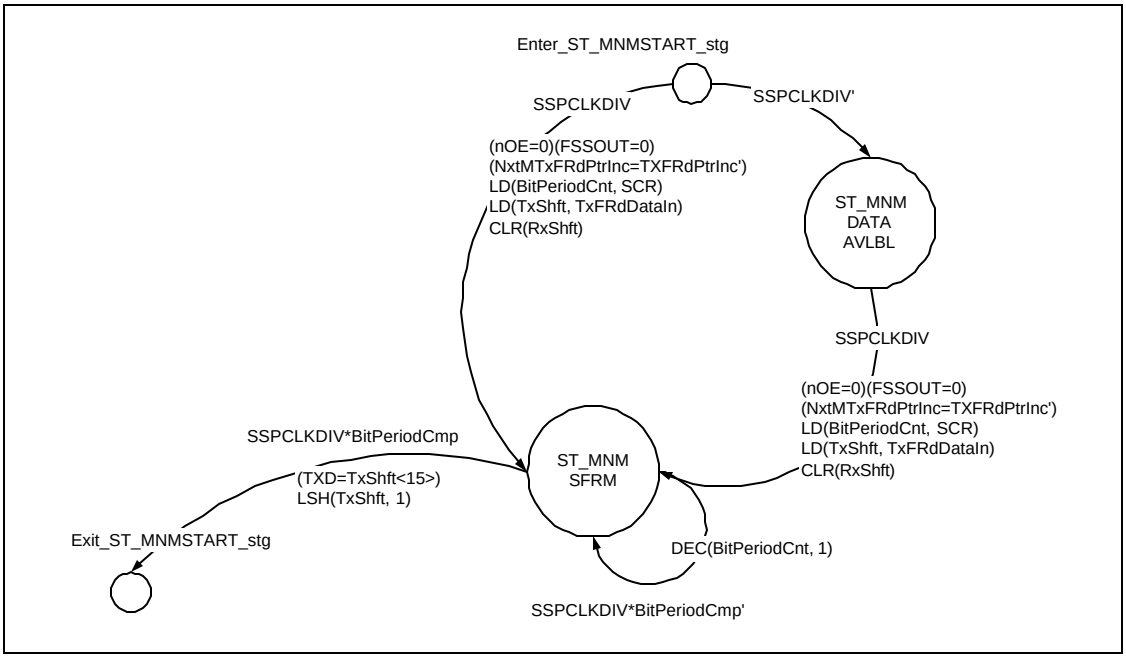

Figure 5.13-15 State diagram for the ST_MNMSTART hierarchical state

The state diagram for the ST_MNMSTART hierarchical state is shown in ***Figure 5.13-15***. The assertion and de-assertion of SSPTXD, nSSPOE and SSPCLKOUT are in sync with pulses on the SSPCLKDIV input. If SSPCLKDIV is not asserted, the state machine waits in the ST_MNMDATAAVLBL state for the next SSPCLKDIV pulse. When SSPCLKDIV is asserted, nSSPOE and SSPFSSOUT are pulled low. Data on the TxFRdDataIn[15:0] input is copied into the Transmit Shift register (TxShft[15:0]) and the TxFRdPtrInc output is toggled to indicate to the transmit FIFO that the read pointer can be incremented. The state machine then enters the ST_MNMSFRM state where it waits for one SSPCLKOUT duration. At the end of this time duration, the most significant bit in the transmit shift register is clocked out onto the SSPTXD line and the contents of the transmit shift register are shifted left by one bit position and the state machine transitions out of the hierarchical state.

Figure 5.13-16 illustrates the state diagram for the ST_MNMTX hierarchical state. When entering the ST_MNMTXBIT state, SSPCLKOUT is pulled low, the BitPeriodCnt[7:0] counter is loaded with SCR[7:0] and the BitCnt[3:0] counter is loaded with 7. The load value for the BitCnt[3:0] counter is 7 because transmit data is always 8-bits wide in this frame format. After waiting for one SSPCLKOUT phase time, the state machine transitions to the ST_MNMTXBIT2 state after pulling SSPCLKOUT high and reloading the BitPeriodCnt[15:0] counter. One SSPCLKOUT phase time later, the state machine transitions back to the ST_MNMTXBIT state after shifting out the most significant bit in the transmit shift register onto the SSPTXD line and shifting the contents of the transmit shift register left by one bit position. The BitCnt[3:0] counter is also decremented by 1 to indicate that a data bit has been shifted out. When all 8 bits in the transmit data have been shifted out, the state machine pulls SSPTXD low and transitions out of the ST_MNMTX hierarchical state. nSSPOE is negated after one SSPCLKOUT phase time in the ST_MNMWAIT hierarchical state.

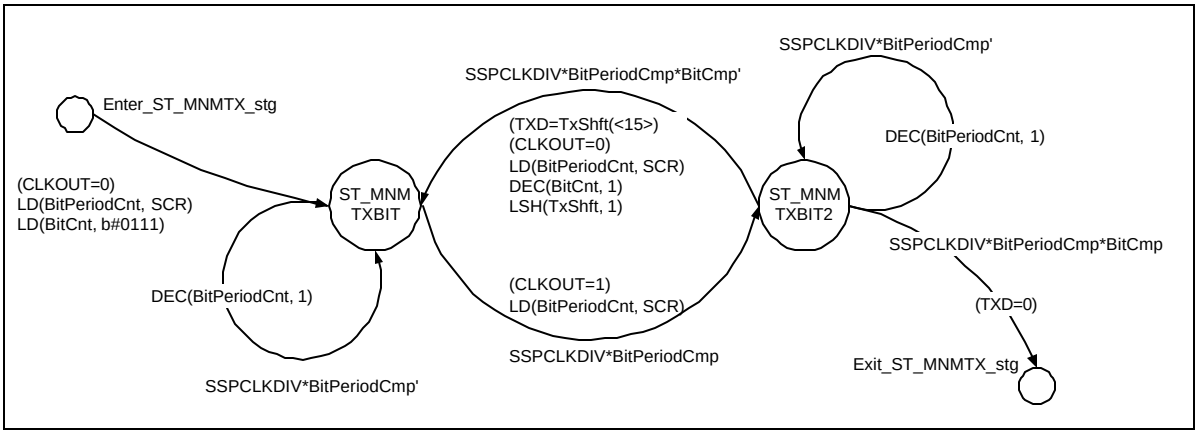

Figure 5.13-16 State diagram for the ST_MNMTX hierarchical state

The state diagram for the ST_MNMWAIT hierarchical state is illustrated in ***Figure 5.13-17***. When entering the ST_MNMWAIT1 state, the BitPeriodCnt[7:0] counter is reloaded with SCR[7:0]. The state machine remains in this state for a time duration corresponding to one SSPCLKOUT phase time. At the end of the phase time, nSSPOE is pulled low and the BitPeriodCnt[7:0] counter is reloaded with SCR[7:0]. The state machine then transitions to the ST_MNMWAIT2 state where the state machine waits for one SSPCLKOUT phase time before exiting from the ST_MNMWAIT hierarchical state. The gap required between transmission and reception, of one SSPCLKOUT duration, is now complete. This is because this frame format is half-duplex although transmission and reception do not occur through the same wire. This is in contrast with the other two frame formats where these signals are negated midway through a frame. The other two formats are full-duplex to the extent that transmission and reception occur simultaneously.

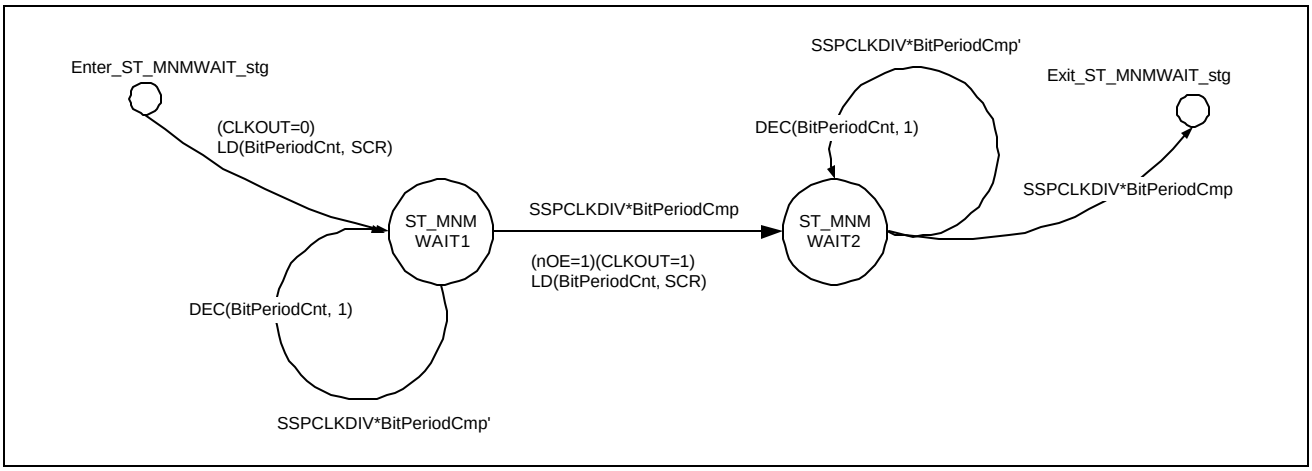

Figure 5.13-17 State diagram for the ST_MNMWAIT hierarchical state

From the ST_MNMWAIT hierarchical state, the state machine transitions into the ST_MNMRX hierarchical state. The state diagram for the ST_MNMRX state is shown in ***Figure 5.13-18***. When the state machine enters the ST_MNMRX1 state, SSPCLKOUT is pulled low and the BitCnt[3:0] counter is loaded with DSS[3:0]. After waiting for a duration of one SSPCLKOUT phase time, SSPCLKOUT is pulled high and the value on the IntSSPRXD input is shifted into the receive shift register. The state machine then transitions to the ST_MNMRX2 state where it waits for one SSPCLKOUT phase time. Then the state machine transitions back to the ST_MNMRX1 state after decrementing the BitCnt[3:0] counter by 1. When in the ST_MNMRX1 state, if the BitCnt[3:0] counter has reached a count of 0, it indicates that the programmed number of bits have been shifted into the receive shift register and the state machine transitions out of the ST_MNMRX hierarchical state.

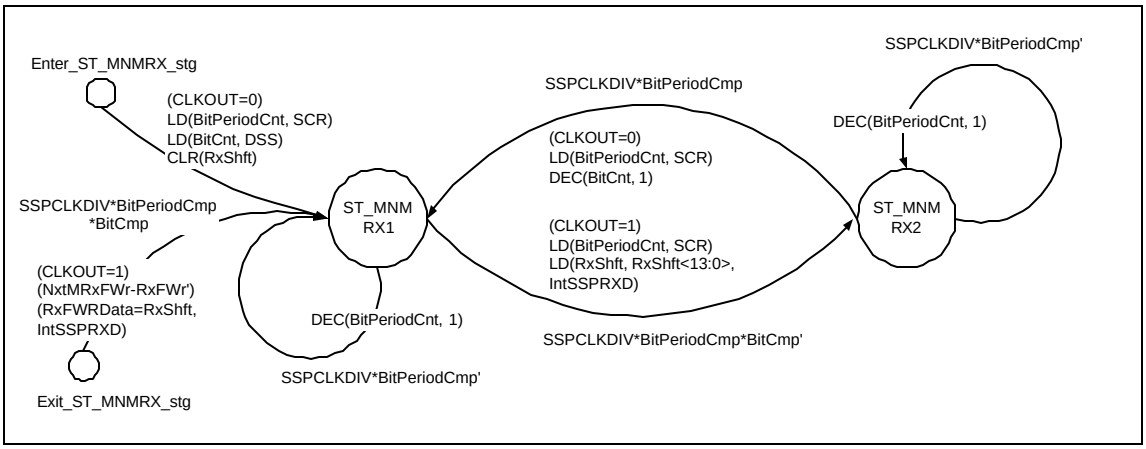

Figure 5.13-18 State diagram for the ST_MNMRX hierarchical state

The state diagram for the ST_MNMLSTRXSFRM state is shown in ***Figure 5.13-19***. The state machine transitions into this hierarchical state after the current frame has been received completely and the TxDataAvlblSync signal is not asserted, thereby indicating that there is no further data available in the transmit FIFO for transmission. After the state machine transitions to the ST_MNMRXLAST state, it waits for a time duration corresponding to one SSPCLKOUT phase time. SSPCLKOUT is then pulled low for the last time. The state machine then transitions to the ST_MNMSFRMLAST state where it waits for one SSPCLKOUT phase duration before transitioning out of the ST_MNMLSTRXSFRM state. The SSPFSSOUT output is then pulled high to signal the end of the frame.

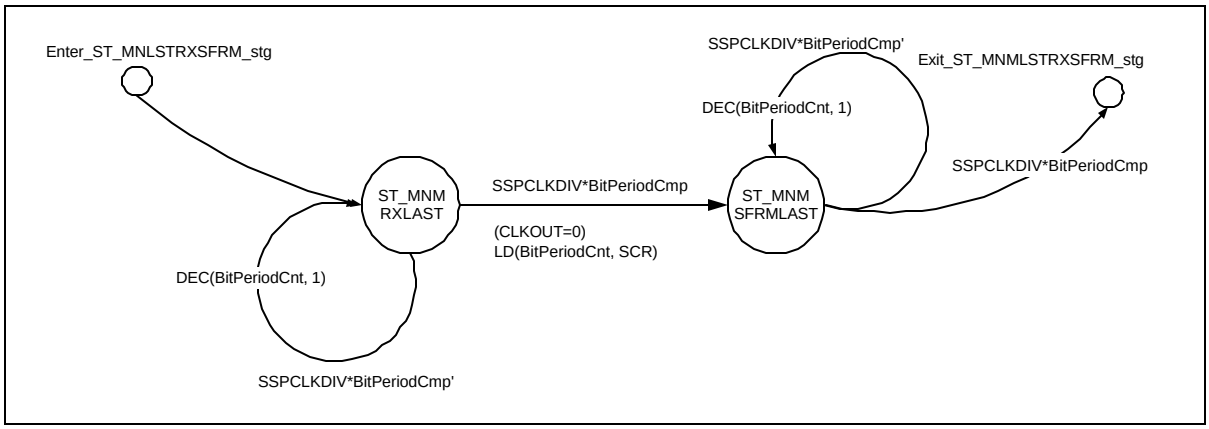

Figure 5.13-19 State diagram for the ST_MNMLSTRXSFRM state

Figure 5.13-20 illustrates waveforms for the start of transmission in the National Microwire Frame format. When the TxDataAvlblSync signal is asserted, transmit data is copied into the transmit shift register and SSPFSSOUT is pulled low. After a wait of one SSPCLKOUT phase duration in the ST_MNMSFRM state, transmission starts with the state machine shuttling between the ST_MNMTXBIT and the ST_MNMTXBIT2 states with every SSPCLKOUT phase time. After every SSPCLKOUT period, the BitCnt[3:0] counter is decremented by 1. One SSPCLKOUT phase time after the completion of transmission, nSSPOE is negated as shown in ***Figure 5.13-20***. ***Figure 5.13-21*** illustrates the reception part of the frame. After a one SSPCLKOUT gap between the end of transmission and the start of reception, the BitCnt[7:0] counter is loaded with DSS[3:0], which corresponds to the number of bits in the receive data. The state machine shuttles between ST_MNMRX1 and ST_MNMRX2 after each SSPCLKOUT phase duration while shifting in receive data into the receive shift register. At the end of reception, the receive

buffer MRxFWrData[15:0] is filled with the received data and the RxFWr output is toggled to indicate to the receive FIFO that valid data is available for clocking into the receive FIFO.

5.14 Slave Transmit / Receive Control state machine (SspSTxRxCntl)

The Slave Transmit / Receive control state machine controls the shifting-out of the transmit data and the shifting-in of the receive data in the slave mode. This block also controls the nSSPOE output in accordance with the programmed transmission parameters.

This block operates on SSPCLK. The SSESync input to this block is the SSPCLK-synchronised version of the SSE bit in the SSPCR1 register. The SSPCLKDIV input is the output of the SspScaleCntr (SSP Clock Pre-Scale Counter) block. The input signals that indicate the transmission / reception parameters are DSS[3:0] (Data Size Select), FRF[1:0] (Frame Format), SCR[7:0] (Serial Clock Rate), SPO (SSPCLKIN Polarity for the Motorola SPI frame format) and SPH (SSPCLKIN Phase for the Motorola SPI frame format). The TxFRdDataIn[15:0] input is the transmit data from the Transmit FIFO. The IntSSPRXD input is the serial receive input.

Activity on the SSPCLKIN signal is monitored by continuously checking for edge transitions. On detection of either a positive or negative edge transition, then the signal SRxRT is asserted. This signal is used to reload the receive timeout counter in the SSPDataStp block. The output signal IncRxTimeout, from the SspMTxRxCntl block, is the enable to this receive timeout counter. If there is data remaining in the receive FIFO and the timeout counter reaches a value of zero, which takes 32 bit periods, then the receive timeout interrupt SSPRTINTR, if not masked, is asserted. This is to alert the system that data still remains and should be read from the receive FIFO.

In the state diagrams in this section, circles represent normal states and circles with a bounding square represent hierarchical states. A Hierarchical state is a representation of a group of states that are functionally related. These hierarchical states, however, do not represent any further block hierarchy within the SspSTxRxCntl block.

The state machine is split into three main divisions corresponding to the three frame formats – TI Synchronous Serial Frame Format, Motorola SPI Frame Format and National Microwire Frame Format. ***Figure 5.14-1*** illustrates the overall structure of the state machine. On reset, the state machine enters the ST_SSPIIDLE state after initialising the internal counters, shift registers, buffers and the block outputs.

In TI mode, the TxFRdPtrInc signal is toggled at the first rising edge of SSPCLKIN after the SSPFSSIN pulse as it denotes the start of a new frame. In the NM mode, this signal is toggled when the slave transmits the MSB onto the SSPTXD output. In the SPI mode, the signal is toggled at the next edge of SSPCLKIN after the MSB of a new frame has been transmitted out.

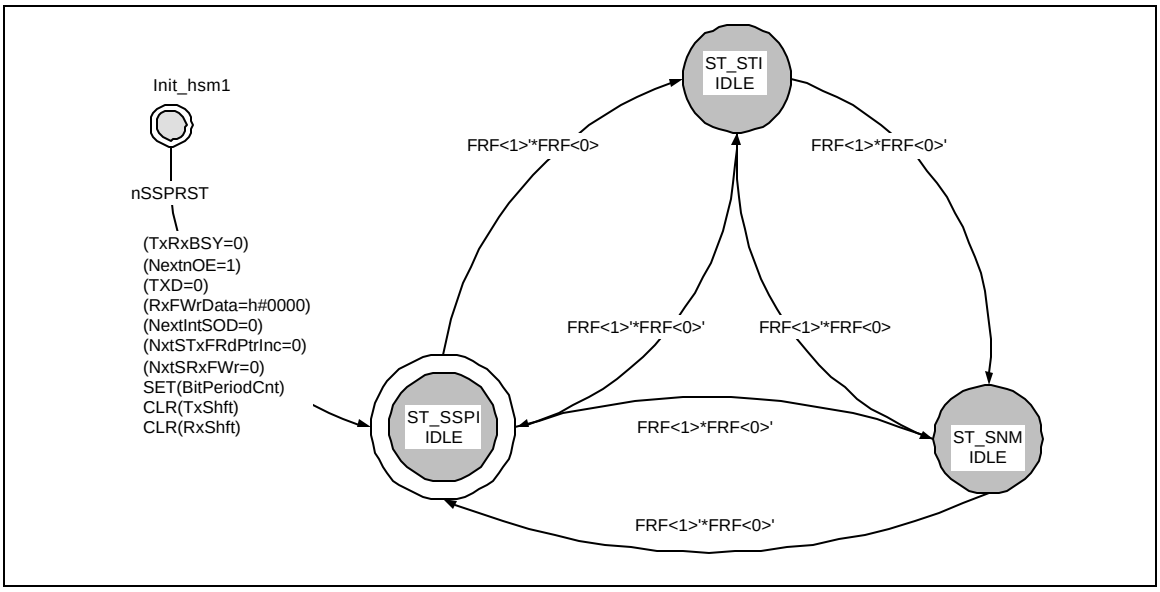

Figure 5.14-1 Three sections of the SspSTxRxCntl state machine

When the SSP is disabled i.e. when the SSESync signal is negated, the state machine transitions to one of the idle states ST_STIIDLE, ST_SNMIDLE or ST_SSPIIDLE based on the frame format programmed in the FRF[1:0] bits. Each of these hierarchical states contains the part of the state machine corresponding to one specific frame format. When the state machine is in the idle state of one specific frame format and the FRF[1:0] bits are re-programmed to require a different frame format, the state machine transitions to the hierarchical state corresponding to the new frame format. The nSSPOE output is intended to be used as the output enable control for the pad associated with the SSPTXD output. To account for pad delays, wherever possible, nSSPOE is asserted one SSPCLKIN phase time before SSPTXD is required to be valid and is negated one SSPCLKIN phase time after SSPTXD is no longer required to be valid.

5.14.1 Exception Handling

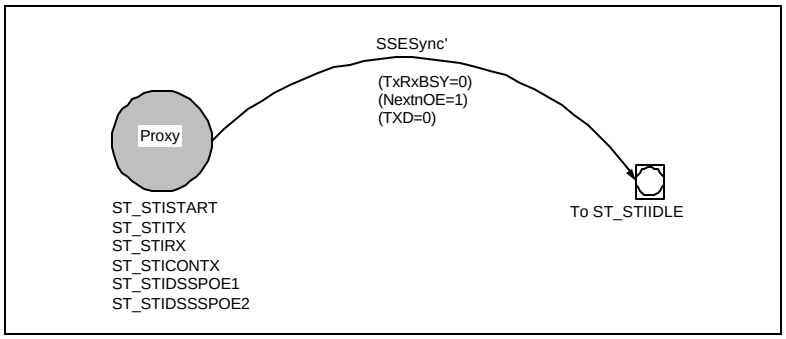

Figure 5.14-2 Exception handling in the SspSTxRxCntl in TI Frame Format

If the SSP is disabled i.e. the SSESync input is negated, the frame being transmitted / received is aborted and the state machine returns to the ST_STIIDLE, ST_SNMIDLE or ST_SSPIIDLE state (based on the programmed frame format) on the next SSPCLK after initialisation.

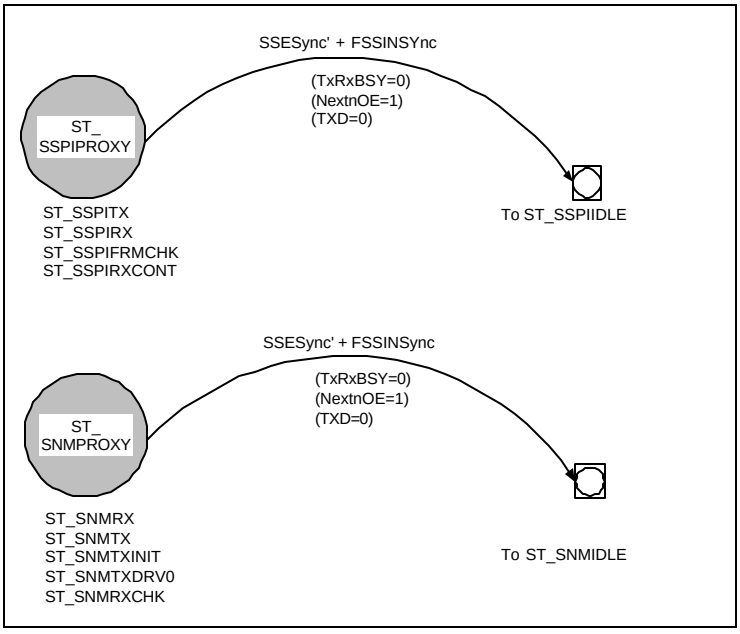

Figure 5.14-3 Exception handling in the SspSTxRxCntl in NM and SPI Frame Format

Similarly, when the SSP is in programmed for the Motorola SPI frame format or the National Microwire frame format and if SSPFSSINSync is negated midway through a frame, the frame being transmitted / received is aborted and the state machine returns to the ST_SSPIIDLE or ST_NMIDLE state (based on the programmed frame format) on the next SSPCLK after initialisation. The TxRxBSY, SSPTXD and nSSPOE signals are also forced to their default values. The state diagrams in ***Figures 5.14-2*** and ***5.14-3*** illustrate this operation.

5.14.2 Texas Instruments' Synchronous Serial Frame Format

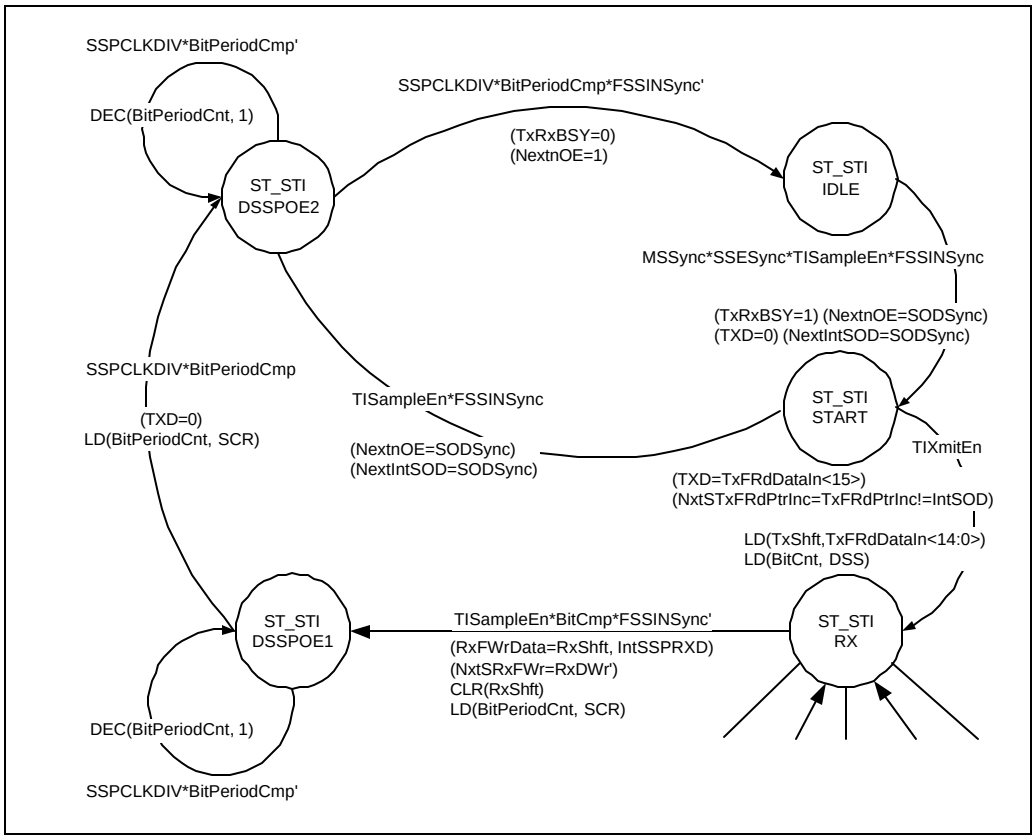

Figure 5.14-4 A section of the SspSTxRxCntl state machine for the TI Synchronous Serial Frame Format

One section of the state machine that handles TI Synchronous Serial Frame Format is shown in ***Figure 5.14-4***. In this frame format, transmit data is driven on the rising edge of SSPCLKINSync and receive data is sampled on the falling edge of SSPCLKINSync. The ST_STIIDLE state is the idle state for the section of the state machine that handles this frame format. Whenever a negative edge is detected on the SSPCLKINSync signal, the TISampleEn signal is set for one SSPCLK duration. The TIXmitEn signal is generated similarly on every positive edge of the SSPCLKINSync signal. When SSPFSSINSync signal is sampled asserted on a falling edge of SSPCLKINSync (i.e. SSPFSSINSync is high when TISampleEn is high), it denotes that the SSP master has started the transmission of a new frame. When the SSP is programmed as a slave, on detecting the start of a new frame, the state machine transitions to the ST_STISTART state after setting TxRxBSY to indicate that the transmit / receive controller is now busy. SOD status is clocked into the IntSOD signal and if SOD is set, nSSPOE is pulled low. The SSPTXD signal is also driven low as it is the default value. In the ST_STISTART state, when the TIXmitEn signal is set, the SSP starts transmitting the MSB of the transmit data and loads the transmit shift register with the lower 15 bits of the transmit data. It also loads the bit counter with the DSS value. The TxFRdPtrInc output is toggled to let the Transmit FIFO increment the Read pointer. The bit counter is decrement by 1 after transmitting each bit. Transmission and reception proceeds till the counter reaches zero.

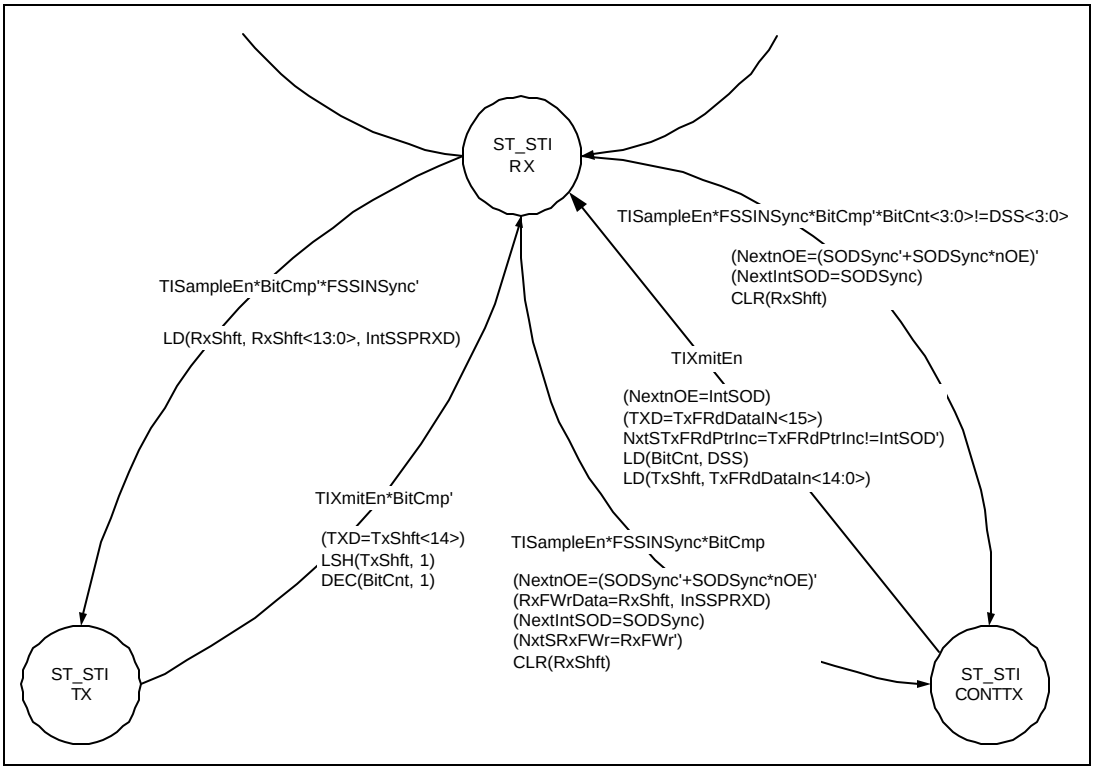

Figure 5.14-5 A section of the SspSTxRxCntl state machine for the TI Synchronous Serial Frame Format

On completion of the start phase, the state machine transitions to the ST_STIRX state and waits for the SSPFSSINSync signal to go low. Then, on a falling edge of SSPCLKINSync (TISampleEn asserted), the serial data input on the IntSSPRXD pin is sampled and shifted into the Receive Shift register. The state machine then transitions to the ST_STITX state where the current data to be transmitted is shifted out bit by bit onto the SSPTXD output. The bit counter is decrement by 1 whenever a bit is transmitted. The state machine then transits back to ST_STIRX state to receive the next bit of data. ***Figure 5.14-5*** illustrates the transition from the transmit state to the receive state and back in the TI frame format in slave mode.

The last receive data bit on the IntSSPRXD input is sampled and clocked into the LSB (SRxFWrData[0]) of the receive buffer. The contents of the receive shift register are copied into the upper 15 bits of the receive buffer SRxFWrData[15:1]. The RxFWr signal is toggled to indicate to the Receive FIFO that valid data is available on the SRxFWrData[15:0] bus to be clocked into the Receive FIFO. The receive shift register is then cleared so that it can start receiving new data. The Receive Buffer holds its contents till the end of the next frame, allowing sufficient time for the FIFO write control signal to be synchronised to the PCLK domain and for the contents of the Receive buffer to be written into the Receive FIFO.

While receiving the last data bit, if the SSPFSSINSync input signal is asserted, it indicates that the master is in Continuous transfer mode and is about to start a new transfer. Hence, the state machine transitions to the ST_STICONTX state to transmit the MSB of the next frame. If SSPFSSINSync remains cleared, it indicates that the master does not have any more data to transmit at present and hence the state machine transitions to ST_STIDnSSPOE hierarchical state to negate nSSPOE. If the SSPFSSINSync signal is sampled asserted anywhere in the middle of a frame, the current transmission/reception is aborted and the state machine transitions to ST_STICONTX state to start a new frame. On this state transition, the receive shift register is cleared and SOD status is clocked into the IntSOD signal which determines whether nSSPOE is to be asserted or not during the

transmission/reception of the next frame. If nSSPOE has to be negated (SOD set), then it is done after a duration corresponding to one SSPCLKIN phase duration. Otherwise if it has to be asserted (SOD cleared), then it is asserted immediately. In the ST_STICONTX state, the MSB of the next frame is transmitted on detecting a rising edge on SSPCLKINSync (TIXmitEn asserted). The transmit shift register is loaded with the remaining lower order bits of the new frame. The TxFRdPtrInc signal is toggled to indicate to the Transmit FIFO that its read pointer can be incremented. nSSPOE is asserted or negated depending upon the IntSOD signal and the bit counter is loaded with DSS. The state machine then transitions to the ST_STIRX state and transmission / reception proceeds till the counter counts down to zero.

If transmission / reception is over, then nSSPOE is negated one SSPCLKIN period after the last data bit is sampled. To time this duration of one SSPCLKIN period, the state machine transitions to the ST_STIDnSSPOE1 state and then to the ST_STIDnSSPOE2 state. On the transitions into each of these states, the BitPeriodCnt[7:0] counter is loaded with the value on the SCR[7:0] input. This counter is used to time the duration of one SSPCLKIN phase. The counter is decrement with every pulse on the SSPCLKDIV input and when it reaches zero, one SSPCLKIN phase time is said to have elapsed. After one SSPCLKIN period has elapsed, the state machine transitions to ST_STIIDLE state. In the ST_STIDnSSPOE1 state and the ST_STIDnSSPOE2 state, the state machine checks whether SSPFSSINSync is asserted in between to denote the start of a new frame. If it detects a new frame, then the SODSync signal is clocked into the IntSOD signal and the state machine transitions to the ST_STISTART state to transmit new data.

5.14.3 Motorola’s SPI Frame Format

The section of the SspSTxRxCntl state machine that handles the Motorola SPI Frame format is illustrated in ***Figure 5.14-7***. The ST_SSPIIDLE state is the idle state for this section of the state machine. The state machine remains in this state if SSPFSSINSync remains high. While in the ST_SSPIIDLE state, if the frame format is changed, the state machine transitions to the idle state corresponding to the new programmed frame format. When the SSPFSSINSync signal goes low denoting the start of a new transmission, the state machine asserts the TxRxBSY signal and transitions to the ST_SSPIRXCONT state. The SODSync signal is clocked into the IntSOD signal and the nSSPOE signal is asserted / negated depending on SODSync status. The SSP starts transmitting the MSB of the transmit data onto the SSPTXD line and the transmit shift register is loaded with the remaining lower order bits of transmit data. The bit counter is initialized with the DSS value. Depending on the programmed values of SPO and SPH, the SPISampleEn and SPIXmitEn signals are set for one SSPCLK duration at the positive and negative edge respectively, of the SSPCLKINSync signal. Thus when SPISampleEn is set, the state machine is expected to sample the IntSSPRXD signal and when SPIXmitEn is set, the state machine is expected to transmit the next data bit.

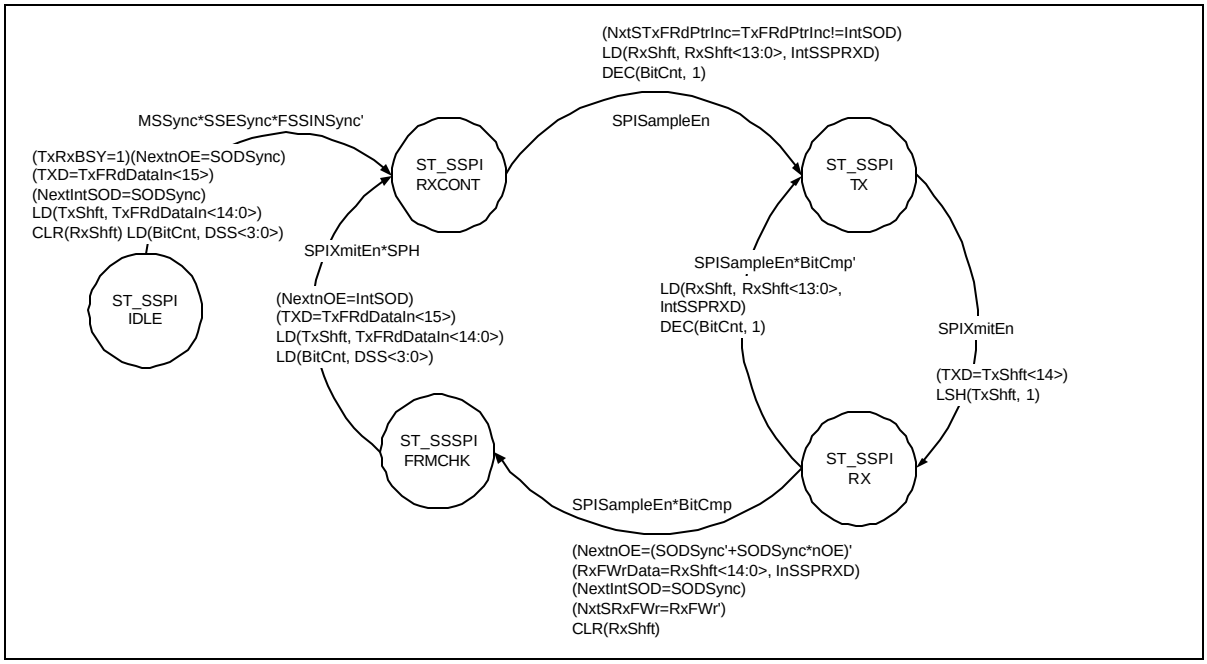

Figure 5.14-7 A section of the SspSTxRxCntl state machine that handles the Motorola SPI Frame Format

In the ST_SSPIRXCONT state, if SPISampleEn is detected set, then the serial data input on the IntSSPRXD pin is sampled and shifted into the Receive Shift register. The bit counter is decrement by 1 and the FIFO read pointer increment signal to the TXFIFO is toggled depending on the IntSOD signal. The Read pointer increment indication to the Transmit FIFO is done after the first receive data bit is sampled and not immediately after the MSB is transmitted. This is to ensure that any spurious low detected on the SSPFSSINSync input does not lead to an inadvertent increment of the TX FIFO read pointer and the consequent loss of transmit data. The delayed toggling of the TxFRdPtrInc signal also enables the SSP Slave to support two different kinds of SPI Masters in the Continuous mode – those that do negate SSPFSSIN for one SSPCLKIN phase time between successive frames and those Masters that do not negate SSPFSSIN between successive frames.

The state machine then transitions to the ST_SSPITX state where the current data to be transmitted is shifted out bit by bit onto the SSPTXD output. The state machine then transits to the ST_SSPIRX state where the bit counter is decrement by 1 whenever a new bit of the frame is received. The sampled bit is shifted into the receive shift register. When the bit counter reaches zero and when SPISampleEn is set, the last receive data bit on the IntSSPRXD input is sampled and clocked into the LSB of receive buffer (SRxFWrData[0]). The contents of the receive shift register are copied into the upper 15 bits of the receive buffer SRxFWrData[15:1]. The RxFWr signal is toggled to indicate to the Receive FIFO that valid data is available on the SRxFWrData[15:0] bus to be clocked into the Receive FIFO. The receive shift register is then cleared so that the next data frame can be received. The Receive Buffer holds its contents till the end of the reception of next frame, allowing sufficient time for the FIFO write control signal to be synchronised to the PCLK domain and for the contents of the Receive buffer to be written into the Receive FIFO. The state machine then transitions to the ST_SSPIFRMCHK state.

In the ST_SSPIFRMCHK state, if the SPIXmitEn signal is set and if the SSPFSSINSync signal remains low with SPH set, then it denotes the start of a new frame. The MSB of the new data is shifted out onto the SSPTXD output and the other bits of the new frame are loaded into the transmit shift register. The nSSPOE signal is asserted or negated depending on the status of the IntSOD signal and the state machine then transitions to the ST_SPIRXCONT state to receive the MSB of the new frame. In the ST_SSPIFRMCHK state, if SPH is not set, then the state machine waits for the SSPFSSINSync signal to be negated to denote that the frame is over. Then, the state machine transitions to the ST_SSPIIDLE state. In the case of SPH = 0, even for continuous transmission, the state machine thus goes to the idle state and starts all over again.

5.14.4 National Microwire Frame Format

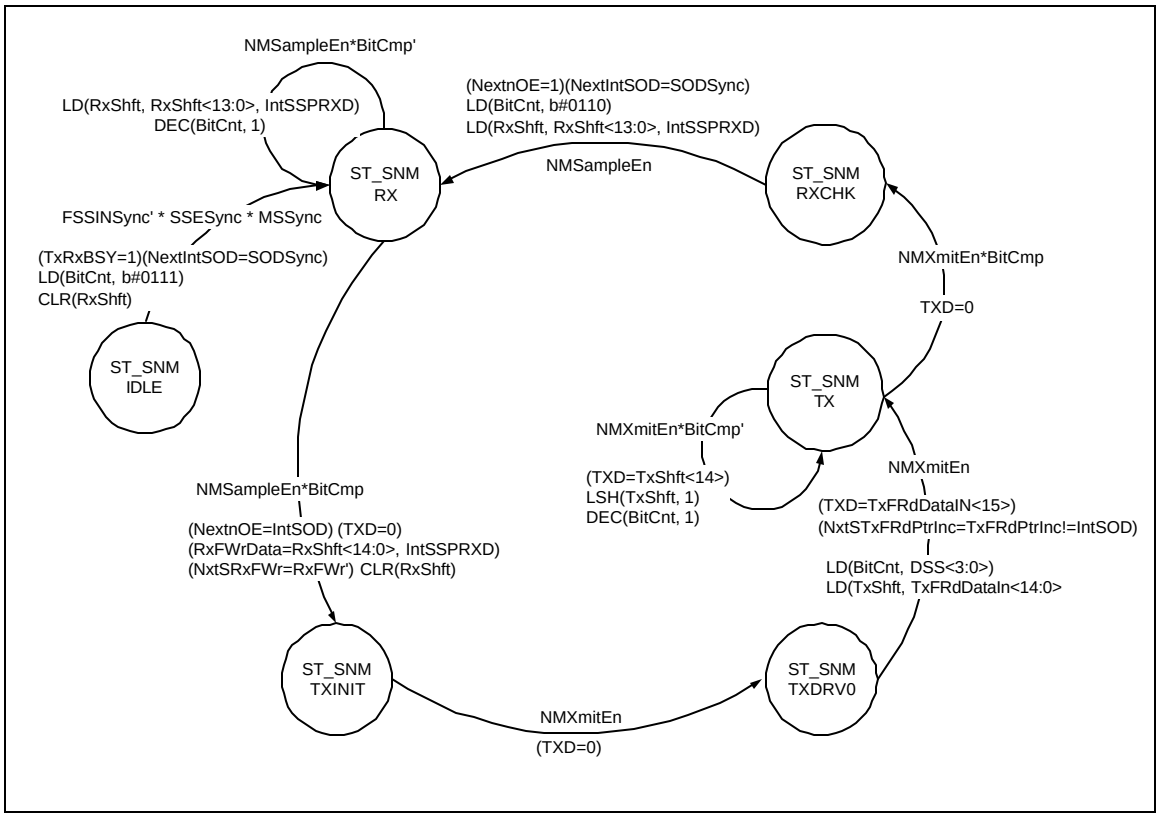

Figure 5.14-10 A section of the SspSTxRxCntl state machine for the National Microwire frame format

The section of the SspSTxRxCntl state machine that handles the National Microwire Frame format is illustrated in ***Figure 5.14-10***. In this frame format, transmit data is driven on the falling edge of SSPCLKINSync and receive data is sampled on the rising edge of SSPCLKINSync. The ST_SNMIDLE state is the idle state for the section of the state machine that handles this frame format. Whenever a falling edge is detected on the SSPCLKINSync signal, the NMSampleEn signal is set for one SSPCLK duration. Similarly on every rising edge of the SSPCLKINSync signal, the NMXmitEn signal is set for one SSPCLK duration. In the idle state, the state machine waits for SSPFSSINSync to go low denoting the start of a new frame. On detecting the start of a new frame, the state machine asserts the TxRXBSY output to indicate that the transmit / receive controller is now busy. SOD status is clocked into the IntSOD signal and if SOD is set, nSSPOE is pulled low during the period at which the slave would normally transmit. During the start of reception, the bit counter is loaded with 7 and the receive shift register is cleared. The state machine then transitions to the ST_SNMRX state to receive the MSB of the frame.

In ST_SNMRX state, the state machine waits for a rising edge on the SSPCLKINSync signal to sample the IntSSPRXD signal and decrement the bit counter by 1. It remains in this state till the last bit has been received i.e. the bit counter reaches zero. After the bit counter has reached zero, at the next rising edge of SSPCLKINSync, the IntSSPRXD input is sampled and clocked into the LSB (RxFWrData[0]) of the receive buffer. The contents of the receive shift register are copied into the upper 15 bits of the receive buffer RxFWrData[15:1]. The RxFWr signal is toggled to indicate to the Receive FIFO that valid data is available on the RxFWrData[15:0] bus to be clocked into the Receive FIFO. The receive shift register is then cleared so that it can start receiving new data. The Receive Buffer holds its contents till the end of the reception of next frame, allowing sufficient time for the FIFO write control signal to be synchronised to the PCLK domain and for the contents of the Receive buffer to be written into the Receive FIFO. It also asserts or negates the nSSPOE signal depending upon the status of the IntSOD signal. The state machine then transits to the ST_SNMTXINIT state and drives SSPTXD the default value of zero.

In the ST_SNMTXINIT state, the state machine waits for the NMXmitEn signal to be asserted (falling edge of SSPCLKINSync detected), after which, the SSPTXD output is driven low and the state machine transitions to the ST_SNMTXDRV0 state. The state machine remains in this state for a duration of one SSPCLKIN period to ensure that the first bit of transmit data is available on the SSPTXD output for one SSPCLKIN period. On the next falling edge of SSPCLKINSync, the MSB of the next frame is transmitted onto the SSPTXD output. The transmit shift register is loaded with the remaining lower order bits of the new frame. The TxFRdPtrInc signal is toggled depending on the IntSOD status to indicate to the Transmit FIFO that its read pointer can be incremented. The bit counter is loaded with the programmed DSS value and the state machine then transitions to the ST_SNMTX state. The current data to be transmitted is shifted out bit by bit onto the SSPTXD output. The bit counter is decrement by 1 whenever a bit is transmitted. The state machine remains in the ST_SNMTX state till the programmed number of data bits have been transmitted i.e. the bit counter reaches zero. The state machine then transits to ST_SNMRXCHK state to check whether the master is continuing with transmission / reception. If NMSampleEn is detected set with SSPFSSINSync low, then it indicates a new transfer in the continuous mode. In this case, SODSync status is clocked into the IntSOD signal and the nSSPOE signal is negated. The bit counter is loaded with 6 and the IntSSPRXD signal is sampled into the receive shift register. The state machine reverts back to ST_SNMRX state to receive the other bits of the frame. Otherwise, if the state machine finds the SSPFSSINSync negated when NMSampleEn is detected set, it concludes that transmission / reception is over and transits to the ST_NMIDLE state.

When the SSP is programmed to operate as a slave in the NM mode, after transmission is over, the state machine transits from the ST_SNMRXCHK state to the ST_SNMRX state before transiting to the ST_SNMIDLE state even in a Non-back to back transfer. The reason for this is as follows. After the last bit has been transmitted, if SSPFSSINSync is not negated before the next rising edge of SSPCLKINSync, a decision on whether the transmission is continuous or not cannot be taken. It is conservatively taken as continuous transmission. Hence the state machine transits to the ST_SNMRX state and starts receiving the next data bit. If SSPFSSINSync is subsequently negated, the state machine transits to ST_SNMIDLE state, aborting the reception.

In the NM mode, the SSP slave samples the first bit of the receive data on the rising edge of SSPCLKIN after SSPFSSIN has gone low. Masters that drive a free running SSPCLKIN should ensure that SSPFSSIN has sufficient setup and hold margins with respect to the rising edge of SSPCLKIN. ***Figure 5.14-13*** illustrates these setup and hold time requirements. With respect to the SSPCLKIN rising edge on which the first bit of receive data is to be sampled by the SSP slave, SSPFSSIN should have a setup of atleast 2 times the period of SSPCLK on which the SSP operates. With respect to the SSPCLKIN rising edge previous to this edge, SSPFSSIN should have a hold of atleast one SSPCLK period.

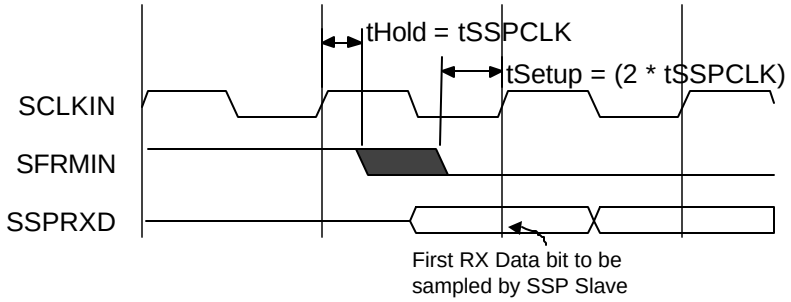

Figure 5.14-13 Setup and Hold time requirements on the SSPFSSIN input in the Microwire mode

5.15 Receive Timeout interrupt generation (SspDataStp)

This block is the source of the SSPRTINTR, the Receive Timeout Interrupt, which is asserted high when data is present in the receive FIFO, in master or slave mode, and there has been no clock activity for 32 bit periods, signifying that the data should really have been read.

In master mode, the SSPCLKOUT signal is monitored for activity, and if present, signified by assertion of the MRxRT signal , then the timeout count is reset by loading it with it's maximum value of 31. If the receive FIFO is not empty (RNESync asserted) and SSPCLKOUT activity is not present for 32 bit periods, then the DataStp signal is asserted.

Similarly, in slave mode, the SSPCLKIN signal is monitored for activity, and if present, signified by assertion of the SRxRT signal , then the timeout count is reset by loading it with it's maximum value of 31. If the receive FIFO is not empty (RNESync asserted) and SSPCLKIN activity is not present for 32 bit periods, then the DataStp signal is asserted.

The DataStp interrupt signal can be reset to it's maximum value by writing to the RTIC bit within the SSPICR register, reading the contents of the receive FIFO until empty or if new activity occurs on the respective clock lines.

5.16 SSP Interrupt generator (SspIntGen)

This block generates the SSPINTR, which is a combined SSP interrupt. This interrupt is generated by ORing the individual interrupts from the SSP, namely the Receive FIFO Service Request Interrupt (SSPRXINTR), the Transmit FIFO Service Request Interrupt (SSPTXINTR), the Receive Timeout Interrupt (SSPRTINTR) and the Receive FIFO Overrun Interrupt (SSPRORINTR). The SSPINTR signal can be considered as being an asynchronous signal since it can be generated from actions from either the SSPCLK or PCLK domains, which may themselves be asynchronous. All interrupts are cleared in the PCLK domain, either dynamically as in the case of the transmit and receive FIFOs or by writing to the appropriate bit within the SSPICR interrupt clear register. This strategy ensures that the interrupts are removed such that the interrupt software routines will not recalled as a consequence of domain synchronising latencies.

5.17 DMA Interface (SspDMA)

This block provides an interface to connect to a DMA controller. The DMA operation of the SSP is controlled through the SSP DMA Control Register (SSPDMACR). Requests for data to be written to the transmit FIFO or read from the receive FIFO in single or burst accesses are controlled by the four registered outputs, TXDMASREQ, TXDMABREQ, RXDMASREQ and RXDMABREQ. These outputs are asynchronously cleared to zero on application of PRESETn and subsequent next values are clocked through by PCLK.

The DMA transmit enable signal, DMATXEn, is asserted when the SSP enable (SSPEN), the transmit logic enable (TXE) and the transmit DMA (TXDMAE) bits are set in the SSPCR and SSPDMACR registers. The DMATXEn signal can also be asserted when the FIFOs need to be tested directly by setting the TXDMAE and TESTFIFO bits within the SSPDMACR and SSPTCR registers.

If DMATXEn is not asserted or SSPTXDMACLRSync is active, then the TXDMASREQ and TXDMABREQ signals are cleared to zero.

If there is one or more spaces available in the transmit FIFO, signified by the TXFLTE7Full signal being set, then TXDMASREQ will be asserted on the next PCLK rising edge.

If the transmit FIFO level falls below that of the fixed watermark level of 4, signified by TXRIS being set, then the TXDMABREQ will be asserted on the next PCLK rising edge.

If there is data available within the receive FIFO, signified by the RXGTE1Full signal being set, then RXDMASREQ will be asserted on the next PCLK rising edge.

If the receive FIFO level exceeds that of the fixed watermark level of 4, signified by RXRIS being set, then the RXDMABREQ will be asserted on the next PCLK rising edge.

When the respective DMA transfer is about to complete, the DMA controller will raise the relevant SSPTXDMACLR or SSPRXDMACLR signal, terminating the transfer. After termination, if more accesses are

required, then the relevant request signals will be clocked out on the next PCLK rising edge after removal of the respective clear signal.

5.18 Synchronisers to SSPCLK domain (SspSynctoSSPCLK)

This block contains all synchronisers in the design that synchronise signals entering the SSPCLK domain.

All signals to be synchronised are double synchronised through D-type flip-flops clocked by SSPCLK.

The primary inputs SSPCLKIN and SSPFSSIN are synchronised to the SSPCLK domain, followed by edge detection logic.

The TxDataAvlbl signal is from the transmit FIFO operating on PCLK, the transmit logic operating in the SSPCLK domain, detects if transmit data is available.

The SSE, MS and SOD signals originate from the SSPCR0 register which is accessed in the PCLK domain. These signals are configuration signals that control the enabling, master or slave mode operation and slave only device selections within the SSPCLK domain.

The CR0Update signal and the CPSRUpdate signal are from the PCLK domain part of the control logic required to synchronise the contents of the SSPCR0 and SSPCPSR registers to the SSPCLK domain. These signals are required by the SspScaleCntr block, which operates in the SSPCLK-domain.

The RTIC (Receive Timeout Interrupt Clear) signal originates from a write operation to the SSPICR (SSP Interrupt Clear Register) in the PCLK domain, but must reset the SSPRINTR logic which is asserted in the SSPCLK domain. Similarly, the RTIM (Receive Timeout Interrupt Clear) signal is derived from the SSPIMSC (SSP Interrupt Mask Set Clear) register, accessed in the PCLK domain but used to mask the SSPRTINTR signal in the SSPCLK domain.

The RNE (Receive FIFO Not Empty) signal is generated from the receive FIFO control logic, operating in the PCLK domain, and has to be synchronised to the SSPCLK domain where it is used in the determination of the SSPRTINTR generation.

5.19 Synchronisers to PCLK domain (SspSynctoPCLK)

This block contains all synchronisers in the design that synchronise signals entering the PCLK domain.

All signals to be synchronised are double synchronised through D-type flip-flops clocked by PCLK.

The TxRxBSY signal is from the SspMTxRxCntl block. It indicates that the SSP is currently busy with transmission / reception. The synchronised version of this signal is used in the SspTxFIFO block to generate the BSY signal.

The TxFRdPtrInc signal is from the SspMTxRxCntl block. It indicates to the transmit FIFO that operates on PCLK, that a data character has been copied into the transmit shift register and the read pointer in the FIFO can be incremented.

The RxFWr signal is from the SspMTxRxCntl block. This signal indicates to the receive FIFO that valid data is available on the MRxFWrData[15:0] / SRxFWrData[15:0] bus to be clocked into the receive FIFO.

The DataStp and RTINTR signals are synchronised to the PCLK domain such that hey can be read as the raw and masked interrupt status of the SSPRTINTR signal through the SspApbif block.

The primary inputs IntTXDMACLR and IntRXDMACLR are synchronised to the PCLK domain and are then used to clear the DMA requests generated from within the respective transmit and receive FIFO control logic.

5.20 Test logic (SspTest)

The Test block performs the following functions:

- Test mode control via SSPTCR register
- Integration testing using SSPITIP and SSPITOP read/set registers.
- Test read and write access of the Tx and Rx FIFOs respectively.
- Loopback facility of SSPTXD to SSPRXD.

The SSPCR test control register contains two programmable bits:

- ITEN, the integration test enable. When set, the SSP is placed into integration test mode. This mode allows point to point connections to be checked between other modules when the SSP is instantiated within a system. Two registers, SSPITIP (Integration Test Input) and SSPITOP (Integration Test Output) are used to implement this test mode.
- TESTFIFO, the test FIFO enable bit. When set, data can be read from the Tx FIFO or written to the Rx FIFO via the SSPTDR test data register.

5.20.1 Integration testing of block inputs

The following sections describe the integration testing for the block inputs:

- *Intra-chip inputs.*
- *Primary inputs.*

Intra-chip inputs

Figure 5.20-1 explains the implementation details of the input integration test harness. The ITEN bit is used as the control bit for the multiplexor, which is used in the read path of the SSPTXDMACLR and SSPRXDMACLR intra-chip inputs.

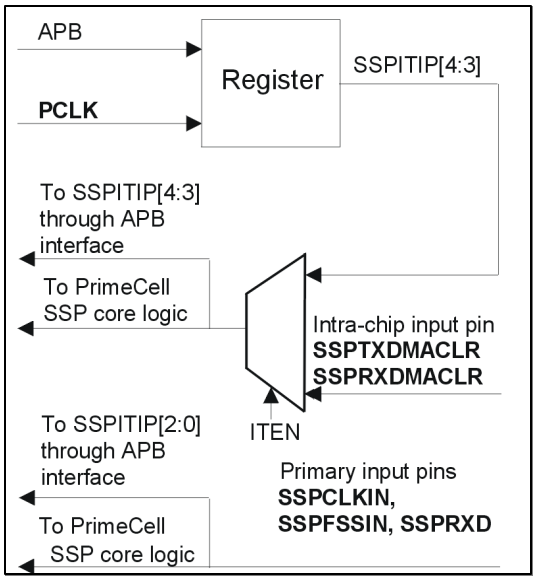

Figure 5.20-1 Input integration test harness

If the ITEN control bit is deasserted, the SSPTXDMACLR and SSPRXDMACLR intra-chip inputs are routed as the internal SSPTXDMACLR and SSPRXDMACLR inputs respectively, otherwise the stored register values are driven on the internal line.

When you run integration tests with the PrimeCell SSP in a standalone test setup:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the SSPITIP[1:0] register bits to the SSPRXDMACLR and SSPTXDMACLR signals.
- Write a 1 and then a 0 to each of the SSPITIP[4:3] register bits, and read the same register bits to ensure that the value written is read out.

When you run integration tests with the PrimeCell SSP as part of an integrated system:

- Write a 0 to the ITEN bit in the control register. This selects the normal path from the external SSPRXDMACLR pin to the internal SSPRXDMACLR signal, and the path from the external SSPTXDMACLR pin to the internal SSPTXDMACLR pin.
- Write a 1 and then a 0 to the internal test registers of the DMA controller to toggle the SSPRXDMACLR signal connection between the DMA controller and the PrimeCell SSP. Read from the SSPITIP[3] register bit to verify that the value written into the DMA controller, is read out through the PrimeCell SSP. Similarly, write a 1 and then a 0 to the internal registers of the DMA controller to toggle the SSPTXDMACLR signal connection between the DMA controller and the PrimeCell SSP. Read from the SSPITIP[4] register bit to verify that the value written into the DMA controller, is read out through the PrimeCell SSP.

Primary inputs

The following primary inputs are tested using the integration vector trickbox:

- SSPRXD, SSPCLKIN and SSPFSSIN.

All primary inputs are fed from the corresponding output through the integration trickbox. Refer to Primary outputs on page 62 for further details.

5.20.2 Integration testing of block outputs

The following sections describe the integration testing for the block outputs:

- *Intra-chip outputs.*
- *Primary outputs.*

Intra-chip outputs

Use this test for the following outputs:

- SSPTXDMASREQ
- SSPTXDMABREQ
- SSPRXDMASREQ
- SSPRXDMABREQ
- SSPINTR
- SSPTXINTR
- SSPRXINTR
- SSPRTINTR
- SSPRORINTR.

When you run integration tests with the PrimeCell SSP in a standalone test setup:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the SSPITOP[13:5] register bits to the intra-chip output signals.
- Write a 1 and then a 0 to the SSPITOP[13:5] register bits, and read the same register bits to verify that the value written is read out.

When you run integration tests with the PrimeCell SSP as part of an integrated system:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the SSPITOP[13:5] register bits to the intra-chip output signals.
- Write a 1 and then a 0 to the SSPITOP[13:5] register bits to toggle the signal connections between the DMA controller/interrupt controller and the PrimeCell SSP. Read from the internal test registers of the DMA controller/interrupt controller to verify that the value written into the SSPITOP[13:5] register bits is read out through the PrimeCell SSP.

Figure 5.20-2 explains the implementation details of the output integration test harness for intra-chip outputs.

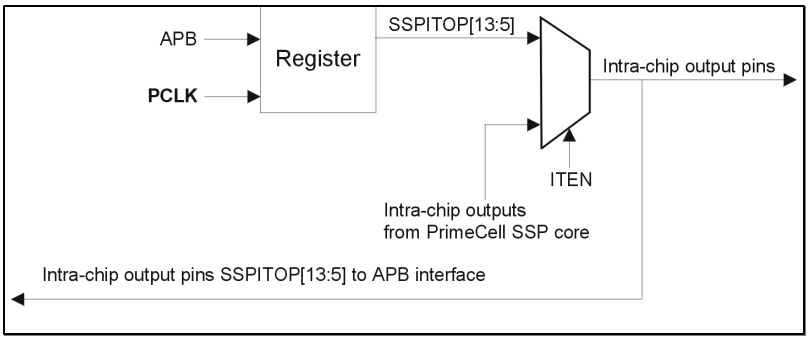

Figure 5.20-2 Output integration test harness, intra-chip outputs

Primary outputs

Integration testing of primary outputs and primary inputs is carried out using the integration vector trickbox. Use this test for the following outputs:

- SSPTXD
- SSPCLKOUT
- SSPFSSOUT
- nSSPCTLOE
- nSSPOE.

Note Only the SSPTXD, SSPCLKOUT, and SSPFSSOUT signals are available at the output pads of a typical configuration. The nSSPOE and nSSPCTLOE signals are internal connections to the pad.

Verify the primary input and output pin connections as follows:

The primary outputs, SSPTXD, SSPCLKOUT, and SSPFSSOUT are directly connected to SSPRXD, SSPCLKIN, and SSPFSSIN respectively by the integration trickbox shown in Figure 5.20-3. To test the nSSPOE and nSSPCTLOE connections, you can apply a weak pull down/pull to the tristate pins through the trickbox.

- All the primary outputs can be accessed through the SSPITOP register. Different data patterns are written to the output pins using the SSPITOP register.
- The looped-back data is read back through the SSPITIP register.

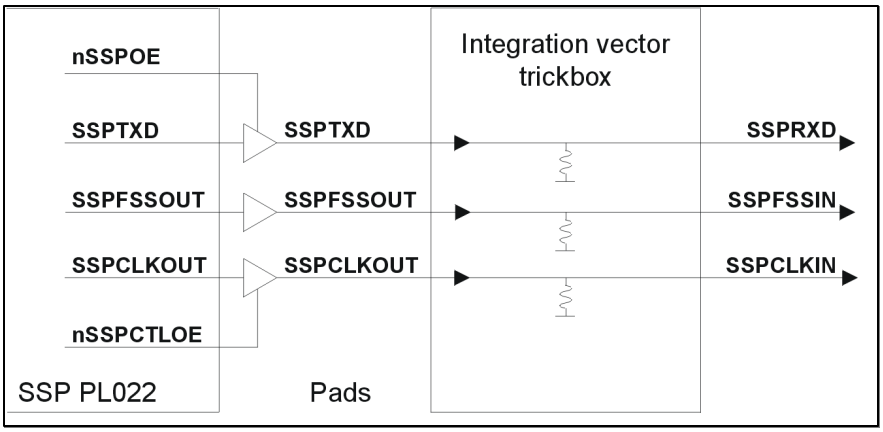

Figure 5.20-3 Primary outputs routed to primary inputs

Figure 4-4 shows implementation details of the output integration test harness in the case of primary outputs.

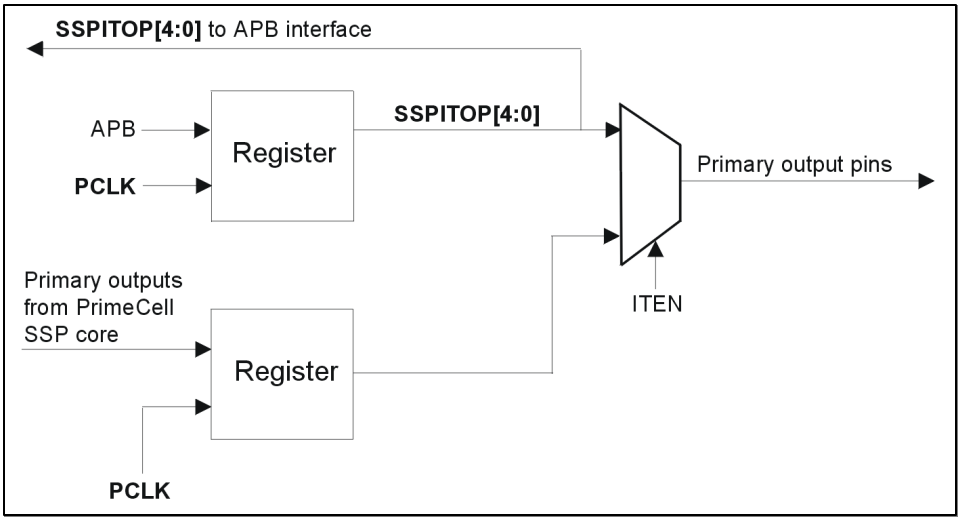

Figure 5.20-4 Output integration test harness, primary outputs

6 SSP FUNCTIONAL VERIFICATION DETAILS

The SSP Functional Verification testbench is an APB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the ***verification/vhdl*** and ***verification/verilog*** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the ***verification/bustest*** directory.

6.1 SSP VHDL Verification Testbench

The ARM PrimeCell SSP VHDL test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbench has been designed such that it can be used within most common VHDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. ***Figure 6.1-1*** illustrates the Bustalk VHDL testbench for running test vectors.

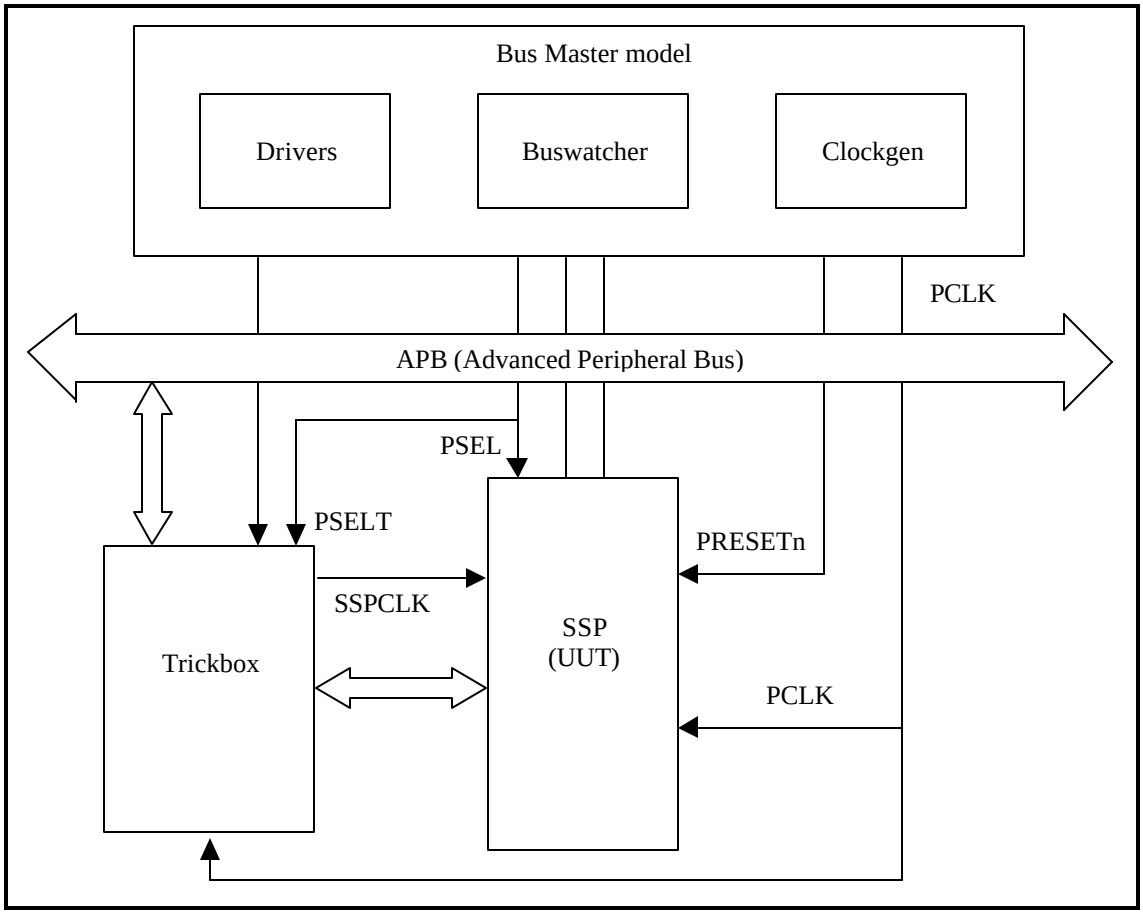

Figure 6.1-1 VHDL testbench for simulating test vectors

Figure 6.1-2 illustrates the hierarchy for the VHDL testbench.

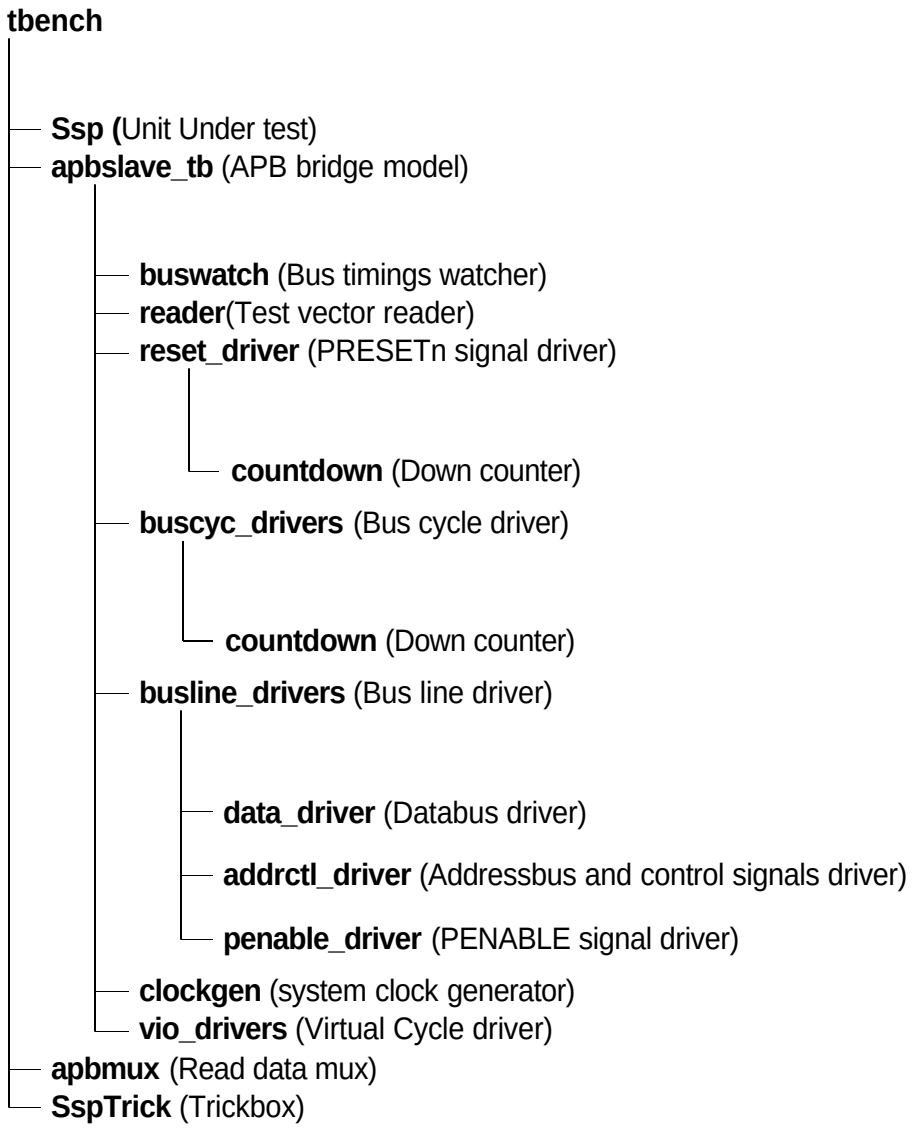

Figure 6.1-2 VHDL Testbench hierarchy

The testbench, **tbench.vhd** is used for functional simulations of the SSP. This module instantiates the SSP (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The SSPCLK is generated by the trickbox to facilitate frequency modifications from within the test code.

The SSPCLK and it's domain inputs to the SSP are driven by the trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency is set at 100MHz. This is the same frequency to which the SSP was constrained during synthesis. PCLK is set to the frequency defined in tbench.vhd using the tclkh and tclkl generic values when instantiating apbslave_tb.vhd.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- SSP VHDL RTL
- SSP VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in ***verification/vhdl*** .

6.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.vhd, instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the infile.bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the TrickBox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

While the default behaviour of the Bridge in the testbench is like a Rev E device, there is an option to force the bridge to issue commands with Rev D timings.

6.1.3 Trickbox

The functionality of the SSP is verified via a trickbox. The trickbox is an APB Slave which mimics the behaviour of an SSP Slave device. Like the SSP, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel converter to line up data received serially from the SSP and it has a parallel to serial converter to transmit the data stored in its FIFO. The trickbox has a checker module which flags off any violations in the SSP transmit/receive protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

6.1.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.vhd file in the ***verification/vhdl/tbench*** directory. The buswatch module which is in the ***verification/vhdl/buswatcher*** directory checks the Read/Write Databus timings and reports any violations in the timings.

6.1.5 APB Multiplexer

The read data buses from the SSP and the trickbox are muxed together in this module before being connected to the APB Bridge model.

6.1.6 Test Vectors

The test vectors for the verification of the SSP are coded into separate C files depending on the logical region of the SSP which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.1.7 Generics

The SSP testbench provides some configurability options using generics and constants. These generics and constants are configurable through the tbench.vhd file.

XonPSEL

XonPSEL is used in the tbench.vhd file that resides in ***verification/vhdl/tbench***.

This generic is used to eliminate the ‘X’s that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to **1**, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.vhd, the reset_driver.vhd and the buscyc_drivers.vhd which reside in ***verification/vhdl/bid***.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.vhd that resides in ***verification/vhdl/bid***.

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged, but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the buswatch.vhd that resides in ***verification/vhdl/buswatcher.***

This generic is used to suppress the checking of the PRDATA timings when the PRESETn signal is asserted. If **SuppressOnReset** is TRUE, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SuppressOnReset** is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

Mesg_enb

Mesg_enb is used in the buswatch.vhd which resides in ***verification/vhdl/buswatcher.***

This generic is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If **Mesg_enb** is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to FALSE in testbenches which include more than one slave.

Data_check

Data_check is used in the apbmux.vhd which resides in ***verification/vhdl/tbench***.

This constant is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **Data_check** has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This constant is used in testbenches which include more than one slave.

6.1.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the SSP test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

- verification/vhdl/Ssp_rtl/Ssp_rtl_modelsim.log

Run time logs for the netlist simulations are:

- verification/vhdl/Ssp_scaninsert_vhdl_net_max/Ssp_scaninsert_vhdl_net_max.log

6.2 SSP Verilog Verification Testbench

The ARM PrimeCell SSP Verilog test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. ***Figure 6.2-1*** illustrates the BusTalk Verilog testbench for simulating test vectors.

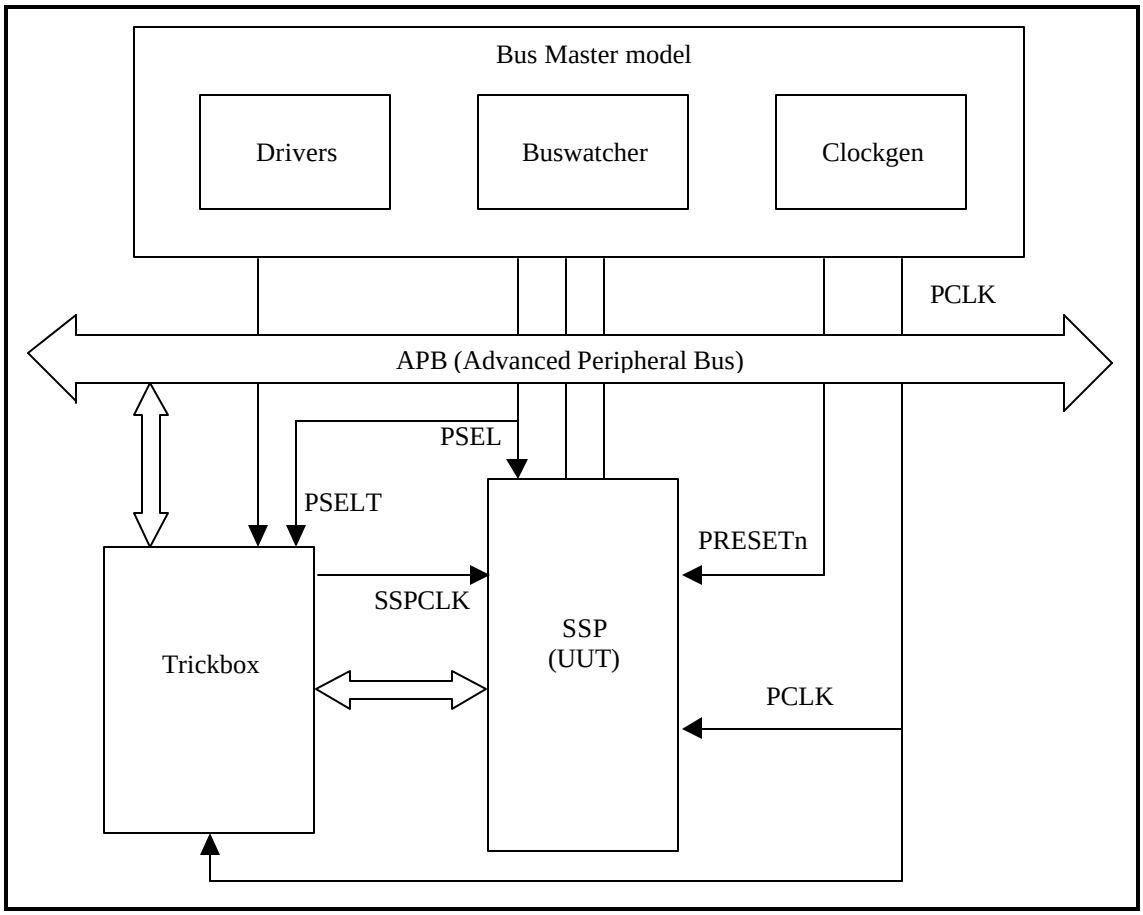

Figure 6.2-1 Verilog Testbench for simulating test vectors

Figure 6.2-1 illustrates the hierarchy for the Verilog testbench.

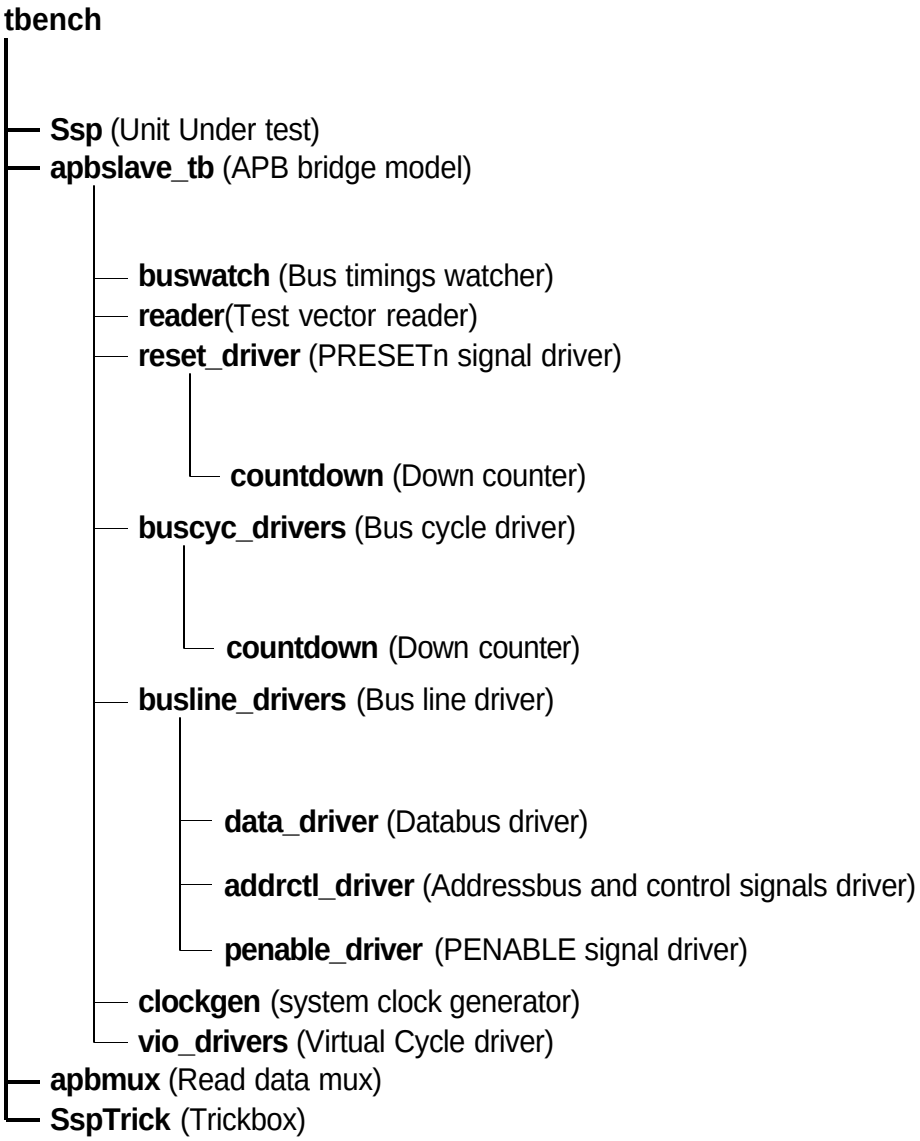

Figure 6.2-2 Verilog Testbench hierarchy

The testbench, **tbench.v** is used for functional simulations of the SSP. This module instantiates the SSP (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The SSPCLK is generated by the trickbox to facilitate frequency modifications from within the test code.

All the SSPCLK and SSPCLK domain inputs to the SSP are driven by the trickbox. All the scan input pins are tied low to keep them from interfering with the test process.

The default test frequency is set at 100 MHz. This is the same frequency to which the SSP was constrained during synthesis. SSPCLK is set to the PCLK frequency defined in timing.v using the Tclkh and Tclkl ‘define values when instantiating apbslave_tb.v in tbench.v. Note that the ‘define values override the local parameters defined in clockgen.v in the ***verification/verilog/common*** directory. All the non-AMBA inputs to the SSP, except the scan data inputs, are driven by the Trickbox.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- SSP Verilog RTL
- SSP Verilog netlist from Verilog Source
- SSP Verilog netlist from VHDL Source

One out of these three versions may be referenced via the corresponding links to uut directory in ***verification/verilog*** directory***.***

6.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.v instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the bif.sim file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

The Poll Command in APB:

Usage: PO(expected value, mask , addr, [limit], [tag]);

On encountering a PO command in the vector file, the APB bridge model issues a read cycle (PSR) to the specified address (addr). If the read value matches the (expected) value, the next command in the vector file is executed. If the values do not match, reads are repeatedly issued to the same address until either the expected value is sampled or the number of reads issued exceeds the [limit] parameter. If the [limit] is not specified, the Polling continues indefinitely until the expected value is returned. One limitation of the POLL command is that two successive PO commands cannot be issued back-to-back. They should be separated by at least one command like PSR , PSW, PI etc.

6.2.3 Trickbox

The functionality of the SSP is verified via a Trickbox model. The trickbox is an APB Slave which mimics the behaviour of an SSP Slave device. Like the SSP, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel converter to line up data received serially from the SSP and it has a parallel to serial converter to transmit the data stored in its FIFO. The trickbox has a checker module which flags off any violations in the SSP transmit/receive protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

6.2.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.v file in the ***verification/verilog/tbench*** directory. The buswatch block which is in the ***verification/verilog/buswatcher*** directory checks the Read/Write data bus timings and reports any violations in the timings.

6.2.5 APB Multiplexer

The read data buses from the SSP and the Trickbox are multiplexed together in this block before being connected to the APB Bridge model.

6.2.6 Vector Sets

The test vectors for the verification of the SSP are coded into separate C files depending on the logical region of the SSP which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.2.7 Parameters

The SSP testbench provides some configurability options via parameters and defines. All the parameters, except MESG_ENB and SUPPRESSONRESET are configurable through the tbench.v file. MES_ENB and SUPPRESSONRESET are configurable through the buswatch.v that resides in ***verification/verilog/buswatcher.*** The define, DATA_CHECK is configurable through the apbmux.v that resides in ***verification/verilog/tbench.***

XonPSEL

XonPSEL is used in the tbench.v file that resides in ***verification/verilog/tbench***.

This define is used to eliminate the ‘X’s that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to **1**, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.v, the reset_driver.v and the buscyc_drivers.v which reside in ***verification/verilog/bid***.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.v which resides in ***verification/verilog/bid***.

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SUPPRESSONRESET

SUPPRESSONRESET is used in the buswatch.v which resides in ***verification/verilog/buswatcher.***

This parameter is used to suppress the checking of the PRDATA timings when PRESETn signal is asserted. If **SUPPRESSONRESET** is **1**, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SUPPRESSONRESET** is set to **0**, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

MESG_ENB

MESG_ENB is used in the buswatch.v which resides in ***verification/verilog/buswatcher.***

This parameter is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to **0**. If **MESG_ENB** is set to **1**, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to **0** in testbenches which include more than one slave.

DATA_CHECK

DATA_CHECK is used in the apbmux.v which resides in ***verification/verilog/tbench***.

This define is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When **DATA_CHECK** has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This define is used in testbenches which include more than one slave.

6.2.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the SSP test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

- verification/verilog/Ssp_rtl/Ssp_rtl_modelsim.log
- verification/verilog/Ssp_rtl/Ssp_rtl_LDV.log

Run time logs for the netlist simulations are:

- verification/verilog/Ssp_noscan_vhdl_net_min/Ssp_noscan_vhdl_net_min_modelsim.log
- verification/verilog/Ssp_scanready_verilog_net_typ/Ssp_scanready_verilog_net_typ_LDV.log

The code coverage results are:

- verification/verilog/Ssp_rtl/Ssp_rtl_modelsim_cover.log

6.3 SSP Trickbox

The Trickbox is a programmable APB device designed to provide a test engineer a means to drive the non-APB input pins of the SSP and sample the non APB responses of the same.

The Trickbox reads the data transmission line of the SSP and performs serial to parallel conversion on data received on it. The Trickbox is capable of converting parallel data to serial data and driving serial data on the data reception lines of the SSP. The transmit and receive paths of the Trickbox are buffered with internal FIFO memories allowing up to 16 bytes to be stored independently in both the transmit and receive modes. The Trickbox has SSPCLKOUT frequency measurement and SSPFSSOUT width measurement capabilities. The Trickbox contains logic to verify SSP operation in Master mode as well as Slave mode. When the SSP is in Master mode, the Trickbox monitors activity on the SSPCLKOUT and SSPFSSOUT outputs of the SSP in addition to monitoring the serial data line (SSPTXD). When the SSP is in Slave mode, the Trickbox drives the SSPCLKIN and SSPFSSIN inputs of the SSP in addition to driving the serial data line (SSPRXD).

The following sections detail the functionality of the Trickbox.

6.3.1 Trickbox Signal Description

APB signal descriptions

Name	Type	Source / Destination	Description
PCLK	Input	From Test Bench	APB clock
PRESETn	Input	From Test Bench	Bus reset signal, Active Low
PADDR [7:2]	Input	From Test Bench	Subset of APB address bus
PRDATA [15:0]	Output	To Test Bench	Subset of unidirectional APB read data bus
PWDATA [15:0]	Input	From Test Bench	Subset of unidirectional APB write data bus
PSELT	Input	From Test Bench	Trickbox select signal from decoder. When set to 1 this signal indicates the slave device is selected and that a data transfer is required.
PENABLE	Input	From Test Bench	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle.
PWRITE	Input	From Test Bench	APB transfer direction signal, indicates a write access when HIGH, read access when LOW.

SSP signals

Name	Type	Source/Destination	Description
SSPRXD	Input	From SSP	SSP transmitted serial data output, defaults to the marking tri-state , when reset.
SSPTXD	Output	To SSP	SSP received serial data input, defaults to the marking tri-state , when reset. Trickbox drives serial data on this line.

SSPFSSIN	Input	From SSP	Frame start signal from the SSP when it is in Master mode
SCLKIN	Input	From SSP	Serial clock signal from the SSP when it is in Master mode.
SFRMOUT	Output	To SSP	Frame start signal to the SSP when it is in Slave mode.
SCLKOUT	Output	To SSP	Serial clock to the SSP when it is in Slave mode.
SSPCLK	Output	To SSP	SSP reference clock signal.
nSSPRST	Output	To SSP	SSP Reset signal for clocked elements in the SSPCLK domain.

SSP Interrupt & DMA lines

Name	Type	Source/Destination	Description
SSPINTR	Input	From SSP	SSP interrupt (active HIGH).
SSPTXINTR	Input	From SSP	SSP Transmit FIFO interrupt (active HIGH).
SSPRXINTR	Input	From SSP	SSP Receive FIFO interrupt (active HIGH).
SSPRTINTR	Input	From SSP	SSP Receive Timeout Interrupt (active HIGH)
SSPRORINTR	Input	From SSP	SSP Receive Overrun Interrupt (active HIGH)
SSPTXDMACLR	Output	To SSP	SSP TX DMA Clear
SSPRXDMACLR	Output	To SSP	SSP RX DMA Clear
SSPTXDMASREQ	Input	From SSP	SSP TX DMA Single Request
SSPTXDMABREQ	Input	From SSP	SSP TX DMA Burst Request
SSPRXDMASREQ	Input	From SSP	SSP RX DMA Single Request
SSPRXDMABREQ	Input	From SSP	SSP RX DMA Burst Request

SSP Scan Test line

Name	Type	Source/Destination	Description
SCANMODE	Output	To SSP	SSP Scanmode line (active HIGH).

6.3.2 Trickbox functional description

The Trickbox models an SSP Master and an SSP Slave. The Trickbox functions as a slave device when verifying the SSP's Master mode functionality. It is capable of responding to the SSP in accordance with the TI Synchronous Serial, Motorola SPI and National Microwire frame formats.

The Trickbox models a Master device when testing the SSP's Slave mode functionality. It is capable of generating SSPCLKIN and SSPFSSIN signals to the SSP in accordance with the TI Synchronous Serial, Motorola SPI and National Microwire frame formats.

APB Interface and the Register Block

The APB is a local secondary bus which provides a low-power extension to the higher AHB (or ASB) within the AMBA system hierarchy. The APB interface generates read and write decodes for accesses to status registers, control registers and transmit/receive FIFO memories. The register block stores data written or to be read across the APB interface. All registers are READ / WRITE, although the writeability feature of some registers is not used. Such registers are specified explicitly.

Trickbox register summary

Address (Offset from TrickboxBase)	Type	Width	Reset Value	Name	Description
0x00	R/W	4	0x00	SSPTBPRE	Contains the factor by which SSPCLK is initially divided. Zero is an illegal value.
0x04	R/W	16	0x00	SSPTBCR0	Trickbox Control Register 0.
0x08	R/W	6	0x00	SSPTBCR1	Trickbox Control Register 1.
0x0C	R/W	16	0x00	SSPTBTDR	Trickbox Transmit Data Register.
0x10	R/W	16	0x00	SSPTBRDR	Trickbox Receive Data Register.
0x14	R/W	13	0x00	SSPTBSR	Trickbox Status Register.
0x18	R/W	6	0x00	SSPTB_SET_PINS	Trickbox Scanmode Register.
0x1C	R/W	16	0x00	SSPTBCLKREG	Trickbox SSP Clock Register
0x20	R/W	11	0x00	SSPTBCR2	Trickbox Control Register 2.
0x24	R/W	16	0x00	SSPTBCLKREG1	Trickbox SSP Clock Register 1.
0x28	R/W	4/2	0x0	SSPTBDMACR	Trickbox SSP DMA Control Register

Transmit and Receive FIFOs

The transmit FIFO is a First-In, First-Out memory buffer, 16-bit wide, 16-word deep. Any data written to the Trickbox data register is stored in this FIFO till it is read out by the transmit logic.

The receive FIFO is a First-In, First-Out memory buffer, 16-bit wide, 16-word deep. Received data is stored in the receive FIFO by the receiver logic until read out by the CPU across the APB interface.

Master-testing Transmitter/ Receiver Logic

This block models a Slave device. It is operational when the SSP is in Master mode. The transmitter logic performs parallel-to-serial conversion on the data read from the transmit FIFO. The Control logic transmits the serial bit stream beginning with MSB (Most Significant Bit) first and LSB (Least Significant Bit) last, according to the programmed frame format and protocol configuration in the control registers. The bit rate is determined according to the following formula

Bit Rate (SSPCLKIN freq.) = (SSPCLK freq.)/ (Prescale value * (SCR + 1))

SCR denotes the value programmed in the SCR0 register bits [15:8].

The receiver logic performs serial-to-parallel conversion on the received bit stream and writes to the receive FIFO. The Trickbox samples the received data according to the programmed frame formats and protocols

Slave-testing Transmitter/ Receiver Logic

This block models a Master device. It is operational when the SSP is in Slave mode. It takes SSPCLK generated by the Trickbox as input and generates the SSPCLKOUT and SSPFSSOUT control signals to the SSP. Besides the parallel-to-serial conversion logic and serial-to-parallel conversion logic, this block has additional logic to test certain specific features of the SSP in the Slave mode.

It generates a free running SSPCLKOUT when the FRC bit is set and when the frame format is programmed to TI Synchronous Serial or National Microwire frame format.

When programmed for the TI Synchronous Serial frame format, SSPFSSOUT may optionally be extended to many SSPCLKOUT periods by setting the ESFRM bit. In the Motorola SPI frame format, SSPFSSOUT may optionally be deactivated between every frame with SPH = 1 by setting the SPISFRMEn bit. This verifies the SSP's operation with Motorola SPI-compatible Masters that do negate SSPFSSIN to the SSP after every frame and with Masters that don't.

Protocol checker

This module monitors the SSPCLKOUT and SSPFSSOUT lines from the SSP to ensure that the protocol requirements for the TI, Motorola SPI and National MicroWire frame formats are complied with. The following specific checks are performed by this block:

- When the SSP is in Master mode, the width of SSPCLKOUT and SSPFSSOUT are checked. Data Setup and Hold Times are measured and it is ensured that a minimum Setup/Hold time of half an SSPCLKOUT period is present. Any discrepancy is reported.
- When the SSP is in Master mode and is programmed for the SPI mode with SPH = 0, the Trickbox checks whether SSPFSSOUT is deactivated between successive frames.
- When the SSP is in the slave mode, and SSPFSSIN to the SSP is de-activated, the Trickbox checks that the serial data output from the SSP is tri-stated.
- When the SSP is disabled or when the SSP is in Slave mode with the SOD bit set (receive-only operation) the Trickbox checks that the serial data output from the SSP is tristated.
- When the SSP is in the Slave mode and is being subjected to an extended SSPFSSIN pulse in the TI Synchronous Serial frame format, the Trickbox checks that the serial data output line from SSP does not change when SSPFSSIN is high.
- When the SSP is in Slave mode the Trickbox checks whether SSPFSSOUT and SSPCLKOUT from the SSP are at their respective inactive values.

6.3.3 Trickbox Register Description

The base address of the Trickbox is not fixed and may be different for any particular system implementation. However, the offset of any particular register from the base address is fixed.

SSPTBPRE : Trickbox Prescale register [4] (+0x00)

The SSP Trickbox Prescale register is a four bits read/write register .The prescale register (PRESCALE) is used to specify the division factor by which the SSPCLK, in the SSP master, is internally divided before further use.

Addr: BaseAddr + 0x0 Trickbox Clock Prescale Register: SSPTBPRE Read-Write

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Reserved												PRESCALE			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bits	Name	Read/Write	Function
3:0	PRESCALE	R/W	Clock prescale divisor. Should be a number from 1 to 15 depending on the frequency of SSPCLKIN.
15:0	RESERVED	RESERVED	RESERVED

This internal division factor in the SSP master is equal to one half of the value written into the SSPCPSR register in the SSP. Based on the actual frequency of the clock signal in the system that is connected to the SSPCLK

input, this register can be programmed with the appropriate divisor value in order to have the desired bit-rate range.

The value programmed into this register should be a number from 1 to 15.

SSPTBCR0: Trickbox Control Register 0 [16] (+0x04)

The SSP Trickbox Control register 0 (SCR0) contains four different bit-fields which control various functions within the SSP Trickbox.

- Data Size Select (DSS)

The 4-bit data size select (DSS) field is used to select the size of the data transmitted and received by the Trickbox. Data can be 4 to 16 bits in length. When data is programmed to be less than 16 bits, received data is automatically right justified and the upper bits in the receive FIFO are zero filled by the receive logic. Transmit data should be right justified by software before being written into the transmit FIFO. However, the upper unused bits are ignored by the Trickbox's transmit logic. Data sizes of 0,1, 2, and 3 bits are illegal. When the National Microwire frame format is selected, this bit-field selects the size of the transmitted data. Note that the size of the received data is always 8-bits in this mode.

- Frame Format (FRF)

The 2-bit frame format (FRF) bit-field is used to select which frame format to use: Motorola SPI (FRF=00), Texas Instruments Synchronous Serial (FRF=01), or National Microwire (FRF=10). Note that FRF=11 is reserved and produces unpredictable results.

- Trickbox Enable (ENABLE)

The Trickbox enable (ENABLE) bit is used to enable and disable all Trickbox operation. When ENABLE=0 the Trickbox is disabled, when ENABLE=1 it is enabled. ENABLE is cleared to zero on reset to ensure that the Trickbox is disabled following a reset.

- Serial Clock Rate (SCR)

The 8-bit serial clock rate (SCR) bit-field is used to select the baud or bit rate of the SSP. A total of 256 different bit rates can be selected, ranging from a minimum of 7.2Kbps to a maximum of 1.8432Mbps. The SSP bit rate is calculated as

Bit Rate = SSPCLK freq / (Prescale value) * (SCR + 1)

Depending on the frame format selected, each transmitted bit is either driven on the rising- or falling-edge of SSPCLKIN, and is sampled on the opposite clock edge.

The table overleaf shows the bit locations corresponding to the four different control bit fields within Trickbox control register 0. The reset state of all control bits is zero. Note that the reset value of 0 is one of the illegal values for DSS and must be initialised to a legal value before enabling the Trickbox.

Addr: Base Addr + 0x04 Trickbox Control Register 0: SSPTBCR0 R/W

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	SCR								Reserved	ENABLE	FRF		DSS			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
3-0	DSS	Data Size Select 0000 – Reserved, undefined operation 0001 – Reserved, undefined operation 0010 – Reserved, undefined operation 0011 – 4-bit data 0100 – 5-bit data 0101 – 6-bit data 0110 – 7-bit data 0111 – 8-bit data 1000 – 9-bit data 1001 – 10-bit data 1010 – 11-bit data 1011 – 12-bit data 1100 – 13-bit data 1101 – 14-bit data 1110 – 15-bit data 1111 – 16-bit data
5-4	FRF	Frame Format 00 – Motorola SPI Frame Format 01 – TI Synchronous Serial Frame Format 10 – National Microwire Frame Format 11 – Reserved, undefined operation
6	ENABLE	Trickbox Enable bit
7	RESERVED	RESERVED
15-8	SCR	Serial Clock Rate Value (from 0 to 255) used to generate the transmission rate of the Trickbox. Bit Rate = (SSPCLK freq) /((Prescale value) *(SCR+1))

SSPTBCR1: Trickbox control register 1 [6] (+0x08)

The Trickbox Control register 1 (SSPTBCR1) contains five different bit fields which control various functions within the Trickbox.

- Polarity (SPO)

The Polarity (SPO) bit-field is used to select the SSPCLKIN Polarity when the FRF field in the SSPCR0 register is programmed for the Motorola SPI Frame Format.

- Phase (SPH)

The Phase (SPH) bit-field is used to select the SSPCLKIN Phase when the FRF field in the SSPCR0 register is programmed for the Motorola SPI Frame Format.

- Receive FIFO watermark level (RXW)

This sets the receive FIFO service flag (RXWFLG in the Trickbox) depending on the programmed value.

- Master/Slave select (MS). Indicates the mode (Master mode / Slave mode) in which the SSP has been programmed to operate.
- Ignore SSP output (OD). Indicates whether the SSP's serial data outputs need to be monitored.

Addr: BaseAddr + 0x08 Trickbox Control Register : SSPTBCR1 Read Write

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	RESERVED										OD	MS	RXW		SPH	SPO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0	SPO	SSPCLKIN Polarity (Applicable to Motorola SPI Frame Format only)
1	SPH	SSPCLKIN Phase (Applicable to Motorola SPI Frame Format only)
3-2	RXW	Set Rx FIFO service flag depending on Programmed value. 00 = Two or more entries in the receive FIFO are valid. 01 = Four or more entries in the receive FIFO are valid. 10 = Six or more entries in the receive FIFO are valid. 11 = Eight or more entries in the receive FIFO are valid.
4	MS	Master/Slave select 0 = SSP is in Master mode 1 = SSP is in Slave mode
5	OD	Ignores SSP output 0 = Normal mode, SSP's serial data output is not ignored. 1 = Trickbox ignores the SSP's serial data output
15-6	RESERVED	RESERVED

SSPTBCR2: Trickbox control register 2 [11] (+0x20)

Addr: BaseAddr + 0x08 Trickbox Control Register : SSPTBCR2 Read Write

Bit	15 - 11	10	9	8	7	6	5	4	3	2	1	0
	RESERVED	TIDASFRM	SPISFRMEn	FRC	ESFRM							
Reset	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0-7	ESFRM	Extended Serial Frame (Applicable to TI Frame Format only) width. The value programmed into this field is governed by the following relationship : ESFRM = {Required width of SSPFSSOUT in terms of number of SSPCLKOUT periods} – 1
8	FRC	Free Running Clock (Applicable to TI and NM Frame formats only) 0 – SSPCLKOUT from the Trickbox is not free-running 1 – SSPCLKOUT from the Trickbox is free-running (SSPCLKOUT is active even when no data transfer is in progress)
9	SPISFRMEn	SPI SFRM Enable 0 – Normal operation 1 – Trickbox negates SSPFSSOUT for one SSPCLKOUT phase duration between successive frames in SPI mode (SPH = 1)
10	TIDASFRM	TI Deasserted SFRM 0 – Normal operation 1 – Trickbox's protocol check on the SSPFSSOUT line from the SSP is disabled.
15-11	RESERVED	RESERVED

The Trickbox Control register 2 (SCR2) contains four different bit fields, which control various functions within the Trickbox.

- Extended Serial Frame (ESFRM)

Indicates the number of SSPCLKOUT periods for which the SSPFSSOUT output from the Trickbox should be activated in TI Synchronous Serial frame format.

- Free Running Clock (FRC)

Selects the mode in which the Trickbox drives a free-running SSPCLKOUT to the SSP in the TI and NM frame formats.

- SPI SFRM Enable (SPISFRMEn)

Selects the mode in which the Trickbox de-activates SSPFSSOUT between successive frames when the SSP is programmed for the Motorola SPI frame format with SPH = 1.

- TI Deasserted SFRM (TIDASFRM)

When set, the Trickbox's protocol check is disabled.

SSPTBTDR : Trickbox Transmit Data register [16] (+0x0C)

The Trickbox transmit data register (STDR) is 16-bits wide for all data transfers from the Trickbox. STDR is zero on reset and when transmit FIFO is empty.

User should right justify data when the Trickbox is programmed for a data size less than 16 bits. The transmit logic ignores the unused bits.

The SSR register holds two status bits associated with the transmit FIFO.

Addr: Base Addr + 0x0C — SSP Data Register: SSPTBTDR — Read / Write

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Transmit FIFO															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
15-0	DATA	Transmit FIFO Note: User should right justify data when the Trickbox is programmed for a data size less than 16-bits. Unused bits at the top are ignored by the transmit logic..

SSPTBRDR : Trickbox Receive Data register [16] (+0x10)

The Trickbox receive data register (SRDR) is 16-bits wide and stores data transmitted by the SSP. On reads, this register gives the 16-bit data word received by the Trickbox and stored in the receive FIFO. The SSR register holds six status bits associated with this.

Addr: Base Addr + 0x10 — Trickbox Data Register: SSPTBRDR — Read/Write

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Receive FIFO															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
15-0	DATA	Receive FIFO Note: The Trickbox right justifies data when programmed for a data size less than 16-bits. Unused bits at the top are zero filled .

The table overleaf shows the bit locations of the Trickbox data register. The transmit FIFO and the Receive FIFO are cleared when the Trickbox is disabled.

SSPTBSR: Trickbox Status register [13] (+0x14)

The Trickbox status register (SSPTBSR) contains thirteen bits which indicate TX and RX FIFO levels, the Trickbox Busy status and the SSP Interrupt line status.

- Receive FIFO Empty Flag (RXFE)

The receive FIFO empty flag (RXFE) is reset whenever the receive FIFO contains one or more entries of valid data and is set when it no longer contains any valid data.

- Receive FIFO Full Flag (RXFF)

The Receive FIFO Full flag (RXFF) is set whenever all the locations in the Receive FIFO contain valid data.

- Transmit FIFO Empty Flag (TXFE)

The transmit FIFO empty flag (TXFE) is set whenever the transmit FIFO does not contain any entries with valid data. This flag is cleared if at least one entry with valid data is present in the transmit FIFO.

- Transmit FIFO Full Flag (TXFF)

The transmit FIFO full flag (TXFF) is set whenever all the locations in transmit FIFO contain valid data This flag is cleared when the FIFO is not completely full.

- Trickbox Busy Flag (BSY)

The Trickbox busy (BSY) flag is set when it is transmitting and /or receiving data.

- SSPINT Flag (SSPINT)

The Trickbox SSPINT flag is set when the SSPINTR line set to high.

- RXWFLG Flag (RXWFLG)

The Trickbox RXWFLG bit is set under the following conditions:

SCR1 [3:2] = 00 and Two or more entries in the receive FIFO are valid.

SCR1 [3:2] = 01 and Four or more entries in the receive FIFO are valid.

SCR1 [3:2] = 10 and Six or more entries in the receive FIFO are valid.

SCR1 [3:2] = 11 and Eight or more entries in the receive FIFO are valid

- RFSFLG Flag (RFSFLG)

The Trickbox RFSFLG flag is set when SSPRXINTR line is high along with SSPINTR.

- TFSFLG Flag (TFSFLG)

The Trickbox TFSFLG flag is set when SSPTXINTR line is high along with SSPINTR.

- RORFLG Flag (RORFLG)

The Trickbox RORFLG flag is set when SSPRORINTR line is high along with SSPINTR.

- PCLKON Flag (PCLKON)

The Trickbox PCLKON flag is set when PCLK is routed to SSPCLK

- REFCLKON Flag (REFCLKON)

The Trickbox REFCLKON flag is set when REFCLK is routed to SSPCLK

- REFCLK1ON Flag (REFCLK1ON)

The Trickbox REFCLK1ON flag is set when REFCLK1 is routed to SSPCLK1. SSPCLK1 is routed to the SSP as its SSPCLK input.

The table below shows the bit locations corresponding to the status and flag bits within the Trickbox status register. All bits are read-only. Writes have no effect.

Addr: BaseAddr + 0x14 Trickbox Status Register: SSPTBSR Read / Write

Bit	15:14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Rsvd	SSP RTINT	REFCLK1 ON	REFCLK ON	PCLK ON	ROR FLG	TFS FLG	RFSF LG	RXW FLG	SSP INT	BSY	TXFF	TXFE	RXFF	RXFE
Reset		0	0	0	0	0	0	0	0	0	0	0	1	0	1

Bit	Name	Description
0	RXFE	Trickbox Receive FIFO Empty 0 – Receive FIFO is not empty 1 – Receive FIFO is empty
1	RXFF	Trickbox Receive FIFO Full 0 – Receive FIFO is not full 1 – Receive FIFO is full
2	TXFE	Trickbox Transmit FIFO Empty 0 – Transmit FIFO is not empty 1 – Transmit FIFO is empty
3	TXFF	Trickbox Transmit FIFO Full 0 – Transmit FIFO is not Full 1 – Transmit FIFO is Full
4	BSY	Trickbox Busy Flag 0 –Trickbox is idle. 1 –Trickbox is currently transmitting and/or receiving data.
5	SSPINT	Trickbox SSPINTR Flag 0 - SSPINTR Line is Low. 1 - SSPINTR Line is High.
6	RXWFLG	Trickbox RXWFLG Flag set when SCR1 [3:2] = 00 and Two or more entries in the receive FIFO are valid. SCR1 [3:2] = 01 and Four or more entries in the receive FIFO are valid. SCR1 [3:2] = 10 and Six or more entries in the receive FIFO are valid. SCR1 [3:2] = 11 and Eight or more entries in the receive FIFO are valid

7	RFSFLG	Trickbox RFSFLG 0 - SSPRXINTR line is Low. 1 - SSPRXINTR line is High
8	TFSFLG	Trickbox TFSFLG 0 - SSPTXINTR line is Low. 1 - SSPTXINTR line is High
9	RORFLG	Trickbox RORFLG 0 - SSPRORINTR line is Low. 1 - SSPRORINTR line is High
10	PCLKON	Trickbox PCLKON 0 - PCLK is not routed to SSPCLK 1 - PCLK is routed to SSPCLK
11	REFCLKON	Trickbox REFCLKON 0 - REFCLK is not routed to SSPCLK 1 - REFCLK is routed to SSPCLK
12	REFCLK1ON	Trickbox REFCLK1ON 0 - REFCLK1 is not routed to SSPCLK1 1 - REFCLK1 is routed to SSPCLK1
13	SSPRTINT	Trickbox SSPRTINTR Flag 0 – SSPRTINTR Line is Low. 1 - SSPRTINTR Line is High.
15-14	-	Reserved

SSPTB_SET_PINS : Trickbox SCANMODE register [7] (+0x18)

The Trickbox scanmode register (SCANMODE) is used to drive the SCANMODE line and to ensure a glitch-free clock switching to the SSP.

Table shows the bit locations corresponding to the SCANMODE register..

Addr: BaseAddr + 0x18 Trickbox SCANMODE Register: Read-Write

Bit	15 – 6	5	4	3	2	1	0
	Reserved	GENCLK1	SSPCLK1SEL	SSPCLKSEL	GENCLK	RSTMODE	SCANMODE
Reset	0	0	0	0	0	0	0

Bit	Name	Description
0	SCANMODE	Trickbox SCANMODE Bit 0 - Drive SCANMODE Line Low (Default) 1 - Drive SCANMODE Line High.
1	RSTMODE	Trickbox RSTMODE bit 0 – Drive nSSPRST line Low (Default) 1 – Drive nSSPRST line High.
2	GENCLK	Trickbox GENCLK bit 0 – Drive SSPCLK line Low (Default) 1 – Generate SSPCLK with the time period value programmed in SSPTBCLKREG
3	SSPCLKSEL	Trickbox SSPCLKSEL 0 - Drive SSPCLK line Low, if GENCLK is set 1 - Generate SSPCLK with the time period value programmed in SSPTBCLKREG, if GENCLK is set
4	SSPCLK1SEL	Trickbox SSPCLK1SEL 0 - Drive SSPCLK1 line Low, if GENCLK1 is set 1 - Generate SSPCLK1 with the time period value programmed in SSPTBCLKREG1, if GENCLK1 is set
5	GENCLK1	Trickbox GENCLK1 bit 0 – Drive SSPCLK1 line Low (Default) 1 – Generate SSPCLK1 with the time period value programmed in SSPTBCLKREG1
15-6	-	Reserved.

SSPTBCLKREG : Trickbox SSP Clock register [16] (+0x1C)

This register specifies the absolute value of the time period of the SSPCLK signal to be fed to the SSP. The unit for this value is given by the simulator resolution.

Addr: BaseAddr + 0x18 Trickbox SSP Clock Register: Read-Write

Bit	15 – 0
	CLKPERIOD
Reset	0x00

Bit	Name	Description
15-0	CLKPERIOD	SSPCLK Time Period Value

SSPTBCLKREG1: Trickbox SSP Clock register 1 [16] (+0x24)

This register specifies the absolute value of the time period of the SSPCLK1 signal connected to the SSP. The unit for this value is given by the simulator resolution.

Addr: BaseAddr + 0x18 Trickbox SSP Clock Register 1: Read-Write

Bit	15 – 0
	CLKPERIOD
Reset	0x00

Bit	Name	Description
15-0	CLKPERIOD	SSPCLK1 Time Period Value

SSPTBDMAC : Trickbox SCANMODE register [6] (+0x28)

The Trickbox scanmode register (SCANMODE) is used to drive the SCANMODE line and to ensure a glitch-free clock switching to the SSP.

Table shows the bit locations corresponding to the SCANMODE register..

Addr: BaseAddr + 0x18 Trickbox SCANMODE Register: Read-Write

Bit	15 – 6	5	4	3	2	1	0
	Rsvd	SSPRX DMASREQ	SSPRX DMABREQ	SSPTX DMASREQ	SSPTX DMABREQ	SSPRX DMACLR	SSPTX DMACLR
Reset	0	0	0	0	0	0	0

Bit	Name	Description
0	SSPTXDMACLR	DMA Transmit Request Clear 0 – no action 1 – Clear the SSP transmit DMA single and/or burst requests
1	SSPRXDMACLR	DMA Receive Request Clear 0 – no action 1 – Clear the SSP recieve DMA single and/or burst requests
2	SSPTXDMABREQ	Transmit FIFO DMA Burst Write Request 0 – Transmit FIFO cannot accept burst data write 1 – Transmit FIFO is requesting burst data write
3	SSPTXDMASREQ	Transmit FIFO DMA Single Write Request 0 – Transmit FIFO cannot accept single data write 1 – Transmit FIFO is requesting single data write
4	SSPRXDMABREQ	Receive FIFO DMA Burst Read Request 0 – Receive FIFO does not require burst data read 1 – Receive FIFO is requesting burst data read
5	SSPRXDMASREQ	Receive FIFO DMA Single Read Request 0 – Receive FIFO does not require single data read 1 – Receive FIFO is requesting single data read
15-6	-	Reserved.

6.4 Free-running Functional Tests

The following sections describe all the free-running test cases that were run during the validation of the SSP. These tests use the Trickbox .

6.4.1 Register Tests

Reset Value Tests

After a reset, each of the following registers are read to check if they contain the specified reset value.

SSP Register Name	Reset Value
SSPCR0	0x00
SSPCR1	0x00
SSPSR	0x03
SSPCPSR	0x00
SSPIMSC	0x00
SSPRIS	0x0
SSPMIS	0x0
SSPDMACR	0x00
SSPTCR	0x0
SSPITOP	0x000
SSPPERIPHID0	0x22
SSPPERIPHID1	0x10
SSPPERIPHID2	0x04
SSPPERIPHID3	0x00
SSPPCELLID0	0x0D
SSPPCELLID1	0xF0
SSPPCELLID2	0x05
SSPPCELLID3	0x81

Table 6.4-1 Reset values of SSP registers

Read/Writeability of the functional registers.

A data pattern is written to each of the writeable registers SSPCR0 , SSPCR1, SSPCPSR, SSPIMSC and SSPDMACR.

The register contents are read back to see if it contains the written value. If the values are the same then the register is declared to be writeable & readable and the test passes.

6.4.2 Non Back-to-Back Test (Master mode)

In this test the SSP is disabled and the Trickbox enabled, 4 data words are written to the data register of the SSP and 16 data words are written to the Trickbox. The SSP status register is read to check whether the BSY bit is set. The SSP is enabled as a Master and then is allowed to transmit till all the words are completely transmitted. The data words are then read and compared. After some idle time, 3 more words are written to the SSP and transmission is allowed to complete. The data words are then read and compared. The above sequence is repeated with varying idle periods between consecutive blocks of transfers. At the end of each block transfer, the transmitted data is read from the Trickbox receive FIFO and compared with the expected values for correctness. At the end of each block of transfers SSPFSSOUT should return to the inactive state.

This test is repeated for TI Synchronous Serial and Motorola SPI (SPO = 0, SPH = 0) frame formats with different word lengths.

6.4.3 Data Setup/Hold Test (Master mode)

The purpose of this test is to check the Setup and Hold times in the non back to back mode. The Transmit FIFO of the SSP is written with data words of 0x5555/0xAAAA such that the Transmit line from the SSP toggles with every bit period. The Trickbox determines whether the Transmit data line meets setup and hold with respect to the relevant SSPCLKOUT edge. This test is repeated in all the modes with the SSP programmed to operate as a Master.

6.4.4 Transmission and Reception Test (Master mode)

In each test, first the SSP is disabled, the Trickbox enabled and 8 data words are written to the data register of both the SSP and the Trickbox. The SSP status register is read to check that SSP TX FIFO is full and the BSY bit is set. The SSP is then enabled as a Master and then allowed to run till the words are completely transmitted. At the end of transmission the SSP status register is read again to check if the TX FIFO is empty and the RX FIFO is full. The data words are then read from the SSP and Trickbox RX FIFO and compared with the transmitted values to check for error free transmission/reception. This test is repeated for all frame formats with all word lengths and different SSPCLKOUT rates and PRESCALE values. In this test set a total of 408 data words are transmitted by the SSP.

6.4.5 Continuous mode Tests (Master Mode)

In these tests the SSP is first disabled and the Trickbox is enabled. Eight data words are written to the data register of both the SSP and the Trickbox. The Trickbox is configured with RX FIFO watermark level to four . The SSP status register is read to check if the SSP TX FIFO is full and the BSY bit is set. The SSP is then enabled and allowed to transmit / receive till the RXWFLG bit in the Trickbox is set. This time four more data words are written to the data register of both the SSP and the Trickbox. Four data words are then read from the SSP and Trickbox RX FIFO to ensure that sufficient number of vacant FIFO locations are available. At the end of transmission the SSP status register is read for TX FIFO being empty. Received data is then compared with the transmitted values to check for error free transmission/reception. The difference between this test and the Transmission / Reception test is that SSP RX FIFO and TX FIFO are accessed during data transmission.

This test is repeated for all frame formats with different word length and different SSPCLKOUT rates and PRESCALE values . In this test set 72 data words are transmitted by the SSP.

6.4.6 Interrupt Tests

Receive FIFO Interrupt tests

In this test the SSP's Receive FIFO Interrupt is enabled and the Trickbox is configured with RX FIFO watermark level of four. Five words of data are written to the Trickbox and the SSP. The SSP status register is read to check if the SSP BSY bit is set. The SSP is then enabled. As soon as four data words are received by the SSP, the

Receive FIFO interrupt occurs. This is confirmed by reading the Trickbox RFSFLG. On reading two words from the SSP and the Trickbox RX FIFOs, the SSPRXINTR line goes low which in turn resets the RFSFLG. The remaining data words are then read from the SSP and the Trickbox and compared with the transmitted values to check for error free transmission/reception.

Transmit FIFO Interrupt Tests

The SSP's Transmit FIFO Interrupt is enabled. Ten words of data are written to the Trickbox and eight words of data written to the SSP. The SSP is then enabled. The contents of the transmit FIFO get transmitted by the Transmit / receive section of the SSP. As soon as the fill level in the SSP's TX FIFO falls to four data words the SSPTXINTR interrupt occurs. This is verified by reading the RXWFLG bit in the Trickbox status register. One more data element is written to the SSP and SSPTXINTR line goes to low. This is verified by reading the TIS bit of SSPIIR register.

6.4.7 FIFO pointer test (Master mode)

This test exercises the pointers in the Transmit and Receive FIFOs in the SSP. In this test the water mark level for the Receive FIFO in the Trickbox is changed from two to four, four to six and then to eight. Data is written and the status flags are checked to see that RXWFLG flag goes high depending upon the number of words transmitted and the value of the water mark level set. Data is read back from the Receive FIFOs of the SSP and the Trickbox and compared.

6.4.8 FIFO pointer cross-over Tests

These are boundary checks on the Transmit FIFO and Receive FIFO. When the FIFO pointers are both at 7 and when the FIFO is FULL, if a read and write occur simultaneously to the FIFO, the read and write should both be allowed to have effect. The test sequence is as follows. The TESTRST bit is set and the FIFO pointers are cleared. The pointers are brought to 7 by filling up all locations in the transmit FIFO. The Transmit FIFO in the trickbox is also filled with transmit data that would eventually get accumulated in the Receive FIFO of the SSP. Transmission / reception is allowed to complete and thus the read and write pointers are brought to 7 in both the FIFOs. The TXFIFO is then made full by writing 8 more words. The pointers in the transmit FIFO wrap around and return to 7 with the difference being that the FIFO is full. The SSP is enabled and when a read from the Transmit FIFO occurs, a write is initiated from the APB to the Transmit FIFO so as to exactly coincide with the read request. The write data is expected to be taken into the Transmit FIFO. Transmission and reception are allowed to occur and the received data is read back to verify correct device operation. Due to the deterministic nature of this test, it is run only when SSPCLK and PCLK are of the same frequency. A similar test is done on the Receive FIFO.

6.4.9 Receive Overrun Interrupt Tests (Master mode)

The SSP is configured as a Master with the Receive Overrun Interrupt Enabled. Twelve words of data are written to the Trickbox and eight words of data are written to the SSP. As soon as four data words are received by the Trickbox four more data words are written to the SSP. As soon as the SSP receives the ninth word the SSPRORINTR interrupt occurs. This interrupt is cleared by writing 0x0 to the SSPIIR register.

6.4.10 Scan Mode Test

This test is for validating the SSP scan test hold input.

Known data patterns are written to the writeable SSP registers. Test reset is asserted by setting and clearing the TESTRST (test reset) bit in the SSPTCR (test control register) of the SSP. After this test reset, the writeable registers should be reset to their reset values.

Then a known data pattern is written to the register again. The SCANMODE pin is driven high through the Trickbox. The test reset cycle is again conducted (that is the TESTRST bit is asserted and negated through the SSPTCR register). This time the registers should retain the known data patterns. This test verifies that the test reset is not effective when the SSP's SCANMODE input is high. This test is run only when SSPCLK and PCLK are

of the same frequency, since there are no synchronisers to the SSPCLK domain for the test reset generated by the Test logic.

6.4.11 Loop back Test

This test is for validating the SSP's loopback functionality in Master mode.

In this test the SSP is disabled and configured with loopback mode bit set. Eight data words are written to the data register of SSP. The SSP status register is read to verify that the SSP's TX FIFO is full and the BSY bit is set. The SSP is enabled as a Master and transmission is allowed to complete. The SSP's status register is read to verify that the TX FIFO is empty and the RX FIFO is full. The received data is then compared with the transmitted data. This test does not require the Trickbox to transmit.

6.4.12 SSP Disable Test (Master mode)

The purpose of this test is to check that the SSP doesn't transmit data when it is programmed to operate as a Master but is not enabled. A block of five data words is written into the TxFIFO of the SSP and some idle cycles are introduced. Then the status flags are checked to see that no transmission has taken place. The Trickbox monitors the outputs of the SSP to check that the SSP's outputs are not driven when it is disabled. Then the SSP is enabled and Transmission/reception is allowed to occur. Data is read and compared for error free transmission. This test is done in all the modes.

6.4.13 Non Back-to-Back Test (Slave mode)

In this test, first the SSP and the Trickbox are disabled, 4 data words are written to the data register of the SSP and 16 data words are written to the Trickbox. The SSP status register is read to check whether the BSY bit is set. The SSP is enabled as a Slave and the Trickbox is enabled as a Master. Data transfer is allowed to take place. The data words are then read and compared. After some idle time, 3 more words are written to the SSP and transmission is allowed to complete. The data words are then read and compared. The above sequence is repeated with varying idle periods between consecutive blocks of transfers. At the end of each block transfer, the transmitted data is read from the Trickbox receive FIFO and compared with the expected values for correctness.

This test is repeated for TI Synchronous Serial, Motorola SPI and NM frame formats with different word lengths.

6.4.14 Data Setup/Hold Test (Slave mode)

The purpose of this test is to check the Setup and Hold times in the non back to back mode. The Transmit FIFO of the SSP is written with data words of 0x5555/0xAAAA such that the Transmit line from the SSP toggles with every bit period. The Trickbox determines whether the Transmit data line meets setup and hold with respect to the relevant SSPCLKIN edge. This test is repeated in all the modes with the SSP programmed to operate as a slave.

6.4.15 Transmission and Reception Test (Slave mode)

In each test, first the SSP is disabled, the Trickbox enabled and 8 data words are written to the data register of both the SSP and the Trickbox. The SSP status register is read to check that SSP TX FIFO is full and the BSY bit is set. The SSP is then enabled as a Slave and then allowed to run till the words are completely transmitted. At the end of transmission the SSP status register is read again to check if the TX FIFO is empty and the RX FIFO is full. The data words are then read from the SSP and Trickbox RX FIFO and compared with the transmitted values to check for error free transmission/reception. This test is repeated for all frame formats with all word lengths and different SSPCLKIN rate and PRESCALE values. In this test set a total of 408 data words are transmitted by the SSP.

6.4.16 Continuous mode Tests (Slave Mode)

In these tests the SSP is first disabled and the Trickbox is enabled. Eight data words are written to the data register of both the SSP and the Trickbox. The Trickbox is configured with RX FIFO watermark level to four . The SSP status register is read to check if the SSP TX FIFO is full and the BSY bit is set. The SSP is then enabled as a Slave and allowed to transmit / receive till the RXWFLG bit in the Trickbox is set. This time four more data words are written to the data register of both the SSP and the Trickbox. Four data words are then read from the SSP and Trickbox RX FIFO to ensure that sufficient number of vacant FIFO locations are available. At the end of transmission the SSP status register is read for TX FIFO being empty. Received data is then compared with the transmitted values to check for error free transmission/reception. The difference between this test and the Transmission / Reception test is that SSP RX FIFO and TX FIFO are accessed during data transmission.

This test is repeated for all frame formats with different word length and different SSPCLKIN rate and PRESCALE values . In this test set 72 data words are transmitted by the SSP.

6.4.17 Receive Overrun Interrupt Tests (Slave mode)

These tests check the generation of the ROR interrupts from the Receive FIFO of the SSP. The values of SSPRORINTR and SSPINTR are checked through the Interrupt Identification Registers. This test is being conducted in the TI mode. The SSP is configured as a Slave with the Receive Overrun Interrupt Enabled. Twelve words of data are written to the Trickbox and eight words of data are written to the SSP. As soon as four data words are received by the Trickbox four more data words are written to the SSP. As soon as the SSP receives the ninth word the SSPRORINTR interrupt occurs. This interrupt is cleared by writing 0x0 to the SSPIIR register.

6.4.18 FIFO pointer test (Slave mode)

This test exercises the pointers in the Transmit and Receive FIFOs in the SSP. In this test the water mark level for the Receive FIFO in the Trickbox is changed from two to four, four to six and then to eight. Data is written and the status flags are checked to see that RXWFLG flag goes high depending upon the number of words transmitted and the value of the water mark level set. Data is read back from the Receive FIFOs of the SSP and the Trickbox and compared.

6.4.19 SOD Test

The purpose of this test is to set the SOD bit and check that the slave does not drive the transmit data line, but receives data from the master (trickbox). A data word is written into the Transmit FIFOs of both the Trickbox and the SSP. The SSP and the Trickbox are enabled and data interchange is allowed to occur via the serial lines with the SOD bit set. The status flags are read to check that the transmit FIFO of the SSP slave contains the data written and that the data is not transmitted to the Trickbox. Now a new data word is written into the Trickbox and the SOD bit in the SSP is cleared. Transmission/Reception is allowed to take place and the data is read and compared from the Receive FIFOs of the trickbox and the SSP. This test is repeated in all the modes.

6.4.20 SOD Continuous Test

The purpose of this test is to check that the slave does not transmit data to the master when the SOD bit is set. Four data words are written into both the Trickbox and the SSP and transmission is allowed to take place. Then status flags are read to check that the transmit FIFO of the slave contains the data written and that the data is not transmitted to the slave. Now again four data words are written into the master and the SOD bit is cleared. Transmission is allowed to take place and the data is read and compared.

6.4.21 Extended SSPFSSIN test (TI mode only)

In this test a non-zero count value is written into the ESFRM field of the SSPTBCR2 register in the Trickbox. This value specifies the number of SSPCLKIN clock cycles for which SSPFSSIN will be high for each frame of transmit

data in the TI mode. Data is written into both the trickbox and the SSP. The trickbox and the SSP are enabled. The SSP Slave is expected to sample the first bit of data on the first falling edge of SSPCLKIN that occurs after SSPFSSIN has gone low. This is checked through the trickbox, which drives invalid value on the Receive data line of the SSP during the time when the SSP should not sample the line and drives valid value only after SSPFSSIN has gone low. If the SSP slave samples the Receive line when it is being driven with an invalid value, this would appear as data mismatches when data is finally read from the SSP's Receive FIFO. The SSP's Transmit data line is checked by the trickbox to determine whether the first bit of transmit data is driven throughout the duration for which SSPFSSIN is high. Transmission / reception is allowed to complete and the data is read and compared for error free transmission.

6.4.22 SSP Disable Test (Slave mode)

The purpose of this test is to check that the SSP doesn't transmit data when it is programmed to operate as a Slave and is not enabled. A block of five data words is written into the TxFIFO of the SSP and some idle cycles are introduced. Then the status flags are checked to see that no transmission has taken place. The Trickbox monitors the outputs of the SSP to check that the SSP's outputs are not driven when it is disabled. Then the SSP is enabled and Transmission/reception is allowed to occur. Data is read and compared for error free transmission. This test is done in all the modes.

6.4.23 Free-running-SSPCLKIN Tests

The purpose of this test is to check whether the transmission or reception of data takes place correctly even when SSPCLKIN is free running i.e SSPCLKIN is active even when there is no data transfer in progress. This test verifies device operation when the data transfer is intermittent with idle times between transfers, during which SSPCLKIN is active. This is done in TI and NM modes. Here the SSP is enabled as a Slave.

6.4.24 Master Slave Test

In this test the functionality of the SSP has been changed from Master to Slave and then to Master. Initially when the SSP is in Master mode two data words are written and transmitted. The two words are then read and compared for error free transmission. Then the SSP and the Trickbox are disabled. Now, the SSP is enabled as a Slave and the data is written and read. Again after disabling, the SSP is enabled as a Master and data is written and read. This is done in all modes.

6.4.25 Master Slave SOD Test

In this test the functionality of the SSP has been changed from Master to Slave with SOD set and then to Master. Initially when the SSP is in Master mode two data words are written and transmitted. The two words are then read and compared for error free transmission. Then the SSP and the Trickbox are disabled. Now, the SSP is enabled as Slave with SOD set and the data is written and read. Again after disabling, the SSP is enabled as a Master and data is written and read.

6.4.26 Frame Deactivation Test

In the SPI mode, when SPH is set, the SSP slave is expected to be compatible with two kinds of Masters - those that negate SSPFSSIN for one SSPCLKIN phase time between successive frames of continuous transmission and those Masters that don't. This test verifies the SSP slave's operation when interfacing with those Masters that do negate SSPFSSIN between successive frames in a continuous transfer. The SPISFRMEN bit in SSPTBCR2 register of the Trickbox is set to force the Trickbox to negate SSPFSSIN between successive frames. Then the SPI Back-to-back tests are executed for SPO = 0 and SPO = 1. The SSP slave's operation when interfacing with SPI Masters that do not negate SSPFSSIN between successive frames in a continuous transfer, is verified as part of the default SSP slave Back-to-back tests.

6.4.27 SSPFSSIN negation Tests

This test verifies that the SSP slave abandons transmission and reception of a frame if SSPFSSIN is negated (in the NM and SPI modes) or if SSPFSSIN is asserted (in TI mode), midway through a frame thereby indicating the end of the current frame (and the start of a new frame in TI mode). In this test the master and the slave are programmed for different word lengths. The Master is programmed for a lesser word length than the Slave. Data is written into both the SSP and the Trickbox. The purpose of this test is to check that the Slave aborts the first data as it is expecting a greater word length than that transmitted by the master. This is done in all modes with the SSP as a slave.

6.4.28 Data Discard Tests

This test verifies that the SSP slave abandons transmission and reception of a frame if the SSP is disabled midway through a frame. In this test one word is written into both the SSP and the Trickbox. Both the master and the slave are disabled during the transmission of the data byte. The status flags are then checked to see that the data byte is discarded. This is done in all the modes with the SSP as a slave.

6.4.29 FRF change Tests

This test verifies that changes in the Frame format are handled correctly by the SSP. In this test the frame format is switched from TI to NM mode and then the Data test is executed to verify that data transfers occur after the mode switch. Similarly, mode switching is done from NM to TI and TI to SPI00.

6.4.30 Receive Timeout interrupt tests

These tests verify that the receive timeout interrupt is asserted when the receive FIFO contains data and there has been no activity on the SSPCLKOUT signal when configured in master mode or SSPCLKIN when configured in slave mode. A single word is transferred from master to slave followed by an idle period of greater than 32 bit periods. The Receive Timeout interrupt is checked for assertion. Another word is then written to the Tx FIFOs of master and slave and transfer allowed to complete. The Receive Timeout remains asserted even though new data has been received. The masking, clearing and idle features are subsequently tested. Tests are performed with the SSP configured as master and slave in the SPI, TI and MicroWire formats.

6.4.31 DMA Interface Transmit tests

The SSP and Trickbox are configured with the FIFO's, TestFifo mode and transmit DMA enabled.

The single and burst transmit requests, SSPTXDMASREQ and SSPTXDMABREQ, are checked as being asserted, signifying that there is space in the SSP Tx FIFO for single characters and a burst.

Seven words are written to the SSP Tx FIFO, in one burst length of four and three single transfers. The requests are cleared by writing a "1" to bit 0 of the SSPTBDMACR register during the transfer of the last data in the burst and after each single write. While there is still at least one burst length space in the SSP Tx FIFO, both requests are asserted. When there is less than a burst length of space in the SSP Tx FIFO, just the single request is asserted.

One word is successively read from the SSP Tx FIFO, whilst the burst request is checked to be asserted after three words have been read out.

The Transmit DMA is disabled and the request checked that it has become de-asserted. The transmit DMA is re-enabled and the request is checked that it has become re-asserted.

The remaining words are read out of the SSP Tx FIFO and the single and burst requests are checked for assertion.

The SSP, Trickbox, Transmit DMA and TestFifo mode are then disabled.

6.4.32 DMA Interface Receive tests

The SSP and Trickbox are configured with the FIFO's, TestFifo mode and receive DMA enabled.

The single and burst transmit requests, SSPRXDMASREQ and SSPRXDMABREQ, are checked for de-assertion, because the FIFO's are currently empty.

Seven words are written in test mode to the SSP Rx FIFO. When there are less than four words in the Rx FIFO, only the single request is set, and that when there are more than 4 words in the Rx FIFO both the single and burst receive requests are set.

The DMA controller would service the burst requests. The data is read out of the Rx FIFO with a single burst of 4 words. The request is cleared by writing a "1" to bit 1 of the SSPTBDMACR register during the transfer of the last data in the burst. The remaining words are read as single characters, clearing the request after each transfer.

Four words are written into the RXFIFO and the receive single and burst request are checked for assertion. The receive DMA is disabled the requests are checked to be de-asserted. The receive DMA is re-enabled and the requests are checked to be re-asserted. The remaining data is read out of the Rx FIFO.

The SSP, Trickbox, Receive DMA and TestFifo mode are then disabled.

6.4.33 Additional Code Coverage Tests

These tests were added to increase the code coverage figures.

Mode Change tests

The SSP is sequenced from format to format such that all modes are entered and a single dataword is transmitted/received whilst in each mode. On return to the idle stage in each case, the frame format is changed and the process repeated.

Disabling the SSP PL022 whilst it is still transmitting and/or receiving

These tests improve coverage of the transmit and receive state machines. They are focused on disabling the SSP at particular stages of the transmit and receive bit sequences.

7 SSP INTEGRATION TEST DETAILS

The SSP integration test vector suite allows the integrator to verify that the SSP has been connected into the target system correctly. These vectors test 3 classes of signals:

a) AMBA signals (SSP signals connected directly to the APB)

b) Intra-Chip signals (SSP signals connected to the DMA and Interrupt Controllers)

c) Primary I/O signals (SSP signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 6].

The SSP Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the SSP has been connected into the target system correctly. The VHDL and Verilog TICTalk testbench files reside in the ***integration/vhdl*** and ***integration/verilog*** directories respectively. The test vectors that are applied to the TICTalk testbenches are in the ***integration/bustest*** directory. The test vectors for integration testing are written in BusTalk and then converted into **.tif** format to be applied to the TICTalk integration testbench.

7.1 SSP VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the SSP is located in the ***integration/vhdl/tbench*** directory.

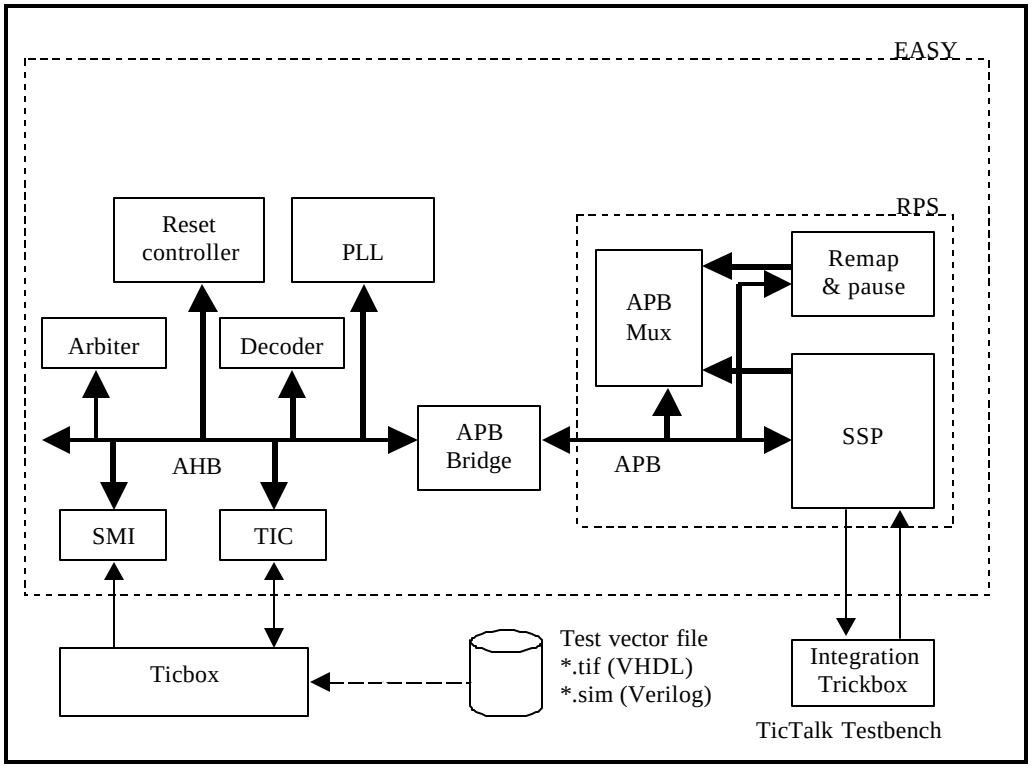

Figure 7.1-1 Testbench for simulating TICTalk test vectors

The Integration test vectors are located in the ***integration/bustest*** directory.

The test vectors used with this testbench can be applied to the SSP when it is part of an AMBA system design.

Figure 7.1-1 illustrates the testbench to apply TICTalk test vectors.

The SSP VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.1-2 illustrates the hierarchy for the TICTalk testbench.

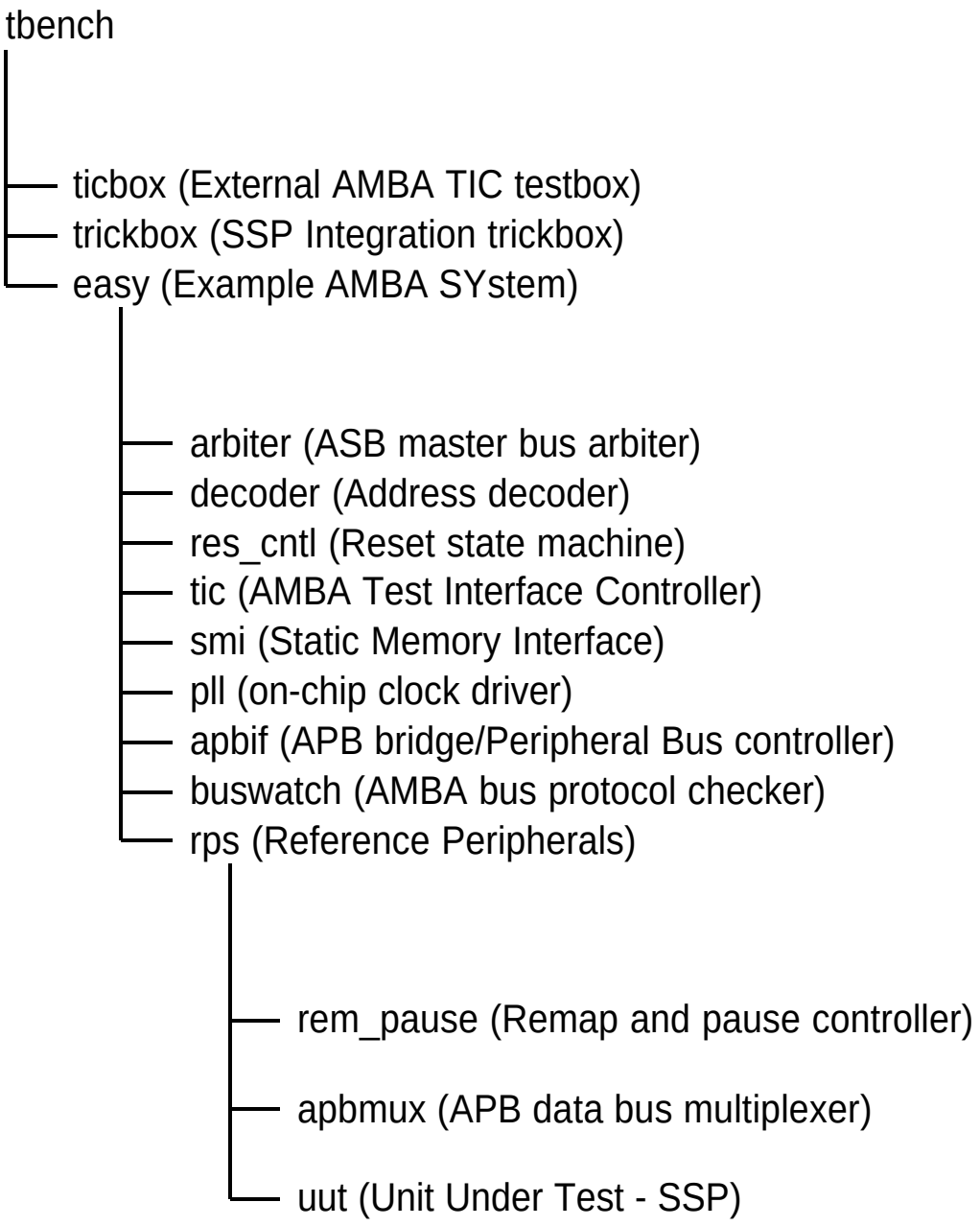

Figure 2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the testbench that is used for running the integration tests of the SSP. This module instantiates the model of the EASY microcontroller, the integration trickbox and the

ticbox. The SSP is instantiated in the rps.vhd, which is a part of the EASY system. The SSPCLK is generated in the rps.vhd. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test I/O pins.

7.1.1 Unit Under Test

The Unit Under Test may be one of the following

- SSP VHDL RTL
- SSP VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in ***integration/vhdl*** .

7.1.2 Test Vectors

The test vectors for the integration testing of the SSP are coded into separate C files for testing the reset values of all the SSP registers and for Integration testing. Make options are provided to run the tests together or individually.

7.1.3 Running Simulations

The **README** file in the ***integration/vhdl/tbench*** directory gives step by step instructions for running simulations using the SSP Integration test vectors on the VHDL source code.

Run time logs are available in the following files:

integration/vhdl/Ssp_rtl/Ssp_rtl_modelsim.log

integration/vhdl/Ssp_scanready_vhdl_net_max/Ssp_scanready_vhdl_net_max_modelsim.log

7.2 SSP Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the SSP is located in the ***integration/verilog/tbench*** directory.

The Integration test vectors are located in the ***integration/bustest/invec*** directory.

The test vectors used with this testbench can be applied to the SSP when it is part of an AMBA system design.

Figure 7.2-1 illustrates the testbench to apply TICTalk test vectors.

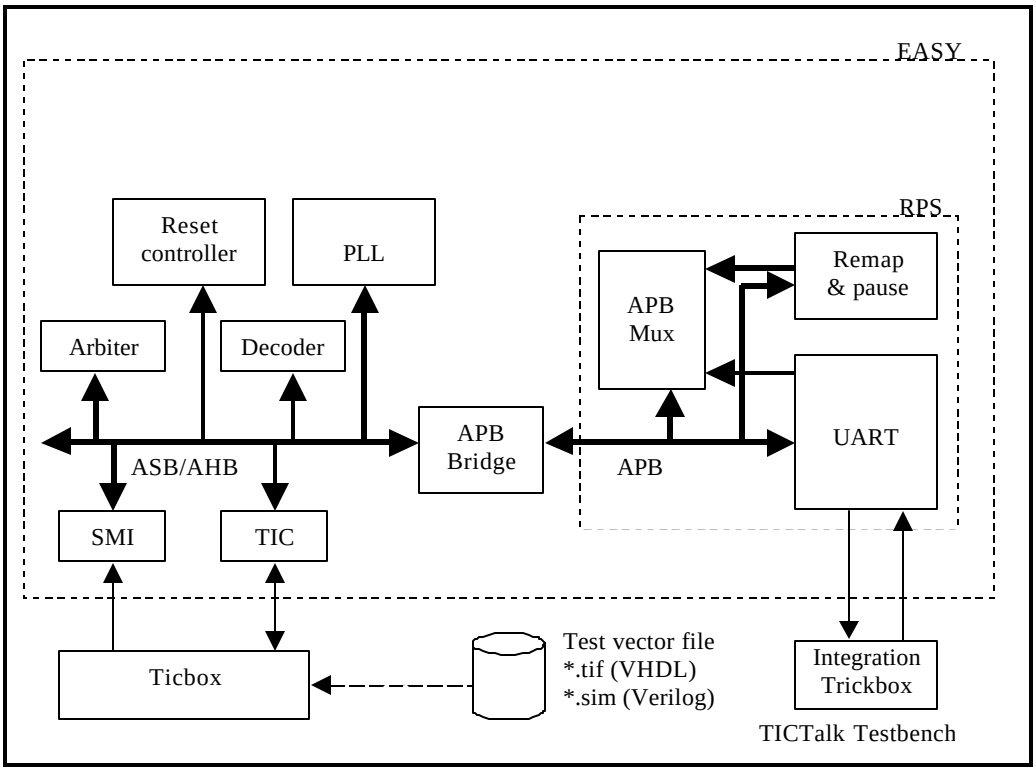

Figure 7.2-1 Testbench for simulating TICTalk test vectors

The SSP Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7.2-2 illustrates the hierarchy for the TICTalk testbench.

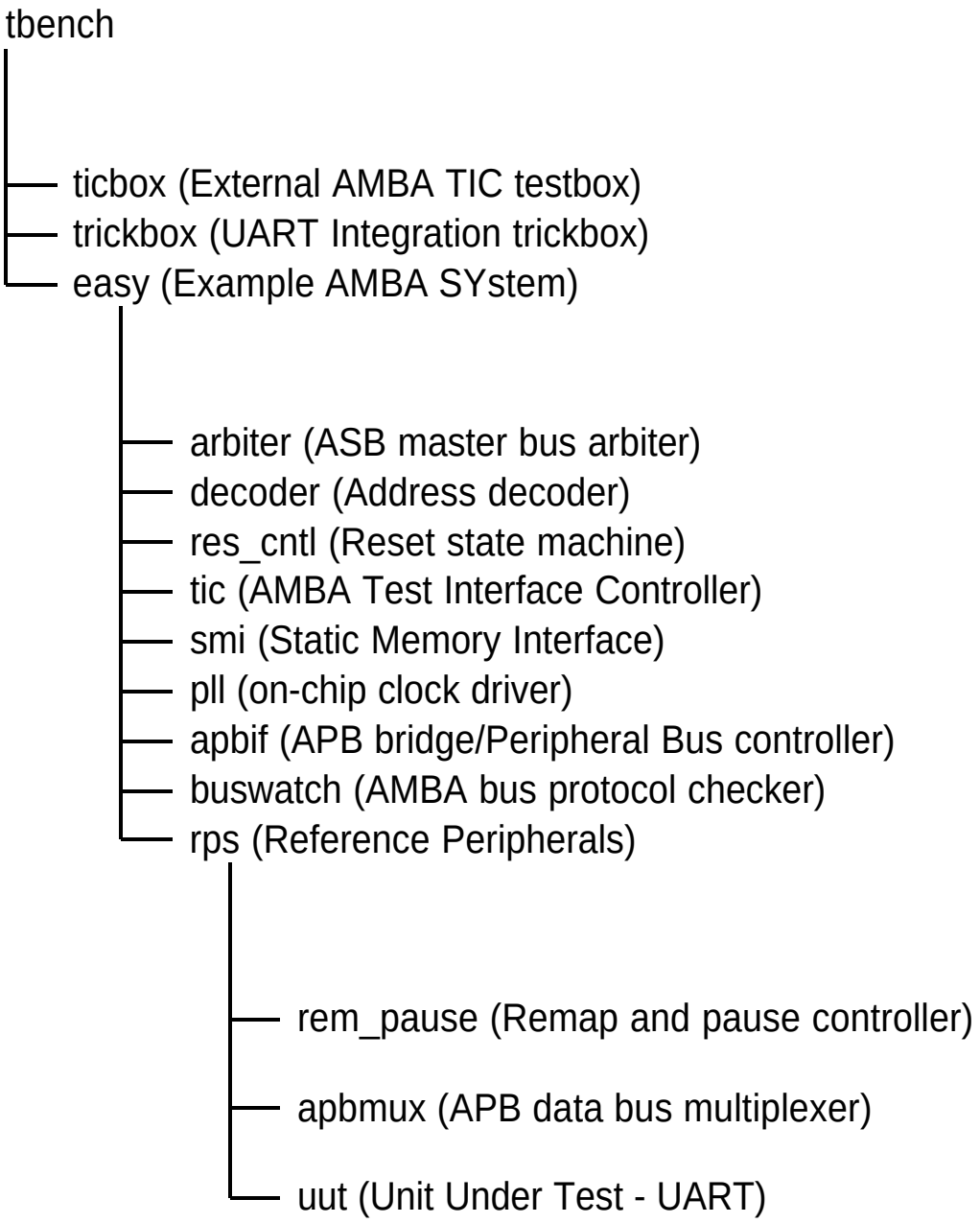

Figure 7.2-2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.v**, the top-level of the testbench that is used for running the integration tests of the SSP. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The SSP is instantiated in the rps.v, which is a part of the EASY system. The SSPCLK is generated in the rps.v. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test I/O pins.

7.2.1 Unit Under Test

The Unit Under Test may be one of the following

- SSP VERILOG RTL
- SSP VERILOG netlist from VHDL RTL Source
- SSP VERILOG netlist from VERILOG RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in ***integration/verilog*** .

7.2.2 Test Vectors

The test vectors for the integration testing of the SSP are coded into a separate C files for testing the reset values of all the SSP registers and for Integration testing. Make options are provided to run the tests together or individually.

7.2.3 Running Simulations

The **README** file in the ***integration/verilog/tbench*** directory gives step by step instructions for running simulations using the SSP Integration test vectors on the VERILOG source code.

Run time logs are available in the following files:

integration/verilog/Ssp_rtl/Ssp_rtl_LDV.log

integration/verilog/Ssp_rtl/Ssp_rtl_modelsim.log

integration/verilog/Ssp_noscan_vhdl_net_min/Ssp_noscan_vhdl_net_min_modelsim.log

integration/verilog/Ssp_scanready_verilog_net_typ/Ssp_scanready_verilog_net_typ_LDV.log

7.3 Integration Tests

7.3.1 Integration Testing of Intra-Chip and Primary Inputs

Please refer back to Section 5.20.1 on Page 57 as it covers the test logic and test sequences.

7.3.2 Integration Testing of Intra-Chip and Primary Outputs

Please refer back to Section 5.20.2 on Page 58 as it covers the test logic and test sequences.